

Componentes/Interface/STIPayment.prw

2749 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "Protheus.ch"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "FWEVENTVIEWCONSTS.CH"
4 #INCLUDE "FWADAPTEREAI.CH"
5 #INCLUDE "FWMVCDEF.CH"
6 #INCLUDE "POSCSS.CH"
7 #INCLUDE "STIPAYMENT.CH"
8 #INCLUDE "STPOS.CH"
9
10 #DEFINE SE1SALDO      6    // Posicao do campo El_SALDO do Array aNCCs
11
12 Static nUsedCards    := 0                // Numero de cartoes usados na venda Homologacao TEF
13 Static oPay          := STBWPayment():STBWPayment()
14 Static aPaym        := STIGetSx5()[1]
15 Static aCopyPaym     := STIGetSx5()[2]
16 Static aNCCPay       := {}
17 Static oModel        := Nil
18 Static oListTpPaym   := Nil
19 Static oListResPg    := Nil
20 Static nAjustCols    := 0                //Largura do listBox de resumo de pagamento para ajuste das colunas
21 Static oLblGroup1    := Nil
22 Static lListWhen     := .T.              //Indica se o listBox de selecao de pagamento é editavel
23 Static oPnlAdconal   := Nil              //Objeto do painel adicional
24 Static oPanBkpPay    := Nil              //Backup do objeto oPanPayment
25 Static lDisableBtn   := .T.              //Desabilita/Habilita os botoes de Limpar e Finalizar pagamentos
26 Static bDataCheck   := Nil
27 Static lIsPAF        := STBIsPAF()
28 Static lHomolPaf     := ExistFunc("STBHomolPaf") .AND. STBHomolPaf() //Homologação PAF-ECF
29 Static lPayImport    := .F.              //Pagamento no momento da importacao do orcamento
30 Static aCallADM      := {}              // Armazena formas de pagamento para informar Adm. Fin.
31 Static aCheckRms     := {}              //Valores pagos em cheque para RMS
32 Static aConvRms      := {}              //Valores pagos em convenio para RMS
33 Static nPayROnly     := 0                //indica se os campos de pagamentos serão Somente Leitura
34 Static oPPanel       := Nil              //Paint Painel para pagamentos modo scroll
35 Static oPanPayment   := Nil
36 Static lSTIAtvTefP   := NIL
37 Static lMFE          := IIF( ExistFunc("LjUsaMfe"), LjUsaMfe(), .F. )          //Se utiliza MFE
38 Static cPPgtSel      := ""                // última forma de pagamento selecionada
39 Static aFormasORC    := {}
40 Static lAtivMotor     := ExistFunc("PshAtivMTP") .AND. PshAtivMTP() // Verifica se o motor de promoções esta ativo
41 Static cMV_LJPGTRO    := AllTrim(SuperGetMV("MV_LJPGTRO",,Iif( cPaisLoc <> "BRA","", "R$")))
42 Static oLblResPay     := Nil //tSay Resumo de Pagamento
43 Static oLblForma      := Nil //tSay forma
44 Static oLblValor      := Nil //tsay valor
45 Static oLblParcelas   := Nil //tsay parcelas
46 Static oLblRest       := Nil //tsay Restante
47 Static oLblTroco      := Nil //tsay Troco
48 Static oButton        := Nil //tbutton confirmar pagamento
49 Static oBtnLimpar     := Nil //tbutton limpar pagamento
50 Static oLblValRest     := Nil //tsay Valor restante
51 Static oLblValTroco   := Nil //tsay valor troco
52 Static aFormMultNeg   := {} //Forma de pagamento contida na multinegociacao

```

[illegible]

```

105 Local cColorSelec := "" //cor Selecao
106 Local cImage := "" // Nome da Imagem
107 Local cTextPgto := "" // Descricao do Pagamento
108 Local aButtonText := {} // Array com botoes de texto para crair os shapes
109 Local aTextPgto := {} // vetor com o texto dos botoes criados
110 Local nLastShape := -1 // Ultimo shape antes da nova seleção
111 Local cButtonText := "" // Texto botao
112 Local cPdvColor := SuperGetMv( "MV_LJCOLOR", "07334C") // Guarda Cor da tela do PDV
113 Local aPdvColors := {} //Paleta de cores do PDV
114 Local IsFly01 := FindFunction('ISFly01') .AND. ISFly01() //Verifica se é Fly01
115 Local oScroll := Nil //Obj scroll
116 Local lFirstTime := .T. // Valida a exibição da tela de CC/CD para criar somente uma tela
117 Local lTotDA1 := ExistFunc('STDGetDA1') //Verifica se existe a funcao STDGetDA1
118 Local nTotNCC := STDGetNCCs("2") //Total de NCC
119 Local lImport := !Empty(STDGPBasket("SL1" , "L1_NUMORIG")) // Variavel de controle que identifica se é importação de orçamento
120 Local lBonificacao := .F. //Identifica se a venda tera alguma bonificacao ou nao
121 Local lRegDesc := SuperGetMV('MV_LJRGDES',,,.F.) .AND. ExistFunc("STBFORMBF") .AND. ExistFunc("STIDSCBONF") .AND. GetAPOInfo("STFTOTALUPDATE.PRW")[4] >=
    Ctod("19/12/2019") //Regra de desconto
122 Local oTotal := STFGGetTot() //Recebe o Objeto totalizador
123 Local lIsOrc := STBIsImpOrc() // -- Indica se é importação de orçamento
124 Local bZeraPay := { || Iif(Empty(oModel), Nil, STIZeraPay(Iif(lIsOrc,.T.,.F.), Iif(lIsOrc,.F.,.T.))) } // -- Zera os pagamentos baseados na Permissão de usuario + a
    variavel lIsOrc
125 Local bLDBLClick := {||lListWhen := .F., lDisableBtn := .F.,STIRestVlr(), If( STICallSFP() .AND. !Empty(oModel) .AND. (Empty(STIGetPgOr()) .Or. STFPROFILE(41)[1]),
    STIAddNewPan(oPanPayment, oListTpPaym), .F. )}
126 Local lPERestFPgt := ExistBlock('STRestFPgt')
127
128 Default lAutom := .T. // Inclui a forma de pagamento automática
129
130 LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "Monta a interface de pagamento" ) //Gera LOG
131
132 oPanPayment := TPanel():New(00,00,"",oPanelMVC,,,,,oPanelMVC:nWidth/2,oPanelMVC:nHeight/2) //Painel do pagamento
133 nAltura := (oPanPayment:nHeight / 2) * 0.20 //Altura
134 nCol := (oPanPayment:nWidth / 2) * 0.03 //Coordenada horizontal
135 nLargura := (oPanPayment:nWidth / 2) - (2 * nCol) //Largura
136
137 nPosHozPan := oPanelMVC:nWidth/76.2 //Posicao horizontal do painel
138 nPosAltPan := oPanPayment:nHeight/4.807 //Posicao: Altura do Painel
139 nTamLagPan := oPanPayment:nWidth/2.09836 //Tamanho: Largura do Painel
140 nTamAltPan := oPanPayment:nHeight/3.4512 //Tamanho: Altura do Painel
141 bCreatePan := {||TPanel():New(nPosAltPan,nPosHozPan,"",oPanPayment;
142     ,,,,,nTamLagPan,nTamAltPan)} // Bloco de codigo para criacao do Painel adicional
143
144 IF lPERestFPgt//Se usa PE STRestFPgt precisa recarregar as forma de pagamento toda vez q for finalizar a venda
145     aPaym := STIGetSx5()[1]
146     aCopyPaym := STIGetSx5()[2]
147 Endif
148 /*
149 Se for orçamento importado e o usuário não tem permissão para Alterar Parcelas,
150 o usuario nao pode editar os campos Data/Valor e Parcela das formas de pagamento
151 */
152 If nPayROnly == 0
153     If !Empty(cNumOrig) .AND. !STFProfile(9)[1]

```

```

154 STISetPayR0(1) //campos somente leitura
155 Else
156 STISetPayR0(-1) //campos com escrita
157 EndIf
158 EndIf
159
160 /* Limpa as mensagens das etapas anteriores ao pagamento */
161 STFCleanInterfaceMessage()
162
163 nAjustCols := nLargura
164 oPanBkpPay := oPanPayment
165 bDataCheck := {|TPanel():New(00,00,"",oPanelMVC,,,,,oPanelMVC:nWidth/2,oPanelMVC:nHeight/2)}
166
167 /* Label: Pagamento */
168 oLbLVendPay := TSay():New(POSVERT_CAB, POSHOR_1, {|STR0006}, oPanPayment,,,,,.T.,,nLargura,11.5) //'Pagamento'
169 oLbLVendPay:SetCSS( POSCSS (GetClassName(oLbLVendPay), CSS_BREADCUMB ))
170
171 /* Label: Selecione a forma de pagamento */
172 oLbLTpPaym := TSay():New(POSVERT_LABEL1, POSHOR_1, {|STR0002}, oPanPayment,,,,,.T.,,nLargura,8) //'Selecione a forma de pagamento'
173 oLbLTpPaym:SetCSS( POSCSS (GetClassName(oLbLTpPaym), CSS_LABEL_FOCAL ))
174
175 lMobile := ValType(lMobile) == "L" .AND. lMobile
176
177 //Não sendo versao mobile exibe um listbox de pagamentos
178 If !lMobile
179
180 //-----
181 // ListBox dos tipos de pagamentos
182 //-----
183 oListTpPaym := TListBox():Create(oPanPayment, POSVERT_GET1, POSHOR_1, Nil, aPaym, nLargura, nTamAltListBox,,,,;
184 .T.,,bLDBLClick,,,{|lListWhen == .T.})
185
186 oListTpPaym:SetCSS( POSCSS (GetClassName(oListTpPaym), CSS_LISTBOX ))
187 //Verifico se tem permissão para alterar a forma de pagamento
188 If lIsOrc .And. STFGetCfg("lChangePay", .F.)
189 oListTpPaym:lReadOnly := !STFPROFILE(41)[1]
190 Else
191 oListTpPaym:lReadOnly := STIGetPayR0()
192 EndIf
193 Else
194
195 //-----
196 // PaineL de pagamentos para versão Mobile
197 //-----
198
199 //Para a Versão Fly os valores das cores sao fixos, para o PDV Movel Totvs de acordo com parametro
200 aPdvColors := IIF( IsFly01, {'FF9300','F37021','F16E01','FF8000','FFA200'}, FWGetMonocromatic(cPdvColor) )
201
202 cColor := "1,0,0,0,0,0.0,#FFFFFF" //Cor Padrão - Branco
203 cColorHover := "1,0,0,0,0,0.0,#" + AllTrim(aPdvColors[5]) //Cor hover
204 cColorSelec := "1,0,0,0,0,0.0,#" + AllTrim(aPdvColors[5]) //cor Selecao
205

```

```

206 oScroll := TScrollArea():New(oPanPayment, POSVERT_LABEL1, POSHOR_1, 65, nLargura)
207
208 // lTracking permite o arrasto atraves dos itens do TPaintPanel
209 oScroll:lTracking := .T.
210 oScroll:setCss(POSCSS( GetClassName(oScroll), CSS_CHOOSE_PAYMENT) )
211
212 // Painele que ira conter os itens
213 oPPanel := TPaintPanel():new(0,0,nLargura,62,oScroll)
214 oPPanel:SetReleaseButton (.T.)
215 oPPanel:SetCss(POSCSS( GetClassName(oScroll), CSS_PAYMENT_ITEM))
216 oScroll:SetFrame( oPPanel )
217
218 For nI := 1 to 3
219
220     nId++
221     cButtonText := ""
222     // Insere botao
223     oPPanel:addShape( "id="+cValToChar(nId)+;
224                      ";type=1;left="+cValToChar(nLeft)+;
225                      ";top="+cValToChar(nTop)+";"+;
226                      "width="+cValToChar(nShapeDim)+;
227                      ";height="+cValToChar(nHeight)+";"+;
228                      "gradient="+cColor+;
229                      ";gradient-hover="+cColorHover+;
230                      ";pen-width=0;"+;
231                      "pen-width=1;"+;
232                      "can-move=0;")
233
234     If nI == 1
235         cImage := "dinheiro"
236         cTextPgto := "DINHEIRO"
237     ElseIf nI == 2
238         cImage := "cartao"
239         cTextPgto := "CARTÃO"
240     ElseIf nI == 3
241         cImage := "cheque"
242         cTextPgto := "CHEQUE"
243     EndIf
244
245     aAdd(aTextPgto, cTextPgto)
246
247 //Imagem
248
249     oPPanel:addShape( "id="+cValToChar(nId)+ ";" +;
250                      "subID=1;" +;
251                      "type=8;" +;
252                      "left="+cValToChar(nLeft)+ ";" +;
253                      "top="+cValToChar(nTop-10)+";"+;
254                      "width="+cValToChar(nShapeDim) + ";" +;
255                      "height="+cValToChar(nHeight)+ ";" +;
256                      "image-file=rpo:"+cImage+".png;" +;
257                      "can-move=0;"+;

```

```

258             "gradient="+cColor+";" + ;
259             "gradient-hover="+cColorHover+";")
260
261         //Texto
262
263         aadd( aButtonText,;
264             "id="+cValToChar(nId)+;
265             ";subID=2;type=7;pen-width=1;font=arial,14,0,0,3;can-move=0;"+;
266             "left="+cValToChar(nLeft)+;
267             ";top="+cValToChar(90)+";"+;
268             "width="+cValToChar(nShapeDim)+;
269             ";height="+cValToChar(nHeight)+";"+;
270             "text="+cTextPgto+";"+;
271             "pen-color=@@COLOR@@;"+;
272             "gradient="+cColor+";" )
273
274         cButtonText := strTran( aButtonText[nI] , "@@COLOR@" , "#000000" )    // Substitui cor (black) "#000000" "#757575"
275         oPPanel:addShape( cButtonText )
276
277         nLeft += (nShapeDim)
278
279     Next nI
280
281     // Define tamanho final do painel dos botoes
282     oPPanel:nWidth := nLeft
283     // Acao do botao
284     oPPanel:blClicked := { |x,y| adjShape(      oPPanel      , nId      , cColor      , cColorHover ,;
285                                             @nLastShape ,aButtonText , aButtonText , oPanPayment ) ,;
286                                             oPPanel:SetGradient(oPPanel:ShapeAtu,.T.,cColorSelec);
287                                             oPPanel:SetGradient(oPPanel:ShapeAtu,.F.,cColorSelec)}
288
289 EndIf
290
291 /* Label: Resumo do pagamento */
292 oLblResPay := TSay():New(oPanelMVC:nHeight/4.2566, POSHOR_1, {||STR0013}, oPanPayment,,,,,.T.,,nLargura,8) //'Resumo do pagamento'
293 oLblResPay:SetCSS( POSCSS (GetClassName(oLblResPay), CSS_LABEL_FOCAL ))
294
295 /* Label: Forma */
296 oLblForma := TSay():New(oPanelMVC:nHeight/3.7966, POSHOR_1, {||STR0014}, oPanPayment,,,,,.T.,,nLargura,8) //'Forma'
297 oLblForma:SetCSS( POSCSS (GetClassName(oLblForma), CSS_LABEL_FOCAL ))
298
299 /* Label: Valor */
300 oLblValor := TSay():New(oPanelMVC:nHeight/3.7966, POSHOR_1 * 7.3, {||STR0015}, oPanPayment,,,,,.T.,,nLargura,8) //'Valor'
301 oLblValor:SetCSS( POSCSS (GetClassName(oLblValor), CSS_LABEL_FOCAL ))
302
303 /* Label: Parcelas */
304 oLblParcelas := TSay():New(oPanelMVC:nHeight/3.7966, POSHOR_1 * 12.3, {||STR0016}, oPanPayment,,,,,.T.,,nLargura,8) //'Parcelas'
305 oLblParcelas:SetCSS( POSCSS (GetClassName(oLblParcelas), CSS_LABEL_FOCAL ))
306
307 /* Label: Vale Presente Restante */
308 oLblRestVP := TSay():New(oPanelMVC:nHeight/4.0966, POSHOR_1 * 17.4, {||STR0027}, oPanPayment,,,,,.T.,,nLargura,8) //'Saldo Vale Presente'
309 oLblRestVP:SetCSS( POSCSS (GetClassName(oLblRestVP),CSS_BREADCUMB ))

```

```

310
311 /* Label: Restante */
312 oLblRest := TSay():New(oPanelMVC:nHeight/3.2966, POSHOR_1 * 18.9, {||STR0028}, oPanPayment,,,,,.T.,,nLargura,9) //'Saldo a Pagar'
313 oLblRest:SetCSS( POSCSS (GetClassName(oLblRest),CSS_BREADCUMB ))
314
315 /* Label: Troco */
316 oLblTroco := TSay():New(oPanelMVC:nHeight/2.7966, POSHOR_1 * 21.1, {||STR0018}, oPanPayment,,,,,.T.,,nLargura,8) //'Troco'
317 oLblTroco:SetCSS( POSCSS (GetClassName(oLblTroco),CSS_BREADCUMB ))
318
319 /* ListBox dos resumos de pagamentos */
320 oListResPg := TListBox():Create(oPanPayment, oPanelMVC:nHeight/3.45882 /*POSVERT_GET5*/, POSHOR_1, Nil, {}, nLargura * 0.65, nTamAltListBox,,,,.T.,,{||.T.},,{||.F.})
321 oListResPg:SetCSS( POSCSS (GetClassName(oListResPg),CSS_LISTBOX ))
322
323 /* Button para finalizacao do pagamento */
324 oButton := TButton():New(POSVERT_BTNFOCAL,POSHOR_BTNFOCAL,STR0019 + IIF(lMobile,"",CRLF+"(CTRL+P)") ,oPanPayment,{ || STIConfPay("Click") }, ; //"Finalizar pagamento"
325     LARGBTN,ALTURABTN,,,,.T.,,{||lDisableBtn})
326 oButton:SetCSS( POSCSS (GetClassName(oButton), CSS_BTN_FOCAL ))
327
328 /* Limpar pagamento */
329 oBtnLimpar := TButton():New(POSVERT_BTNFOCAL,POSHOR_1,STR0020 + IIF(lMobile,"",CRLF+"(CTRL+L)") ,oPanPayment,bZeraPay, LARGBTN,ALTURABTN,,,,.T.,,{||lDisableBtn}) //"Limpar
    Pagto(s)." e checa se existe Brinde ou Bonificacao já concedidos
330 oBtnLimpar:SetCSS( POSCSS (GetClassName(oBtnLimpar), CSS_BTN_ATIVO ))
331
332 SetKey(6, {|| "" }) //Desabilita CTRL + F neste momento
333 SetKey(16, {|| STIConfPay("Atalho")})
334 SetKey(12, bZeraPay)
335
336 /*Chamada da Regra da Promocao Desconto por Desconto*/
337 If lAutom
338     STWRuleTotal()
339 Endif
340
341 /* Model Payment */
342 If ValType(oModel) == "U"
343     ModelPayme()
344 EndIf
345
346 /* Ativacao do model Parcelas */
347 oMdlParc := oModel:GetModel("PARCELAS")
348 oMdlParc:Activate()
349
350 /* Ativacao do model aPayments */
351 oMdlPaym := oModel:GetModel("APAYMENTS")
352 oMdlPaym:Activate()
353
354 If lAutom .And. !STIBlqMnTef()
355     STILmpTef(oMdlParc,oMdlPaym)
356 Else
357     oMdlPaym:ClearData()
358     oMdlPaym:InitLine()
359     oMdlParc:ClearData()
360     oMdlParc:InitLine()

```

```

361 EndIf
362
363 /* Label: 0.00 Saldo Vale Presente*/
364 oLbLValVPRest := TSay():New(oPanelMVC:nHeight/3.7666, POSHOR_1 * 20.9,, oPanPayment,,,,,.T,,,nLargura,8)
365 oLbLValVPRest:SetCSS( POSCSS (GetClassName(oLbLValVPRest), CSS_LABEL_FOCAL ))
366
367 /* Label: 0.00 */
368 oLbLValRest := TSay():New(oPanelMVC:nHeight/3.0666, POSHOR_1 * 20.9, {||Str(STBCalcSaldo("1"),10,2)}, oPanPayment,,,,,.T,,,nLargura,8)
369 oLbLValRest:SetCSS( POSCSS (GetClassName(oLbLValRest), CSS_LABEL_FOCAL ))
370
371 /* Label: 0.00 */
372 oLbLValTroco := TSay():New(oPanelMVC:nHeight/2.6386, POSHOR_1 * 20.9, {||Str(STBGetTroco(),10,2)}, oPanPayment,,,,,.T,,,nLargura,8)
373 oLbLValTroco:SetCSS( POSCSS (GetClassName(oLbLValTroco), CSS_LABEL_FOCAL ))
374
375 STISetaCallADM({}) //Limpa Array Adm. Fin.
376 If ExistFunc("STISetnCards")
377     STISetnCards(1) //Configura o array de Contador de Cartões
378 EndIf
379
380 If lRegDesc
381     nVlBonif := oTotal:GetValue("L1_BONIF")
382     If nVlBonif > 0
383         lBonificacao := .T.
384         STIAddPay("BF", Nil, 1, Nil, Nil, nVlBonif)
385     EndIf
386 EndIf
387
388 /* Inclui o primeiro pagamento com base no parametro */
389 If lAutom .AND. !lMobile .AND.;
390     !( lImport .AND. ExistFunc("STBGetDiscTotPDV") .AND. STBGetDiscTotPDV() ) .AND. !lBonificacao
391     aResPay := STBUdpResPg(nAdjustCols,.T.,nVlBonif,oListTpPaym)
392     oListResPg:Reset()
393     oListResPg:SetArray(aResPay)
394 Endif
395
396 /* Verifica se foi concedido desconto do Total pelo PDV */
397 If lImport .AND. ExistFunc("STBGetDiscTotPDV") .AND. STBGetDiscTotPDV()
398     STISetaCallADM( {} )
399 Else
400     aCallAdm := STIGetaCallADM()
401 EndIf
402
403 /* Tratamento para venda com cartão na tela de multinegociacao */
404 If !lAutom .And. IIf(ExistFunc("STIFMultNeg"),STIFMultNeg(),.F.)
405
406     // Verifica forma de pagamento da multinegociacao
407     aFormMultNeg := IIf(ExistFunc("STITEFMultNeg"),STITEFMultNeg(),.F.)
408
409     For nI := 1 To Len(aFormMultNeg)
410         // Se forma de pagamento da multinegociacao conter CC ou CD abre a tela do cartão
411         If (Len(aFormMultNeg[nI]) > 3 .And. !aFormMultNeg[nI][04] .And. aFormMultNeg[nI][01] $ "CC|CD") .Or. (aFormMultNeg[nI][01] $ "CC|CD")
412             oPnlAdconal := STICriaPnlAdconal(bCreatePan)

```

```

413         /* Tela do cartao */
414         STIPayCard(oPnlAdconal, aFormMultNeg[nI][01], aFormMultNeg[nI][02], aFormMultNeg[nI][03] )
415     EndIf
416     Next
417 EndIf
418
419 For nI := 1 To Len(aCallAdm)
420     /* Tratamento para abrir a tela de financiamento */
421     If aCallAdm[nI][5] == "FI"
422         oPnlAdconal := Eval(bCreatePan)
423         /* Tela do financiamento */
424         STIFinOrc(oPnlAdconal, aCallAdm[nI][2] , aCallAdm[nI][3], aCallAdm[nI][5], aCallAdm[nI][1])
425
426     ElseIf aCallAdm[nI][5] $ "CC|CD|PD|PX" //retiramos o trecho que faz a chamada do TEF quando um orçamento é importado
427         If lFirstTime
428             oPnlAdconal := STICriaPnlAdconal(bCreatePan)
429             /* Tela do cartao */
430             STIPayCard(oPnlAdconal, aCallAdm[nI][5] , aCallAdm[nI][2], aCallAdm[nI][3] )
431
432             lFirstTime := .F.
433         EndIf
434     Else
435         oPnlAdconal := STICriaPnlAdconal(bCreatePan)
436         //Tela do formas genericas (criadas pelo usuario ou diferentes das citadas acima)
437         STIFinOrc(oPnlAdconal, aCallAdm[nI][2] , aCallAdm[nI][3], aCallAdm[nI][5], aCallAdm[nI][1])
438     EndIf
439 Next nI
440
441 STIRfshVP(.T.) // esconde saldo vale presente até que o usuario selecione o vale presente como forma de pagamento
442
443 If lMobile
444     STIZeraPay(.T.)
445 EndIf
446
447 //Limpa o desconto caso o cliente tenha escolhido NCC e a regra
448 //de desconto esteja configurado com tabela de preco
449 If lTotDA1 .AND. STDGetDA1() > 0 .AND. nTotNCC > 0
450     STIClearDisc(.T.)
451 EndIf
452
453 /* Posiciona no primeiro registro e seta o focus */
454 //Se nao for versao mobile foca no List
455 If !lMobile
456     oListTpPaym:GoTop()
457     oListTpPaym:SetFocus()
458     //Caso tenha uma linha no resumo de pagamento e a forma de pagamento em branco, permite definir uma nova forma de pagamento
459     If oMdIParc:Length() == 1 .And. Empty(oMdIParc:GetValue('L4_FORMA'))
460         oListTpPaym:bWhen := {||.T.}
461     EndIf
462 EndIf
463
464 Return oPanPayment

```

```

465
466 //-----
467 /*{Protheus.doc} AdjShape
468 Retorna shapes a cor original e chama rotina correspondente
469
470 @param      oPPanel      - Paint Painei dos Objetos com shape
471 @param      nId          - Id do shape selecionado
472 @param      cColor       - Cor do shape
473 @param      cColorHover  - Cor do shape quando o ponteiro estiver posicionado sobre o objeto
474 @return Nil
475 @obs
476 @sample
477 */
478 //-----
479 Static Function AdjShape( oPPanel      , nId          , cColor       , cColorHover ,;
480                          nLastShape , aButtonText , aTextPgto   , oPanPayment )
481
482 Local nX                := 0          //Contador
483 Local cButtonText       := ""         //texto do botao
484
485 Default oPPanel         := Nil
486 Default nId             := 1
487 Default cColor          := ""
488 Default cColorHover     := ""
489 Default nLastShape      := 1
490 Default aButtonText     := {}
491 Default aTextPgto       := {}
492 Default oPanPayment     := Nil
493
494 // Retorna shape anterior a cor original
495 If nLastShape > -1
496     cButtonText := strTran( aButtonText[nLastShape], "@@COLOR@", "#000000" ) // Substitui cor (black)
497     oPPanel:updateShape( cButtonText )
498 EndIf
499
500 // Assumi novo "LASTSHAPE"
501 nLastShape := oPPanel:ShapeAtu
502
503 // Muda a cor do texto do item selecionado
504 cButtonText := strTran( aButtonText[oPPanel:ShapeAtu], "@@COLOR@", "#FFFFFF" ) // Substitui cor (white)
505 oPPanel:updateShape( cButtonText )
506
507 For nX := 1 to nId
508     oPPanel:SetGradient(nX,.T., cColorHover)
509     oPPanel:SetGradient(nX,.F., cColor)
510 Next
511
512
513 If STBCalcSald("1") > 0
514
515     //Seleciona forma de pagamento de acordo com ID
516     //OBS: os Ids sao fixos

```

[illegible]

```

569 Local lClearDesc      := IIF(ExistFunc('STBVldDesc'),STBVldDesc( '3', oModel:GetModel('PARCELAS') ),.F.) //Verifica se limpa o desconto antes de abrir a tela para uma nova
        forma de pagamento
570 Local lZeraDesc       := .F.
571 Local lTemPagto       := ExistFunc("STBGetPgto") .AND. STBGetPgto()>0
572 Local lDescGrupo      := ExistFunc("STDGetGrp") .AND. STDGetGrp()
573 Local lDscTotSPg      := ExistFunc("STDGetDTSPg") .AND. STDGetDTSPg() //Aplicada regra de desconto varejo no total sem forma e sem grupo
574
575 Default oPanPayment    := Nil
576 Default oListTpPaym    := Nil
577 Default cPgtoSelected  := ""
578 Default aPayment      := ARRAY(2)
579
580 //Limpa as mensagens anteriores a cada novo pagamento
581 STFCleanInterfaceMessage()
582
583 lMobile := ValType(lMobile) == "L" .AND. lMobile
584
585 If !lMobile .And. oListTpPaym <> Nil
586     cPgtoSelected := aCopyPaym[aScan(aCopyPaym,{ |x| x[2] == AllTrim(oListTpPaym:GetSelText()) }))[1]
587     oListTpPaym:SetFocus()
588 EndIf
589
590 cPgtoSelected := AllTrim(cPgtoSelected) //Pagamento selecionado
591
592 cFPgtSel := cPgtoSelected
593
594
595 // Função presente no Fonte STFTotalUpdate, responsavel por realizar o Back-Up de valores dos totalizadores do Rodapé
596 If ExistFunc("STFBkpTot")
597     STFBkpTot()
598 EndIf
599
600 // Se tiver panel ativo eh retirado, pois soh pode ser criado se entrar em algum Case a baixo
601 If oPnlAdconal <> NIL
602     oPnlAdconal:Hide()
603 EndIf
604 lVldDesc := IIF(ExistFunc('STBVldDesc'),STBVldDesc( '1', oModel:GetModel('PARCELAS') ),.T.)
605
606 lZeraDesc:= SuperGetMv("MV_LJRGDES",,.F.) .AND. (!lDescGrupo .AND. !lDscTotSPg) .AND. !STBPromPgto(cPgtoSelected) .AND. !STIGetRecTit() .AND. STIDescRegVar() .AND. !lTemPagto
607
608 //"Motor de promoções" Faz a chamada e aplica a promoção
609 If lAtivMotor
610     PshMtPdvp(Alltrim(cPgtoSelected))
611 EndIf
612
613 If lClearDesc .OR. lZeraDesc
614     STIClearDisc(.T.)
615 EndIf
616
617
618 Do Case
619     /* Se o saldo da venda zerou, entao nao abre mais nenhuma forma de pagamento */

```

```

620 Case IIF(lPayImport .And. cPgtoSelected == 'R$', .F., .T.) .And. STBCalcSald("1") == 0
621     STIEnblPaymentOptions()
622     Return .T.
623
624 /* Opção de pagamento dinheiro */
625 Case cPgtoSelected == 'R$'
626
627     /* So cria o painel se entrar dentro de algum case */
628     oPnlAdconal := STICriaPnlAdconal(bCreatePan)
629     If lPromPgto .AND. STBPromPgto("R$") .AND. lVldDesc //Verifica se existe regra de bonificacao
630         STBTotRldi(,,, .T., "R$")
631     EndIf
632
633     /* Tela de dinheiro */
634     STIPayCash(oPnlAdconal)//Dentro da PayCash que validará se existe regra por Brinde, porque não precisará adicionar nenhuma forma de pagto somente exibir as opções de
brinde para selecionar
635 /* Opção de pagamento credito ou debito */
636 Case cPgtoSelected $ 'CC|CD|PD|PX'
637
638     // Homologacao TEF
639     // Se for homologacao limita o Numero de cartoes em no Maximo 2 por venda
640     If SuperGetMV("MV_LJHMTF",, .F.) .AND. nUsedCards >= 2
641         STFMessage(ProcName(),"POPUP",STR0021) //"Maximo de 2 cartoes utilizados na venda"
642         STFShowMessage(ProcName())
643     Else
644         /* So cria o painel se entrar dentro de algum case */
645         oPnlAdconal := STICriaPnlAdconal(bCreatePan)
646         cTypeCard := cPgtoSelected
647
648         If lPromPgto .AND. STBPromPgto(cTypeCard) .AND. lVldDesc .AND. lCart //Verifica se existe regra de bonificacao
649             STBTotRldi(,,, .T., cTypeCard, oPnlAdconal)
650         EndIf
651         /* Tela do cartao */
652         STIPayCard(oPnlAdconal, cTypeCard, aPayment[1], aPayment[2])
653
654     Endif
655
656 /* Opção de pagamento financiado */
657 Case cPgtoSelected == 'FI'
658
659     cTpForm := cPgtoSelected
660
661     If Len(STDGetAdmFin(cTpForm)) > 1
662         /* So cria o painel se entrar dentro de algum case */
663         oPnlAdconal := Eval(bCreatePan)
664
665         If lPromPgto .AND. STBPromPgto("FI") .AND. lVldDesc //Verifica se existe regra de bonificacao
666             STBTotRldi(,,, .T., "FI", oPnlAdconal)
667         EndIf
668         STIPayFinancial(oPnlAdconal, cTpForm)
669
670     Endif

```

```

671
672 /* Opção de pagamento Shop Card (Cartao Fidelidade) */
673 Case cPgtoSelected == 'FID'
674
675
676 /* So cria o painel se entrar dentro de algum case */
677 oPnlAdconal := Eval(bCreatePan)
678
679 If !STDGUpdShopCardFundsResult() // Caso haja um produto de recarga de cartao fidelidade, a opcao fica indisponivel.
680     If lPromPgto .AND. STBPromPgto("FID") .AND. lVldDesc //Verifica se existe regra de bonificacao
681         STBTotRlDi( , , , .T., "FID", oPnlAdconal)
682     Else
683         STIPayShopCard(oPnlAdconal)
684     Endif
685 Else
686     STFMessage(ProcName(), "STOP", STR0022) //"Por se tratar de recarga de cartão fidelidade, essa opção está indisponível."
687     STFSshowMessage(ProcName())
688     STFCleanMessage(ProcName())
689 EndIf
690
691 /* Opção de pagamento cheque */
692 Case cPgtoSelected == 'CH'
693
694 /* So cria o painel se entrar dentro de algum case */
695 oPnlAdconal := STICriaPnlAdconal(bCreatePan)
696 If lPromPgto .AND. STBPromPgto("CH") .AND. lVldDesc //Verifica se existe regra de bonificacao
697     STBTotRlDi( , , , .T., "CH", oPnlAdconal)
698 EndIf
699
700 /* Telado do cheque */
701 STIPayCheck(oPnlAdconal, bDataCheck)
702
703 /* Opção de pagamento vale presente */
704 Case cPgtoSelected == 'VP'
705
706 /* So cria o painel se entrar dentro de algum case */
707 oPnlAdconal := Eval(bCreatePan)
708
709 If lPromPgto .AND. STBPromPgto("VP") .AND. lVldDesc //Verifica se existe regra de bonificacao
710     STBTotRlDi( , , , .T., "VP", oPnlAdconal)
711 End If
712 /* Tela do vale presente */
713 STIPayGiftV(oPnlAdconal)
714
715 /* Opção de condicao de pagamento */
716 Case cPgtoSelected == 'CP'
717
718 lContinua := STIVerCTef()
719
720 If lContinua
721     lDisableBtn := .T.
722

```

```

723      /* So cria o painel se entrar dentro de algum case */
724      oPnlAdconal := Eval(bCreatePan)
725
726      If lPromPgto .AND. STBPromPgto("CP") //Verifica se existe regra de bonificacao
727          STBTotRlDi( , , , .T., "CP", oPnlAdconal)
728      Else
729          /* Tela da condicao de pagamento */
730          STIPayCdPg(oPnlAdconal)
731      Endif
732  Else
733      If !lMobile .AND. oListTpPaym <> NIL
734          oListTpPaym:SetFocus()
735      EndIf
736  EndIf
737
738  /* Opção de Multi Negociacao */
739  Case cPgtoSelected == 'MN'
740
741      lContinua := STIVerCTef()
742
743      If lContinua
744          // Reutilizacao da NCC
745          // Quando vai alterar para multinegociacao, revalida a Ncc escolhida anteriormente
746          // para ser considerada nos calculos.
747          STIZeraPay(.T.) // limpa tudo
748          aNCCs := STDGetNCCs("1")
749          For nX := 1 to Len(aNCCs)
750              If aNCCs[nX,1]
751                  STDSetNCCs("2",aNCCs[nX,SE1SALD0]) // Atualiza o saldo total de NCCs selecionadas
752              EndIf
753          Next nX
754          STIPayment(.F.)
755          STIAddPay("CR", Nil, 1, Nil, Nil, STDGetNCCs("2"))
756
757          /* So cria o painel se entrar dentro de algum case */
758          If Len( STIGetRules(STBCalcSald("1")-STWGetIncrease()) ) > 1 //aqui seta o valor que sera utilizado na multinegociacao
759              /* So cria o painel se entrar dentro de algum case */
760              oPnlAdconal := NIL
761
762              If lPromPgto .AND. STBPromPgto("MN") //Verifica se existe regra de multinegociacao
763                  STBTotRlDi( , , , .T., "MN", oPnlAdconal)
764              Else
765                  /* Tela da condicao de pagamento */
766                  STIExchangePanel({|| STIPnlMulti(oPnlAdconal) })
767              Endif
768          Else
769              STIEnblPaymentOptions()
770              If !lMobile .AND. oListTpPaym <> NIL
771                  oListTpPaym:SetFocus()
772              EndIf
773          EndIf
774      Else

```

```

775 If !lMobile .AND. oListTpPaym <> Nil
776     oListTpPaym:SetFocus()
777 EndIf
778 EndIf
779
780 /* Nota de Crédito Cód. Barras */
781 Case cPgtoSelected == 'NB'
782
783     /* So cria o painel se entrar dentro de algum case */
784     oPnlAdconal := Eval(bCreatePan)
785
786     If lPromPgto .AND. STBPromPgto("NB") //Verifica se existe regra de bonificacao
787         STBTotRldi( ,,, .T., "NB", oPnlAdconal)
788     Else
789         /* Tela da Nota de Crédito Cód. Barras */
790         STIPayNCCBC(oPnlAdconal)
791     Endif
792 Otherwise
793
794     cTpForm := cPgtoSelected
795     If Len(STDGetAdmFin(cTpForm)) > 1
796         /* So cria o painel se entrar dentro de algum case */
797         oPnlAdconal := STICriaPnlAdconal(bCreatePan)
798
799         If lPromPgto .AND. STBPromPgto(cTpForm) .AND. lvldDesc //Verifica se existe regra de bonificacao
800             STBTotRldi( ,,,, .T., cTpForm, oPnlAdconal)
801         EndIf
802
803         //Ponto de Entrada que permite customizar o painel de pagamentos da condicao de pagamento especifica
804         If ExistBlock("STIPAYCST")
805             lPayFin := ExecBlock("STIPAYCST", .F., .F., {oPnlAdconal, cTpForm})
806         Endif
807
808         If lPayFin
809             STIPayFinancial(oPnlAdconal, cTpForm)//Retirar a chamada STBTotRldi desse fonte
810         Endif
811
812     Else
813         STIEnblPaymentOptions()
814         If !lMobile .AND. oListTpPaym <> Nil
815             oListTpPaym:SetFocus()
816         EndIf
817     EndIf
818 EndCase
819
820 Return .T.
821
822 ////////////////////////////////////////////
823 /*/{Protheus.doc} ModelPayme
824 Model - Escolha parcelas
825
826 @param
```

[illegible]

```

879 oStru:AddField( STR0007      ,; //[01] Titulo do campo
880                STR0007      ,; //[02] Desc do campo
881                "L4_FILIAL" ,; //[03] Id do Field
882                "C"          ,; //[04] Tipo do campo
883                8            ,; //[05] Tamanho do campo
884                0            ,; //[06] Decimal do campo
885                Nil          ,; //[07] Code-block de validacao do campo
886                Nil          ,; //[08] Code-block de validacao When do campo
887                Nil          ,; //[09] Lista de valores permitido do campo
888                Nil          ,; //[10] Indica se o campo tem preenchimento obrigatorio
889                Nil          ,; //[11] Code-block de inicializacao do campo
890                Nil          ,; //[12] Indica se trata-se de um campo chave
891                Nil          ,; //[13] Indica se o campo pode receber valor em uma operacao de update.
892                .T.          ) //[14] Indica se o campo e virtual
893
894 oStru:AddField( STR0008      ,; //[01] Titulo do campo
895                STR0008      ,; //[02] Desc do campo
896                "L4_CONDPG" ,; //[03] Id do Field
897                "BT"         ,; //[04] Tipo do campo
898                005          ,; //[05] Tamanho do campo
899                0            ,; //[06] Decimal do campo
900                {|| .T. }    ,; //[07] Code-block de validacao do campo
901                Nil          ,; //[08] Code-block de validacao When do campo
902                Nil          ,; //[09] Lista de valores permitido do campo
903                Nil          ,; //[10] Indica se o campo tem preenchimento obrigatorio
904                Nil          ,; //[11] Code-block de inicializacao do campo
905                Nil          ,; //[12] Indica se trata-se de um campo chave
906                Nil          ,; //[13] Indica se o campo pode receber valor em uma operacao de update.
907                Nil          ) //[14] Indica se o campo e virtual
908
909 Else
910
911 oStru:AddField( STR0009      ,; //[01] Titulo do campo
912                STR0009      ,; //[02] Desc do campo
913                "L4_PARC"    ,; //[03] Id do Field
914                "N"          ,; //[04] Tipo do campo
915                TamSX3("L1_PARCELA")[1],; //[05] Tamanho do campo
916                0            ,; //[06] Decimal do campo
917                Nil          ,; //[07] Code-block de validacao do campo
918                Nil          ,; //[08] Code-block de validacao When do campo
919                Nil          ,; //[09] Lista de valores permitido do campo
920                Nil          ,; //[10] Indica se o campo tem preenchimento obrigatorio
921                Nil          ,; //[11] Code-block de inicializacao do campo
922                Nil          ,; //[12] Indica se trata-se de um campo chave
923                Nil          ,; //[13] Indica se o campo pode receber valor em uma operacao de update
924                .T.          ) //[14] Indica se o campo e virtual
925
926 oStru:AddField( "Tef"        ,; //[01] Titulo do campo
927                "Tef"        ,; //[02] Desc do campo
928                "L4_TEF"     ,; //[03] Id do Field
929                "L"          ,; //[04] Tipo do campo
930                1            ,; //[05] Tamanho do campo

```

[illegible]

[illegible]

```

AllTrim(cForma) }) == 0 )
1035 //Nao encontrou forma de pagamento na SX5
1036 STFMessage("STIPayment","POPOP", STR0034 + AllTrim(cForma) + STR0035 + "SX5." ) //"A Forma de pagamento: " ### " selecionada não está cadastrada neste PDV Tabela "
1037 STFShowMessage("STIPayment")
1038 lRet := .F.
1039 EndIf
1040
1041 If lRet .AND. (!Empty(aPgto) .And. cForma == "BF" )
1042 // Atualiza o grid aPayments
1043 oMdlGrd:acols := aPgto
1044 Endif
1045
1046 If lRet .AND. lPromPgto .AND. nTotDA1 > 0 .AND. !(AllTrim(cForma) $ 'CC|CD') .AND. ExistFunc('STBVldDesc')
1047 STBVldDesc('2',,IIF(ValType(oModelFormaPag) == '0', oModelFormaPag:GetValue('L4_VALOR'),nValor))
1048 STBSetReg(.T.)
1049 ElseIf lRet .AND. (lPromPgto .And. STBCalcSald("1") > nValFPgt) .AND. nTotDA1 == 0
1050 nValRgDes := (STDGPBasket( "SL1" , "L1_VLRTOT" ) - STBCalcSald("1")) + nValFPgt
1051 STBTotRlDi(nValRgDes,,, cForma ) //Atualiza valor, caso regra de pagamento se aplique
1052 EndIf
1053
1054 If lRet .AND. ((ValType(oModelFormaPag) == '0' .AND. oModelFormaPag:GetValue('L4_VALOR') > 0) .OR. (nValor > 0))
1055
1056 oMdlGrd:SetNoInsertLine(.F.)
1057
1058 /* Tratamento para somar vários pagamentos em dinheiro ou NCC*/
1059 If alltrim(cForma) == 'R$' .Or. AllTrim(cForma) == 'CR'
1060 //O metodo SeekLine substitui o aScan no acols (que nao esta sendo mais utilizado para a versao 12)
1061 //Se retornar .T. é que foi encontrado a busca e automaticamente ja esta sendo posicionada na linha desejada.
1062 If oMdlGrd:SeekLine({"L4_FORMA", alltrim(cForma)})
1063 nJ := oMdlGrd:GetLine()
1064
1065 //Neste trecho contem o tratamento (adotado como padrão seguindo FrontLoja) para substituir o valor em dinheiro
1066 //quando ja possua a forma dinheiro informado anteriormente.
1067 //Caso necessario basta alterar para a seguinte informação para começar a somar os valores (o valor ja informado e novo valor)
1068 //oMdlGrd:LoadValue( "L4_VALOR",oModelFormaPag:GetValue("L4_VALOR")+oMdlGrd:aCols[nJ,aScan( oMdlGrd:aheader,{|x| alltrim(x[2])=='L4_VALOR'} )],nJ)
1069 oMdlGrd:LoadValue( "L4_VALOR",oModelFormaPag:GetValue("L4_VALOR"),nJ)
1070 lTemDinNCC := .T.
1071 Else
1072 nJ := 1
1073 Endif
1074 Endif
1075
1076 If !lTemDinNCC
1077 /* Caso seja necessário, e adicionada uma linha ao grid */
1078 IIF(oMdlGrd:Length() == 1 .AND. Empty(oMdlGrd:GetValue("L4_DATA")), Nil, nJ := oMdlGrd:AddLine(.T.))
1079
1080 LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "Gravacao Model SL4" )
1081
1082 /*
1083 Busca nos campos da SL4 os campos que foram preenchidos pela forma de pagamento.
1084 Para isso, os campos devem ter o seu nome (ID) exatamente igual
1085 */

```

```

1086 For nX := 1 To Len(aFieldsParcelas)
1087   cCampo := aFieldsParcelas[nX][MODEL_FIELD_IDFIELD]
1088   nFieldPos := aScan(aFieldsFormPag,{|aCampo| aCampo[MODEL_FIELD_IDFIELD] == cCampo})
1089   If nFieldPos > 0
1090     If lLJRBAC .AND. lSTDGetDias .AND. ValType(oRetTef) == "0" .AND. ALLTRIM(cCampo) == "L4_DATA" .AND. !Empty(oRetTef:cAdmFin) .AND. AllTrim(cForma) == "CD"
1091       oMdlGrd:LoadValue( cCampo, oModelFormaPag:GetValue(cCampo) + STDGetDias(SubStr(oRetTef:cAdmFin,1,TamSx3('L4_ADMINIS')[1])),nJ)
1092     Else
1093       oMdlGrd:LoadValue( cCampo, oModelFormaPag:GetValue(cCampo),nJ)
1094     EndIf
1095   EndIf
1096 Next nX
1097 If Len(aFieldsFormPag) == 0
1098   oMdlGrd:LoadValue( "L4_DATA", dDataBase, nJ)
1099   oMdlGrd:LoadValue( "L4_VALOR", nValor, nJ)
1100 EndIf
1101
1102 oMdlGrd:LoadValue( "L4_FORMA" , cForma,nJ)
1103 oMdlGrd:LoadValue( "L4_PARC" , nParc,nJ)
1104 oMdlGrd:LoadValue( "L4_TEF" , lTef,nJ)
1105 oMdlGrd:LoadValue( "L4_CODVP" , cCodVp,nJ)
1106 oMdlGrd:LoadValue( "L4_NUMCART" , cCodVp,nJ)
1107 oMdlGrd:LoadValue( "L4_CONTDOC" ,STDGPBasket( "SL1" , "L1_CONTDOC" ))
1108 oMdlGrd:LoadValue( "L4_SERPDV" , STFGetStat("SERPDV"),nJ)
1109
1110 STISetPg0r(cForma, .T.)
1111
1112 If ValType(oRetTef) == "0"
1113
1114   LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "Grava Dados TEF" )
1115
1116   If ValType(oRetTef:DDATA) == "D"
1117     oMdlGrd:LoadValue( "L4_DATATEF" , DTOS(oRetTef:DDATA),nJ)
1118   Else
1119     oRetTef:DDATA := StrTran(oRetTef:DDATA, "/")
1120     If Left(oRetTef:DDATA,4) == Str(Year(Date()),4)
1121       oMdlGrd:LoadValue( "L4_DATATEF" , oRetTef:DDATA,nJ)
1122     Else
1123       oMdlGrd:LoadValue( "L4_DATATEF" , Right(oRetTef:DDATA,4) + Substr(oRetTef:DDATA,3,2)+ Left(oRetTef:DDATA,2) ,nJ)
1124     EndIf
1125   EndIf
1126   oMdlGrd:LoadValue( "L4_HORATEF" , Replace(oRetTef:CHORA,":",""),nJ )
1127   oMdlGrd:LoadValue( "L4_DOCTEF" , oRetTef:CNSUAUTOR,nJ )
1128   oMdlGrd:LoadValue( "L4_AUTORIZ" , oRetTef:CAUTORIZ,nJ )
1129   oMdlGrd:LoadValue( "L4_INSTITU" , oRetTef:CREDE,nJ )
1130   oMdlGrd:LoadValue( "L4_NSUTEF" , oRetTef:CNSU,nJ )
1131   oMdlGrd:LoadValue( "L4_TIPCART" , "",nJ ) // Verificar o tipo de cartao
1132   oMdlGrd:LoadValue( "L4_PARCTEF" , oRetTef:CPARCTEF,nJ )
1133   oMdlGrd:LoadValue( "L4_DESCFIN" , oRetTef:nVlrDescTEF,nJ )
1134   oMdlGrd:LoadValue( "L4_ADMINIS" , AllTrim(oRetTef:cAdmFin), nJ)
1135
1136   If SL4->(ColumnPos("L4_BANDEIR")) > 0 .And. lCodBand //Verifica se o atributo cCodBand existe no objeto
1137     //Codigo da Bandeira que foi utilizada

```

```

1138         oMdlGrd:LoadValue( "L4_BANDEIR" , oRetTef:cCodBand,nJ )
1139     EndIf
1140
1141     If SL4->(ColumnPos("L4_REDEAUT")) > 0
1142         //Codigo da Rede que autorizou a transacao TEF
1143         oMdlGrd:LoadValue( "L4_REDEAUT" , oRetTef:cTipCart,nJ )
1144     EndIf
1145
1146     oMdlGrd:LoadValue( "L4_NOMECLI" , PadR( "RD: " + Alltrim(oRetTef:CREDE) + " / BD: " + Alltrim(oRetTef:cAdmFin), TamSX3("L4_NOMECLI")[1]) ,nJ )
1147
1148     If SL4->(ColumnPos("L4_TRNID")) > 0 .And. AttIsMemberOf(oRetTef, "cExternalTransactionId", .T.) //Verifica se existe o campo e o Atributo da classe
LJARetTransacaoTef (LOJA1934)
1149         oMdlGrd:LoadValue( "L4_TRNID" , oRetTef:cIdtransaction,nJ )
1150     EndIf
1151     If SL4->(ColumnPos("L4_TRNPCID")) > 0 .And. AttIsMemberOf(oRetTef, "cProcessorTransactionId", .T.) //Verifica se existe o campo e o Atributo da classe
LJARetTransacaoTef (LOJA1934)
1152         oMdlGrd:LoadValue( "L4_TRNPCID" , oRetTef:cProcessorTransactionId,nJ )
1153     EndIf
1154     If SL4->(ColumnPos("L4_TRNEXID")) > 0 .And. AttIsMemberOf(oRetTef, "cExternalTransactionId", .T.) //Verifica se existe o campo e o Atributo da classe
LJARetTransacaoTef (LOJA1934)
1155         oMdlGrd:LoadValue( "L4_TRNEXID" , oRetTef:cExternalTransactionId,nJ )
1156     EndIf
1157
1158     If lHomolPaf .AND. lIsPAF .AND. LJAnalisaLeg(63)[1] //Troco em Santa Catarina
1159         nTotPago := STIGetTotal() //Retorna o Total Pago
1160         nTotVenda := STDGPBasket( "SL1" , "L1_VLRTOT" )
1161
1162         If nTotPago > nTotVenda
1163             oMdlGrd:LoadValue( "L4_TROCO" , nTotPago - nTotVenda,nJ )
1164         EndIf
1165     EndIf
1166
1167     STBSetRetTef()
1168 Else
1169     LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "Venda sem TEF" )
1170 EndIf
1171 Endif
1172
1173 /* controle sobre o ID do CARTAO */
1174 If ExistFunc("STBSetIDTF") .AND. oMdlGrd:HasField("L4_FORMAID") //verifica se o campo existe na estrutura
1175     If cForma == "CC"
1176         //incrementa o ID do cartao de CREDITO
1177         STBSetIDTF("CC")
1178         oMdlGrd:LoadValue( "L4_FORMAID" , STBGetIDTF("CC"), nJ )
1179     ElseIf cForma == "CD"
1180         //incrementa o ID do cartao de DEBITO
1181         STBSetIDTF("CD")
1182         oMdlGrd:LoadValue( "L4_FORMAID" , STBGetIDTF("CD"), nJ )
1183     ElseIf cForma == "PX"
1184         //incrementa o ID da forma PX
1185         STBSetIDTF("PX")
1186         oMdlGrd:LoadValue( "L4_FORMAID" , STBGetIDTF("PX"), nJ )

```

```

1187     ElseIf cForma == "PD"
1188         //incrementa o ID da forma PD
1189         STBSetIDTF("PD")
1190         oMdlGrd:LoadValue( "L4_FORMAID" , STBGetIDTF("PD"), nJ )
1191     EndIf
1192 EndIf
1193
1194 // Grava informações do troco na SL4
1195 nTotPago := STIGetTotal() //Retorna o Total Pago
1196 nTotVenda := STDGPBasket( "SL1" , "L1_VLRTOT" )
1197 If nTotPago > nTotVenda
1198     LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "Grava valor troco L4_TROCO." )
1199     oMdlGrd:LoadValue( "L4_TROCO" , nTotPago - nTotVenda,nJ )
1200 EndIf
1201
1202 /* Atualiza o grid aPayments */
1203 LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "Antes chamada STBUdpPaym" )
1204 STBUdpPaym(oMdl)
1205
1206 /* Atualiza a lista de resumo de pagamento */
1207 If ValType(oListResPg) <> "U"
1208     aResPay := STBUdpResPg(nAjustCols,.F.)
1209
1210     oListResPg:Reset()
1211     oListResPg:SetArray(aResPay)
1212 EndIf
1213 oMdlGrd:GoLine(1)
1214
1215 For nX := 1 To len(oMdlGrd:acols)
1216     If !Empty(oMdlGrd:acols[1][4])
1217         If !Empty(aPgto)
1218             If aPgto[nX][4] <> oMdlGrd:acols[1][4]
1219                 For nY := 1 To len(oMdlGrd:acols[nX])
1220                     aPgto[nX][nY] := oMdlGrd:acols[nX][nY]
1221                 Next nY
1222             Endif
1223         Else
1224             aPgto:= oMdlGrd:acols
1225         Endif
1226     Endif
1227 Next nX
1228
1229 ElseIf lRet
1230     IIF(ValType(oModelFormaPag) == '0', nValInvalid := oModelFormaPag:GetValue('L4_VALOR'), nValInvalid := nValor )
1231     STFMessage("STIPayment","STOP", STR0025 + AllTrim(Str(nValInvalid,10,2))) //'Valor inválido. '
1232     STFShowMessage("STIPayment")
1233     LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "Valor Inválido" )
1234 EndIf
1235
1236 If lRet .AND. ( ValType(oListTpPaym) <> "U" .AND. !lMobile )
1237     STIEnblPaymentOptions()
1238     oListTpPaym:SetFocus()

```

[illegible]

```

1290
1291 If STIGetPayR0()
1292
1293     STFMessage( ProcName(),"ALERT", STR0029)    //"Sem permissão para Alterar Parcelas"
1294     STFShowMessage( ProcName() )
1295 Else
1296     If lCheca .And. GetApoInfo("LOJA120.PRW")[4] >= CTOD("11/11/2016") .And. !lPermis //Chamada pelo botao Limpa Pagto(s) - CTRL + L
1297         lPermis := STFProfile(41)[1] //Valida permissao de usuario para zerar pagamentos
1298     EndIf
1299
1300     If ExistFunc("STBGetDPgChck")
1301         STBGetDPgChck()
1302     EndIf
1303
1304     If lPermis //Permissao zerar pagamentos
1305
1306         lTemTEF := STIGetCard()
1307         oParc:Activate()
1308         oPaym:Activate()
1309
1310         /*
1311             Caso os fontes estejam em datas diferentes não afeta o
1312             funcionamento anterior a esta implementação no sistema
1313         */
1314         If !STIAtvTefP()
1315             lZeraTEF := .T.
1316         EndIf
1317
1318         If lRegDesc
1319             For nCols := 1 to Len(oListResPg:aItems)
1320                 If Left(oListResPg:aItems[nCols],2) = "BF" //oListResPg:aItems
1321                     lBonif := .T.
1322                 EndIf
1323             Next
1324         EndIf
1325
1326         If lZeraTEF .Or. !lTemTEF .Or. (lTemTEF .And. cMV_TEFPEND == "0")
1327
1328             oParc:ClearData()
1329             oParc:InitLine()
1330
1331             oPaym:ClearData()
1332             oPaym:InitLine()
1333
1334             If lBonif
1335                 STBFormBf()
1336             EndIf
1337
1338             lMantemCrd := .F.
1339         Else
1340             STILmpTef(@oParc,@oPaym,lBonif)
1341             If oParc:Length() > 0

```

```

1342         lMantemCrd := .T.
1343     EndIf
1344 EndIf
1345
1346 STISetCard(lMantemCrd)
1347 STBSetCheck()
1348
1349 If nOpcDesc == 1 .AND. nTotDA1 > 0 .AND. ProcName(3) == 'STIPOSMAIN'
1350     STIClearDisc(.T.)
1351 EndIf
1352
1353 If ExistFunc('STBSetCart')
1354     STBSetCart(.T.)
1355 EndIf
1356
1357 //Se for multinegociacao e tem acrescimo, limpa o
1358 //acrescimo ao limpar as formas de pagamento
1359 If lMNeg .AND. STWGetIncrease() > 0
1360     STWAddIncrease( 0 , 0 )
1361     STISetMult(.F.)
1362 EndIf
1363
1364 //Limpa NCC's selecionadas.
1365 STDSetNCCs("2")
1366
1367 //Limpa backup da aSl4
1368 If ExistFunc("STBLMPaSl4")
1369     STBLMPaSl4()
1370 EndIf
1371
1372 //Volta ao estado original da variavel que indica se houve alteração em alguma informação do orçamento do tipo FI
1373 If ExistFunc("STISFiAltImp")
1374     STISFiAltImp()
1375 EndIf
1376
1377 If lMantemCrd
1378     If lBonif
1379         STIAddPay("BF", Nil, 1, Nil, Nil, oTotal:GetValue("L1_BONIF") )
1380     EndIf
1381     aFormas      := STBUpdResPg(nAjustCols,.F.)
1382
1383     If ValType(oListResPg) <> "U"
1384         oListResPg:Reset()
1385         oListResPg:SetArray(aFormas)
1386     EndIf
1387
1388 Else
1389     STBSetRetTef(lTemTEF)
1390     STBSetEnt()
1391
1392     If ValType(oListResPg) <> "U"
1393         oListResPg:Reset()

```

```

1394         If lBonif
1395             STIAddPay("BF", Nil, 1, Nil, Nil, oTotal:GetValue("L1_BONIF") )
1396         EndIf
1397     EndIf
1398
1399     STBSetParc()
1400
1401     // Retorna o objeto do model com todos os totais definidos
1402     STFGetTot()
1403
1404     If ExistFunc("STBSetIDTF")
1405         STBSetIDTF() //reseta a variavel de controle dos IDs dos CARTOES
1406     EndIf
1407
1408     //Limpa variaveis da condição de pagamento da venda
1409     If ExistFunc("STBCDPGAtuBasket")
1410         STBCDPGAtuBasket(.T.)
1411     EndIf
1412 EndIf
1413
1414 If ExistFunc("STISetVPSaldo")
1415     STISetVPSaldo() // define saldo do vale presente
1416 EndIf
1417 STIRfshVP(.T.) // esconde o campo saldo do vale presente
1418
1419 If ExistFunc("STDDelBonificacao")
1420     //Exclui produto comprado por bonificacao via forma de pagamento, precisa vir antes para poder atualizar o valor da cesta
1421     STDDelBonificacao(lCheca)
1422 EndIf
1423
1424 /* Limpa o desconto e acrescimo financeiro */
1425 If ExistFunc("STBLDsAcFin")
1426     STBLDsAcFin()
1427 EndIf
1428
1429 If AliasInDic("MGC") .And. ExistFunc("STDDelBrindeCesta")
1430     //Deleta brinde por Forma de pagamento
1431     STDDelBrindeCesta(lCheca)
1432 EndIf
1433
1434 //Limpar o desconto no total. e limpar array de promoções.
1435 If lBtZeraPay .AND. lAtivMotor .AND. PshAplProm()
1436     STIClearDisc(.T.)
1437     PshClearPromo()
1438     PshCodMTP()
1439 EndIf
1440
1441 /*/ Verifica se Regra Desconto esta ativa e existe regra cadastrada
1442 Verifica se venda teve desconto no total vindo de alguma regra cadastrada,
1443 caso positivo limpa desconto pois preciso manter a forma de pagamento. /*/
1444 If Len(STIGetOrc()) > 0
1445     //1=Considera; 2=Desconsidera o desconto; 3=Pergunta

```

[illegible]

[illegible]

```

1550
1551 /* Inclui no arrays o tipo NCC com BarCode */
1552 If SuperGetMv("MV_LJNCCBC", , "0" ) <> "0"
1553     Aadd(aPayments, AllTrim(Str(nI)) + ' - ' + STR0026 ) //"Nota de Crédito Cód. Barras"
1554     Aadd(aCopyPaym, {'NB',AllTrim(Str(nI++)) + ' - ' + STR0026 , STR0026 }) //"Nota de Crédito Cód. Barras"
1555 EndIf
1556
1557 Return { aPayments, aCopyPaym }
1558
1559 //-----
1560 /*{Protheus.doc} STIFpgtBlq
1561 Formas de Pagamentos Bloqueadas no PDV
1562
1563 @param      cFrmPag  Forma de Pagamento
1564 @return      lRet  Retorna .T. caso a forma de pagamento estiver bloqueada para uso no PDV
1565 @obs
1566 @sample
1567 */
1568 //-----
1569 Function STIFpgtBlq(cFrmPag)
1570
1571 Local lRet      := .F.
1572
1573 If cFrmPag=='BOL' .OR. cFrmPag=='CR' .OR. cFrmPag== 'BF'
1574     lRet:= .T.
1575 Endif
1576
1577 Return lRet
1578
1579 //-----
1580 /*{Protheus.doc} STIConfPay
1581 Confirmacao dos pagamentos
1582
1583 @param      lShowRgIt      Chama registro de item ao finalizar
1584 @return
1585 @obs
1586 @sample
1587 */
1588 //-----
1589 Function STIConfPay( lShowRgIt, cKey )
1590
1591 Local oMdl      := Nil                                //Recupera o model ativo
1592 Local oMdlGrd   := Nil                                //Seta o model do grid
1593 Local oMdlPaym   := Nil                                //Seta o model do pagamento
1594 Local lRet       := .F.                                //Variável de retorno
1595 Local lMobile    := STFGetCfg("lMobile", .F.)          //Smart Client Mobile
1596 Local cMV_FORMCRD := SuperGetMV("MV_FORMCRD",,"CH/FI") //Forma de Pagamento para CRD
1597 Local aCRD       := {}                                //Traz informações do cartão utilizado para CRD
1598
1599 Default lShowRgIt := .T.
1600 Default cKey := "Buffer > Desconhecido"
1601

```

```

1602 STIResPayObj(.T.)
1603
1604 SetKey(16, Nil)
1605
1606 lMobile := ValType(lMobile) == "L" .AND. lMobile
1607
1608 LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "Confirmação dos pagamentos" ) //Gera LOG
1609 LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "Acionamento via : " + cKey ) //Gera LOG
1610
1611 oMdl      := oModel
1612 oMdlGrd   := Iif(oMdl <> Nil, oMdl:GetModel("PARCELAS"), Nil)
1613 oMdlPaym  := Iif(oMdl <> Nil, oMdl:GetModel("APAYMENTS"), Nil)
1614
1615 If STBCalcSaldo("1") == 0 .And. oMdl <> Nil
1616
1617     If STIGetTotal() > 0
1618
1619         /* Solicita a selecao da Adm. Financeira p/ Condicao de Pagamento cujo Forma é do tipo "FI" */
1620         STIAdmCdPg(aCopyPaym, oListTpPaym, oMdl)
1621
1622         lRet := .T.
1623
1624         //Análise para Integração SIGACRD
1625         If ExistFunc("STBGetCrdIdent") .AND. ExistFunc("STWAvalCRDIntegration") .AND.;
1626             AllTrim(oMdlGrd:GetValue("L4_FORMA")) $ cMV_FORMCRD .AND. CrdXInt(.F.,.F.)
1627
1628             //Primeiro passo: Vejo se o cartão foi habilitado. Se não, não prossigo e volto para a forma de pagamento.
1629             aCrd := STBGetCrdIdent()
1630
1631             If Len(aCRD) >= 2 .AND. Empty(aCrd[2])
1632                 lRet := .F.
1633                 STFMessage(ProcName(),"POPUP",STR0045) //"A seleção do Cliente Identificado com CPF/CNPJ é obrigatória antes da venda. Escolha outra forma de pagamento."
1634                 STFShowMessage(ProcName())
1635             EndIf
1636
1637             If lRet
1638                 lRet := STWAvalCRDIntegration(STBRPayaCols(oMdlGrd)) //Se .F., voltar para a condição de pagamento.
1639             EndIf
1640
1641             If !lRet .AND. Len(oListResPg:aItems) > 0 //Se retornar false, e se estiver preenchido a forma de pagamento, eu limpo a forma.
1642                 STIZeraPay(.T.)
1643             EndIf
1644         EndIf
1645
1646         If lRet
1647             STISetPayImp(.F.)
1648             lRet := STBConfPay(oMdlGrd, aCopyPaym, oMdlPaym)
1649         EndIf
1650
1651         If lRet
1652             nUsedCards := 0
1653             STIGridCupRefresh()

```

[illegible]

[illegible]

```

1758     EndIf
1759 EndIf
1760
1761
1762 For nX := 1 To oMdlGrd:Length()
1763
1764     If nX > 1
1765         STDPBAddLine("SL4")
1766     EndIf
1767
1768     oMdlGrd:GoLine(nX)
1769
1770     STDSPBasket("SL4", "L4_FILIAL" ,cFillL4 , nX )
1771     STDSPBasket("SL4", "L4_NUM" ,cNumL4 , nX )
1772
1773     If lMFE .And. Len(aIdPgtoMfe) > 0 .And. AllTrim(oMdlGrd:GetValue("L4_FORMA")) $ "CC|CD"
1774         //Tratamento para posicionar no Id MF-e correto para todas as parcelas da transação Tef
1775         If nX > 1 .And. !Empty(cNsuTef) .And. cNsuTef <> oMdlGrd:GetValue('L4_NSUTEF')
1776             nI++
1777         EndIf
1778
1779         STDSPBasket("SL4", "L4_IDPGVFP" ,aIdPgtoMfe[nI][1] , nX )
1780         cNsuTef := oMdlGrd:GetValue('L4_NSUTEF')
1781     EndIf
1782
1783     //Serie do PDV
1784     STDSPBasket("SL4", "L4_SERPDV" ,STFGetStat("SERPDV") , nX )
1785
1786     //Grava o valor do acrescimo financeiro do campo L4_ACRSFIN no model
1787     If ExistFunc("STBGetAcresFin")
1788         aAcresFin := STBGetAcresFin()
1789     EndIf
1790
1791     If lAcrRet .AND. Len(aSL4bkp) >= nX .AND. nPosACRS > 0
1792         oMdlGrd:LoadValue( "L4_ACRSFIN" ,aSL4bkp[nX][nPosACRS][2])
1793     ElseIf lAcrRet .AND. Len(aAcresFin) > 1 .AND. aAcresFin[1] > 0
1794         oMdlGrd:LoadValue( "L4_ACRSFIN" ,aAcresFin[1] )
1795     EndIf
1796
1797     For nJ := 1 To Len(aFieldsGrd)
1798         If !(aFieldsGrd[nJ][3] $ 'L4_FILIAL|L4_NUM|L4_IDPGVFP') .AND. !aFieldsGrd[nJ][14]
1799             STDSPBasket("SL4", aFieldsGrd[nJ][3], oMdlGrd:GetValue(aFieldsGrd[nJ][3]), nX)
1800         EndIf
1801     Next nJ
1802
1803     // tratamento de troco:
1804     If IsMoney(oMdlGrd:GetValue('L4_FORMA'))
1805         If ExistFunc("STBGrvTroco")
1806             STBGrvTroco(oMdlGrd,nX)
1807         Else
1808             LjGrvLog( "L4_NUM: " + cNumL4, "Gravação de troco" ,"Atualizar o fonte STBPayment.prw para uma data superior a 15/02/2016." ) //Gera LOG
1809             STDSPBasket("SL4", "L4_VALOR",oMdlGrd:GetValue("L4_VALOR") - STBGetTroco(),nX)

```

[illegible]

```

1862 For nI := 1 To oModel:Length()
1863   oModel:GoLine(nI)
1864   Do Case
1865     Case IsMoney(AllTrim(oModel:GetValue('L4_FORMA')))
1866       STDSPBasket("SL1", "L1_DINHEIR", oModel:GetValue('L4_VALOR') - STBGetTroco() )
1867     Case AllTrim(oModel:GetValue('L4_FORMA')) == 'CH'
1868       STDSPBasket("SL1", "L1_CHEQUES", oModel:GetValue('L4_VALOR') )//Valores de Cheque ja foram acumulados na rotina STIUpdBask
1869     Case AllTrim(oModel:GetValue('L4_FORMA')) $ 'CC|CD|PD|PX'
1870       IF !STBIsRecovered()
1871         IF AllTrim(oModel:GetValue('L4_FORMA')) == 'CC'
1872           STDSPBasket("SL1", "L1_CARTAO", STDGPBasket( "SL1" , "L1_CARTAO" ) + oModel:GetValue('L4_VALOR') )
1873         ElseIf AllTrim(oModel:GetValue('L4_FORMA')) == 'PD'
1874           STDSPBasket("SL1", "L1_VLRPGDG", STDGPBasket( "SL1" , "L1_VLRPGDG" ) + oModel:GetValue('L4_VALOR') )
1875         ElseIf AllTrim(oModel:GetValue('L4_FORMA')) == 'PX'
1876           STDSPBasket("SL1", "L1_VLRPGPX", STDGPBasket( "SL1" , "L1_VLRPGPX" ) + oModel:GetValue('L4_VALOR') )
1877         Else
1878           STDSPBasket("SL1", "L1_VLRDEBI", STDGPBasket( "SL1" , "L1_VLRDEBI" ) + oModel:GetValue('L4_VALOR') )
1879         EndIf
1880       Else
1881         IF AllTrim(oModel:GetValue('L4_FORMA')) == 'CC'
1882           STDSPBasket("SL1", "L1_CARTAO", oModel:GetValue('L4_VALOR') )
1883         ElseIf AllTrim(oModel:GetValue('L4_FORMA')) == 'PD'
1884           STDSPBasket("SL1", "L1_VLRPGDG", oModel:GetValue('L4_VALOR') )
1885         ElseIf AllTrim(oModel:GetValue('L4_FORMA')) == 'PX'
1886           STDSPBasket("SL1", "L1_VLRPGPX", oModel:GetValue('L4_VALOR') )
1887         Else
1888           STDSPBasket("SL1", "L1_VLRDEBI", oModel:GetValue('L4_VALOR') )
1889         EndIf
1890       EndIf
1891       STDSPBasket("SL1", "L1_VENDTEF", 'S' )
1892       STDSPBasket("SL1", "L1_DATAEF", oModel:GetValue('L4_DATAEF') )
1893       STDSPBasket("SL1", "L1_HORATEF", oModel:GetValue('L4_HORATEF') )
1894       STDSPBasket("SL1", "L1_DOCTEF", oModel:GetValue('L4_DOCTEF') )
1895       STDSPBasket("SL1", "L1_AUTORIZ", oModel:GetValue('L4_AUTORIZ') )
1896       STDSPBasket("SL1", "L1_INSTITU", oModel:GetValue('L4_INSTITU') )
1897       STDSPBasket("SL1", "L1_NSUTEF", oModel:GetValue('L4_NSUTEF') )
1898       STDSPBasket("SL1", "L1_TIPCART", oModel:GetValue('L4_TIPCART') )
1899       STDSPBasket("SL1", "L1_PARCTEF", oModel:GetValue('L4_PARCTEF') )
1900     Case AllTrim(oModel:GetValue('L4_FORMA')) == 'CO'
1901       STDSPBasket("SL1", "L1_CONVENI", STDGPBasket( "SL1" , "L1_CONVENI" ) + oModel:GetValue('L4_VALOR') )
1902     Case AllTrim(oModel:GetValue('L4_FORMA')) == 'VA'
1903       STDSPBasket("SL1", "L1_VALES", STDGPBasket( "SL1" , "L1_VALES" ) + oModel:GetValue('L4_VALOR') )
1904     Case AllTrim(oModel:GetValue('L4_FORMA')) == 'FI'
1905       STDSPBasket("SL1", "L1_FINANC", STDGPBasket( "SL1" , "L1_FINANC" ) + oModel:GetValue('L4_VALOR') )
1906     Case AllTrim(oModel:GetValue('L4_FORMA')) == 'CR'
1907       STDSPBasket("SL1", "L1_CREDITO", oModel:GetValue('L4_VALOR') )//Valores de NCC ja estao acumulados
1908     Case AllTrim(oModel:GetValue('L4_FORMA')) <> 'CR'
1909       STDSPBasket("SL1", "L1_OUTROS", STDGPBasket( "SL1" , "L1_OUTROS" ) + oModel:GetValue('L4_VALOR') )
1910   EndCase
1911   nParcels += oModel:GetValue('L4_PARC')
1912   STDSPBasket("SL1", "L1_PARCELA", nParcels )
1913   STDSPBasket("SL1", "L1_FORMPG", PadR(oModel:GetValue('L4_FORMA'),TamSX3("L1_FORMPG")[1]) )

```

```

1914 // Grava o valor do troco
1915 If lHabTroco .And. oModel:GetValue('L4_TROCO') > 0 .And. ((AllTrim(oModel:GetValue('L4_FORMA')) $ "CH|CD|CC") .Or. (lMvLjTrDin .And.
1916 IsMoney(AllTrim(oModel:GetValue('L4_FORMA')))))
        STDSPBasket("SL1", "L1_TROC01", oModel:GetValue('L4_TROCO'))
1917 EndIf
1918 nValEnt := STBGetEnt() - STBGetTroco()
1919 If nValEnt < 0 // O valor minimo nao pode ser negativo
1920     nValEnt := 0
1921 EndIf
1922 STDSPBasket("SL1", "L1_ENTRADA", nValEnt )
1923 //Realizamos o Reset da variavel Static lRecovered para gravação correta dos campos em caso de mais de uma forma de pagamento.
1924 If STBIsRecovered() .And. AllTrim(oModel:GetValue('L4_FORMA')) $ 'CC|CD|PD|PX' .And. oModel:length() > 1
1925     STB7Recovered() //Retorna a variavel lRecovered para .F.
1926 EndIf
1927 Next nI
1928
1929 //Se existir NCC e o seu valor for maior que o total da venda
1930 //deve ser abatido o troco (condição esta somente quando o parametro MV_LJCPNCC estiver igual a 4
1931 //obrigatoriamente passando pela função STISNCCPayment().
1932 nVlrCredito := STDGPBasket("SL1", "L1_CREDITO")
1933 If nVlrCredito > 0
1934     oTotal := STFGetTot()
1935     If nVlrCredito >= oTotal:GetValue("L1_VLRTOT")
1936         STDSPBasket("SL1", "L1_CREDITO", nVlrCredito - STBGetTroco())
1937         If lHabTroco .And. lMvLjTrDin .And. STBGetTroco() > 0
1938             STDSPBasket("SL1", "L1_TROC01", STBGetTroco() )
1939         EndIf
1940     EndIf
1941 EndIf
1942
1943 STBSetEnt()
1944 Return .T.
1945
1946 ////////////////////////////////////////
1947 /*{Protheus.doc} STIGetLstB
1948 Retorna o objeto ListBox
1949 @param
1950 @return oListTpPaym
1951 @obs
1952 @sample
1953 */
1954 ////////////////////////////////////////
1955 Function STIGetLstB()
1956 Return oListTpPaym
1957
1958 ////////////////////////////////////////
1959 /*{Protheus.doc} STIPayCancel
1960 Retornar a tela das formas de pagamento
```

[illegible]

[illegible]

```

2069 nUsedCards += 1
2070
2071 Return
2072
2073
2074 //-----
2075 /*{Protheus.doc} STISetUsedCard
2076 Faz um reset do numero de cartoes no fim da venda
2077 @param
2078 @return      Nil
2079 @obs
2080 @sample
2081 */
2082 //-----
2083 Function STISetUsedCard()
2084
2085 nUsedCards := 0
2086
2087 Return
2088
2089 //-----
2090 /*{Protheus.doc} STIGetPan
2091 Retorna o objeto oPanBkpPay que é uma copia do oPanPayment
2092 @param
2093 @return      oPanBkpPay
2094 @obs
2095 @sample
2096 */
2097 //-----
2098 Function STIGetPan()
2099 Return oPanBkpPay
2100
2101 //-----
2102 /*{Protheus.doc} STIGetBlCod
2103 Retorna um bloco de codigo para criacao de painel de dados do cheque
2104 @param
2105 @return      bDataCheck
2106 @obs
2107 @sample
2108 */
2109 //-----
2110 Function STIGetBlCod()
2111 Return bDataCheck
2112
2113
2114 //-----
2115 /*{Protheus.doc} STISetPayImp
2116 Set o lPayImport
2117 Variavel controla para nao passar duas vezes no pagamento quando e
2118 importacao de orcamento
2119 @param
2120 @return      .T.

```

```

2121 @obs
2122 @sample
2123 /*/
2124 //-----
2125 Function STISetPayImp(lImp)
2126
2127 If !lImp .AND. (IsInCallStack("STFRESTART") .OR. IsInCallStack("STIExchangePanel"))
2128     STISetPgOr(, .T.)
2129 Endif
2130
2131 lPayImport := lImp
2132
2133 Return .T.
2134
2135 //-----
2136 /*/{Protheus.doc} STIGetPayImp
2137 Retorna o lPayImport
2138 Variavel controla para nao passar duas vezes no pagamento quando e
2139 importacao de orcamento
2140 @param
2141 @return      lPayImport
2142 @obs
2143 @sample
2144 /*/
2145 //-----
2146 Function STIGetPayImp()
2147 Return lPayImport
2148
2149 //-----
2150 /*/{Protheus.doc} STISetaCallADM
2151 Seta o array de cartões a serem chamados para ser informado Adm. Fin.
2152 @param      aSet          Array com as formas a ser chamado Adm. Fin.
2153 @return      lPayImport
2154 @obs
2155 @sample
2156 /*/
2157 //-----
2158 Function STISetaCallADM( aSet )
2159 aCallADM := aSet
2160 Return
2161
2162
2163 //-----
2164 /*/{Protheus.doc} STIGetaCallADM
2165 Retorna o aCallADM
2166 @param
2167 @return      lPayImport
2168 @obs
2169 @sample
2170 /*/
2171 //-----
2172 Function STIGetaCallADM()

```

[illegible]

```

2225 /*/{Protheus.doc} STIRfshVP
2226 Atualiza o campo "Saldo Vale Presente" para exibir ou não o saldo
2227 @param      lUpdVP Para zerar ou não o saldo
2228 @return Nil
2229 @obs
2230 @sample
2231 */
2232 //-----
2233 Function STIRfshVP(lUpdVP)
2234
2235 If Valtype(oLblRestVP) == "0"
2236     If lUpdVP
2237         oLblRestVP:SetText("")
2238     else
2239         oLblRestVP:SetText(STR0027)
2240     Endif
2241 Endif
2242 If Valtype(oLblValVPRest) == "0"
2243     If lUpdVP
2244         oLblValVPRest:SetText("")
2245     else
2246         oLblValVPRest:SetText(Str(STIGetVPSaldo(),10,2))
2247     Endif
2248 Endif
2249
2250 Return Nil
2251
2252
2253 /*/{Protheus.doc} STISetPayR0
2254 Atribui um valor a variavel estática nPayR0Only
2255 @param      nAuxR0Only - valor que será atribuido a variavel estatica
2256 @return      Nil
2257 @obs        [0] reseta a permissão [-1] com escrita [1] somente leitura
2258 @sample
2259 */
2260 //-----
2261 Function STISetPayR0(nAuxR0Only)
2262
2263 Default nAuxR0Only := 0
2264
2265 nPayR0Only := nAuxR0Only
2266
2267 Return Nil
2268
2269
2270 //-----
2271 /*/{Protheus.doc} STIGetPayR0
2272 Indica se os campos de pagamento serão Somente Leitura. Pode retornar uma variavel do tipo N ou L
2273 @param      cVarType - indica se o retorno será Numérico ou Lógico
2274 @return      xRet - retorna um valor Lógico ou Numérico
2275 @obs        0 retorno numérico é util para verificar se a permissão já foi verificada
2276 @sample      STIGetPayR0() / STIGetPayRo("N")

```

```

2277 /*/
2278 //-----
2279 Function STIGetPayR0(cVarType)
2280
2281 Local xRet := .F.
2282
2283 Default cVarType := "L"
2284
2285 If cVarType == "L"
2286
2287     If nPayROnly > 0
2288         xRet := .T.      //Somente Leitura
2289     Else
2290         xRet := .F.      //Escrita
2291     EndIf
2292
2293 ElseIf cVarType == "N"
2294
2295     xRet := nPayROnly
2296
2297 EndIf
2298
2299 Return xRet
2300
2301 //-----
2302 /*/{Protheus.doc} STIPayMdlc
2303 Funcao para limpar o model e outros objetos da interface de pagamentos.
2304 @return      Nil
2305 /*/
2306 //-----
2307 Function STIPayMdlc()
2308
2309 If ValType(oPnlAdconal) == '0'
2310     LjGrvLog("STIPayMdlc","eliminar o objeto o oPnlAdconal ")
2311     oPnlAdconal:FreeChildren()
2312     FreeObj( oPnlAdconal )
2313     oPnlAdconal:=Nil
2314 Endif
2315
2316 If Valtype(oListTpPaym) == "0"
2317     LjGrvLog("STIPayMdlc","eliminar o objeto oListTpPaym ")
2318     oListTpPaym:FreeChildren()
2319     FreeObj( oListTpPaym )
2320     oListTpPaym:=Nil
2321 Endif
2322
2323 If Valtype(oListResPg) == "0"
2324     LjGrvLog("STIPayMdlc","eliminar o objeto oListResPg ")
2325     oListResPg:FreeChildren()
2326     FreeObj( oListResPg )
2327     oListResPg:=Nil
2328 Endif

```

```

2329
2330 If ValType(oPanPayment) == '0'
2331     LjGrvLog("STIPayMdlc","eliminar o objeto oPanPayment ")
2332     oPanPayment:FreeChildren()
2333     FREEOBJ( oPanPayment )
2334     oPanPayment:= Nil
2335 Endif
2336
2337 If ValType(oModel) == '0'
2338     LjGrvLog("STIPayMdlc","eliminar o objeto oModel ")
2339     oModel:DeActivate()
2340     FREEOBJ(oModel)
2341     oModel := Nil
2342     SetKey(16, Nil)
2343 EndIf
2344
2345 //Limpa os objetos de interface do pagamento
2346 If ExistFunc("STIClrObj")
2347     STIClrObj()
2348 Endif
2349
2350 aFormMultNeg := {}
2351
2352 Return Nil
2353
2354 //-----
2355 /*{Protheus.doc} STILmpTef
2356 Funcao para limpar o model de pagamentos quando TEF não pode apagar
2357
2358 @return      Nil
2359 */
2360 //-----
2361 Function STILmpTef(oParc,oPaym,lBonif)
2362 Local nX, nI      := 0
2363 Local lRet        := .T.
2364 Local aStruct     := {}
2365 Local aFormPg     := {}
2366
2367 Default oParc     := oModel:GetModel('PARCELAS') //Model de parcelas
2368 Default oPaym     := oModel:GetModel('APAYMENTS') //Model de pagamentos
2369 Default lBonif    := .F.                        //Se for bonificação
2370
2371 If lBonif
2372     STBFormBf()
2373 EndIf
2374
2375 aStruct := oParc:GetStruct():aFields
2376 For nX := 1 to oParc:Length()
2377     oParc:GoLine(nX)
2378     If AllTrim(oParc:GetValue('L4_FORMA')) $ "CC|CD"
2379         Aadd(aFormPg , {})
2380         aFormPg[Len(aFormPg)] := Array(Len(aStruct),2)

```

```

2381         For nI := 1 To Len(aStruct)
2382             aFormPg[Len(aFormPg)][nI][1] := AllTrim(aStruct[nI][3])
2383             aFormPg[Len(aFormPg)][nI][2] := oParc:GetValue(aStruct[nI][3])
2384         Next nI
2385     EndIf
2386 Next nX
2387
2388 oParc:ClearData()
2389 oPaym:ClearData()
2390 oParc:InitLine()
2391 oPaym:InitLine()
2392
2393 For nX := 1 To Len(aFormPg)
2394     If nX > 1
2395         oParc:AddLine(.T.)
2396         oPaym:AddLine(.T.)
2397     EndIf
2398
2399     For nI:= 1 to Len(aFormPg[nX])
2400         oParc:LoadValue(aFormPg[nX][nI][1], aFormPg[nX][nI][2])
2401         oPaym:LoadValue(aFormPg[nX][nI][1], aFormPg[nX][nI][2])
2402     Next nI
2403 Next nX
2404
2405 STIBlqTlTef()
2406 STIMBlPTef()
2407
2408 Return lRet
2409
2410 //-----
2411 /*{Protheus.doc} STIBlqTlTef
2412 Funcao para bloquear os botes ao finalizar a venda TEF
2413 que possua o param MV_TEFPEND impedindo apagar a forma
2414
2415 @return      Nil
2416 */
2417 //-----
2418 Function STIBlqTlTef()
2419 Local lRet      := .T.
2420 Local cMV_TEFPEND := AllTrim(SuperGetMv("MV_TEFPEND",,"0"))
2421
2422 If STIAtvTefP() .And. cMV_TEFPEND <> "0"
2423     STIBtnDeActivate()
2424 EndIf
2425
2426 Return lRet
2427
2428
2429 //-----
2430 /*{Protheus.doc} STIBlqMnTef
2431 Função para verificação se tem TEF e se permite excluir a venda
2432 já efetuada ou também se prossegue com acesso aos menus (STBSalesOperations)

```

```

2433
2434 @return      lRet, logico, pode excluir (S) ou (N)?
2435 */
2436 //-----
2437 Function STIBlqMnTef()
2438 Local lRet      := .T.
2439 Local cMV_TEFPEND := AllTrim(SuperGetMv("MV_TEFPEND",,"0"))
2440
2441 If STIAtvTefP() .And. cMV_TEFPEND <> "0" .And. STIGetCard()
2442     lRet := .F.
2443 EndIf
2444
2445 Return lRet
2446
2447 //-----
2448 /*{Protheus.doc} STIMBlPTef
2449 Funcao para emitir mensagem sobre a utilização do parametro
2450 MV_TEFPEND, no momento que o sistema impede o prosseguimento
2451 ou acesso a outras rotinas.
2452
2453 @return      cMsg, caracter, mensagem emitida
2454 */
2455 //-----
2456 Function STIMBlPTef(cType)
2457 Local cMsg      := ""
2458 Local cRotina   := ProcName()
2459 Local cMV_TEFPEND := AllTrim(SuperGetMv("MV_TEFPEND",,"0"))
2460
2461 Default cType := "POPUP"
2462
2463 cMsg := STR0038 + cMV_TEFPEND +; ///<0 parametro MV_TEFPEND está configurado com "
2464 STR0039 + CHR(13) +; ///< conforme documentação não será permitido limpar todas as formas/acesso a outras telas/incluir condição negociada ou multinegociação pois
2465 existe vendas do tipo TEF (CC/CD) aprovadas! "
2466 STR0040 ///<"Por favor selecione: uma nova forma de pagamento/cancele ou encerre a venda / reconfigurar o parametro MV_TEFPEND."
2467
2467 STFMessage(cRotina,cType,cMsg)
2468 STFSShowMessage(cRotina)
2469
2470 Return cMsg
2471
2472 //-----
2473 /*{Protheus.doc} STIVerCTef
2474 Valida se pode limpar o TEF
2475
2476 @param      lEmitMsg , lógico , emite mensagem de explicação
2477 @return      lRet, logico, se continua ou não
2478 */
2479 //-----
2480 Function STIVerCTef(lEmitMsg)
2481 Local lRet := .T.
2482
2483 Default lEmitMsg := .T.

```

```

2484
2485 If STIAtvTefP() .And. !STIBlqMnTef()
2486     lEmiteMsg := lEmiteMsg .And. !IsInCallStack("STIRInPay")
2487
2488     If lEmiteMsg
2489         STIMBlPTef()
2490     EndIf
2491
2492     lRet := .F.
2493     STIPayCancel(oPnlAdconal)
2494 EndIf
2495
2496 Return lRet
2497
2498 //-----
2499 /*{Protheus.doc} STIAtvTefP
2500 Validação para data compatível de todos os fontes para que o
2501 parâmetro MV_TEFPEND possa ser usado
2502
2503 @return     lRet, logico, se continua ou não
2504 */
2505 //-----
2506 Function STIAtvTefP()
2507 Local lRet := .T.
2508 Local dDataF:= Ctod("22/02/2017")
2509
2510 If lSTIAtvTefP == NIL
2511     lRet := GetApoInfo("STIDATACHECK.PRW")[4] >= dDataF .And.;
2512     GetApoInfo("STBPAYCARD.PRW")[4] >= dDataF .And.;
2513     GetApoInfo(Upper("STBSalesOperations.PRW"))[4] >= dDataF .And.;
2514     GetApoInfo("STICANCELSALE.PRW")[4] >= dDataF .And.;
2515     GetApoInfo(Upper("STWPAYCdPg.PRW"))[4] >= dDataF
2516
2517     lSTIAtvTefP := lRet
2518 EndIf
2519
2520 lRet := lSTIAtvTefP
2521
2522 If !lRet
2523     LjGrvLog("STIAtvTefP", "Para ativação do MV_TEFPEND atualize os seguintes fontes com data" +;
2524         " igual ou superior a 21/02/2017 : STIPayment , STIDataCheck , STBPayCard, STBSalesOperations ," +;
2525         " STICancelSale e STWPAYCdPg")
2526 EndIf
2527
2528 Return lRet
2529
2530
2531 //-----
2532 /*{Protheus.doc} STISetPgOr
2533 Controla as formas de pagamentos do Orçamento, quando na permissão de caixa está bloqueado mas quando incluir mais itens durante a importação
2534 deve permitir incluir forma de pagamento para pagar, porém não pode permitir alterar forma de pagamento do orçamento.
2535

```

```

2536 @param      cForma - forma de pagamento a ser adicionada ou excluída do array (depende do parametro lClear)
2537 @param      lClear - .T., limpa o array
2538 @return      nil
2539 @obs
2540 @sample
2541 /*/
2542 //-----
2543
2544 Function STISetPgOr(cForma, lClear )
2545 Local nPos := 0
2546
2547 Default cForma := ""
2548 Default lClear := .F.
2549
2550 IF !STFPROFILE(41)[1]
2551
2552     If lClear
2553         If !Empty(cForma)
2554             nPos := Ascan(aFormasORC,{|x| x[1] == cForma })
2555             If nPos > 0
2556                 ADel( aFormasORC,nPos )
2557                 ASize( aFormasORC, Len(aFormasORC) -1 )
2558             Endif
2559         Else
2560             aFormasORC :={}
2561         Endif
2562     Else
2563         aadd(aFormasORC,{cForma})
2564     Endif
2565
2566 Endif
2567
2568 Return
2569
2570 //-----
2571 /*/{Protheus.doc} STIGetPgOr
2572 Controla as formas de pagamentos do Orçamento, quando na permissão de caixa está bloqueado mas quando incluir mais itens durante a importação
2573 deve permitir incluir forma de pagamento para pagar, porém não pode permitir alterar forma de pagamento do orçamento.
2574
2575 @return      retorna o array aFormasORC
2576 @obs
2577 @sample
2578 /*/
2579 //-----
2580
2581 Function STIGetPgOr()
2582
2583 Return aFormasORC
2584
2585
2586 /*/{Protheus.doc} STIRestVlr()
2587     Restaura os valores dos totais caso for chamado STIAddNewPan() atraves do duplo click enquanto oPnlAdicional estiver visivel.

```

```

2588     Somente válido quando estiver com Regra de Desconto Varejo Ativo.
2589     @type Static Function
2590 */
2591 Static Function STIRestVlr()
2592 Local lTemPagto      := ExistFunc("STBGetPgto") .AND. STBGetPgto()>0
2593
2594 If oPnlAdconal <> NIL .AND. oPnlAdconal:LVISIBLE .AND. SuperGetMv("MV_LJRGDES",,.F.) .AND. !STIGetRecTit() .AND. STIDescRegVar() .AND. lTemPagto
2595     STIRestVlr()
2596 EndIf
2597
2598 Return
2599
2600
2601 *//{Protheus.doc} STIPermTrc
2602     Rotina responsavel por retornar se a forma de pagamento permite dar troco ou não de acordo com parametro MV_LJPGTR0
2603     @type Function
2604     @param cForma, caractere, forma de pagamento
2605     @return lRet, lógico, se .T. a forma de pagamento permite dar troco
2606     */
2607 Function STIPermTrc(cForma)
2608 Local lRet := (cForma $ cMV_LJPGTR0)
2609
2610 If !lRet
2611     cMsg:= Iif(Len(aCopyPaym)>0, Alltrim(aCopyPaym[aScan(aCopyPaym,{ |x| x[1] == cForma }))[3]), cForma)
2612     STFMessage(ProcName(),"POPUP", STR0034 + cMsg+ " "+STR0046)//não permite Troco, favor rever o valor!
2613     STFShowMessage(ProcName())
2614     STFCleanMessage(ProcName())
2615 Endif
2616
2617 Return lRet
2618
2619 *//{Protheus.doc} STICallSFP
2620 Faz a chamada do Ponto de Entrada STIPAYSFP caso esteja compilado
2621 O PE STIPAYSFP é chamado na Seleção da Forma de Pagamento
2622 @type Static Function
2623 @return lRet, logico, Autoriza ou não o uso da FOrma de Pagamento
2624 */
2625 Static Function STICallSFP()
2626 Local lRet      := .T.
2627 Local nVlrTot   := STDGPBasket('SL1','L1_VLRTOT') // Valor total da venda
2628 Local cCliente := STDGPBasket('SL1','L1_CLIENTE') // Cliente
2629 Local cLojaCli := STDGPBasket('SL1','L1_LOJA') // Loja Cliente
2630 Local cVend     := STDGPBasket('SL1','L1_VEND') // Vendedor
2631 Local cUserCx   := STDGPBasket('SL1','L1_OPERADO') // Usuario caixa
2632 Local cFormPg   := aCopyPaym[aScan(aCopyPaym,{ |x| x[2] == AllTrim(oListTpPaym:GetSelText()) }))[1] // Forma de Pagamento selecionada
2633 Local aPagtosAnt := {} // Formas de pagamentos selecionadas anteriormente
2634 Local oMdl       := NIL // Model dos pagamentos
2635 Local oModel     := Nil // Modelo de Parcelas
2636 Local nX         := 0 // Variavel de loop
2637
2638 //Ponto de Entrada que permite customizar o painel de pagamentos da condicao de pagamento especifica
2639 If ExistBlock("STIPAYSFP")

```

```

2640
2641     oMdl := STISetMdlPay() //Retorna o model dos pagamentos
2642
2643     If ValType(oMdl) == "0"
2644         oModel := oMdl:GetModel('APAYMENTS') // Pega o modelo de Parcelas
2645         For nX := 1 To oModel:Length()
2646             oModel:GoLine(nX)
2647             If !oModel:IsDeleted() .AND. !Empty(oModel:GetValue("L4_FORMA"))
2648                 AADD(aPagtosAnt,{oModel:GetValue("L4_FORMA"),oModel:GetValue("L4_VALOR")})
2649             EndIf
2650         Next nX
2651     EndIf
2652
2653     LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "Antes da chamado do PE STIPAYSFP " )
2654     lRet := ExecBlock("STIPAYSFP",.F.,.F., {nVlrTot,cCliente,cLojaCli,cVend,cUserCx,cFormPg,aPagtosAnt})
2655     LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "Antes da chamado do PE STIPAYSFP ", lRet )
2656 EndIf
2657
2658 Return lRet
2659
2660 /*/{Protheus.doc} nomeFunction
2661 Chamma o Ponto de Entrada STISelForm
2662 @type Function
2663 @param cForma, caracter, Forma de Pagamento selecionada
2664 nParc, caracter, Parcelas
2665 nValor, caracter, Valor
2666 @return lRet, logico, Permite ou não prosseguir com a Forma de Pagamento
2667 */
2668 Function STICallPSF(cForma, nParc, nValor)
2669 Local lRet := .T.
2670 Local oMdl := oModel // Recupera o model ativo
2671 Local oMdlGrd := oMdl:GetModel("PARCELAS")// Seta o model do grid
2672 Local nX := 0
2673 Local aPagtosAnt := {} // Formas de pagamentos selecionadas anteriormente
2674 Local aPgOrcImp := {"",0,0} // Formas de pagameto do Orcamento importado
2675 Local lIsMultNeg := .F. // Indica se existe multinegociacao na venda
2676
2677 If !Empty(cForma) .AND. ExistBlock("STISelForm")
2678     lIsMultNeg := STIGetMult()
2679
2680     If ValType(oMdlGrd) == "0"
2681         For nX := 1 To oMdlGrd:Length()
2682             AADD(aPagtosAnt,{AllTrim(oMdlGrd:GetValue('L4_FORMA')),oMdlGrd:GetValue('L4_VALOR'),oMdlGrd:GetValue('L4_PARC')})
2683         Next nX
2684     EndIf
2685
2686     If FindFunction("GetOrcPOri")
2687         aPgOrcImp := GetOrcPOri()
2688     EndIf
2689
2690
2691

```

```

2692 LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "Antes de chamar o Ponto de Entrada STISelForm" )
2693 lRet := ExecBlock("STISelForm",.F.,.F.,{cForma, nParc, nValor, aPagtosAnt, aPgOrcImp, lIsMultNeg})
2694 LjGrvLog( "L1_NUM: "+STDGPBasket('SL1','L1_NUM'), "Depois de chamar o Ponto de Entrada STISelForm", lRet )
2695 EndIf
2696
2697 Return lRet
2698
2699 /*{Protheus.doc} STICriaPnlAdconal
2700     rotina criada para definir CSS do tPanel utilizado para informação de dados das formas de pagamentos
2701     @type Static Function
2702     @param bConstrutor, bloco de código, bloco de código para criar oPnlAdconal
2703     @return oPnlAdconal, tpanel, objeto oPnlAdconal criado
2704 */
2705 Static Function STICriaPnlAdconal(bConstrutor)
2706
2707 STIResPayObj(.F.)
2708
2709 oPnlAdconal:= Eval(bConstrutor)
2710 oPnlAdconal:SetCSS( POSCSS (GetClassName(oPnlAdconal), CSS_PANEL_CONTEXT ))
2711 LjGrvLog("STICriaPnlAdconal" , "Constroi oPnlAdconal Hparent: "+ Str(oPnlAdconal:Hparent) + " oPanel2 Hwnd : " + Str(oPnlAdconal:Hwnd) )
2712
2713 Return oPnlAdconal
2714
2715
2716 /*{Protheus.doc} STIResPayObj
2717     Rotina Static para controle de show/hide dos objetos de Resumo de pagamento
2718     contorno para interface webapp devido ao appsever criar panel dos pagamentos com id errado
2719     por isso o panel fica para tras desse.
2720     @type Static Function
2721     @param lShow, logico, .T. Show, .F. Hide
2722 */
2723 Static Function STIResPayObj(lShow)
2724
2725 Iif (Valtype(oListResPg)=="0", Iif(lShow, oListResPg:Show(), oListResPg:Hide()), nil)
2726 Iif (Valtype(oLblResPay)=="0", Iif(lShow, oLblResPay:Show(), oLblResPay:Hide()), nil)
2727 Iif (Valtype(oLblForma)=="0", Iif(lShow, oLblForma:Show(), oLblForma:Hide()), nil)
2728 Iif (Valtype(oLblValor)=="0", Iif(lShow, oLblValor:Show(), oLblValor:Hide()), nil)
2729 Iif (Valtype(oLblParcelas)=="0", Iif(lShow, oLblParcelas:Show(), oLblParcelas:Hide()), nil)
2730 Iif (Valtype(oLblRest)=="0", Iif(lShow, oLblRest:Show(), oLblRest:Hide()), nil)
2731 Iif (Valtype(oLblTroco)=="0", Iif(lShow, oLblTroco:Show(), oLblTroco:Hide()), nil)
2732 Iif (Valtype(oButton)=="0", Iif(lShow, oButton:Show(), oButton:Hide()), nil)
2733 Iif (Valtype(oBtnLimpar)=="0", Iif(lShow, oBtnLimpar:Show(), oBtnLimpar:Hide()), nil)
2734 Iif (Valtype(oLblValRest)=="0", Iif(lShow, oLblValRest:Show(), oLblValRest:Hide()), nil)
2735 Iif (Valtype(oLblValTroco)=="0", Iif(lShow, oLblValTroco:Show(), oLblValTroco:Hide()), nil)
2736
2737 Return
2738
2739 //-----
2740 /*{Protheus.doc} STIGetFormMN
2741 Retorna o aFormMultNeg
2742 @param
2743 @return aFormMultNeg

```

```
2744 @obs
2745 @sample
2746 /*/
2747 //-----
2748 Function STIGetFormMN()
2749 Return (aFormMultNeg)
```

Componentes/Interface/STIPosMain.prw

2005 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "FWBROWSE.CH"
3 #INCLUDE "POSCSS.CH"
4 #INCLUDE "STIPOSMAIN.CH"
5 #INCLUDE "STPOS.CH"
6
7 Static oDlg
8 Static oView
9 Static oViewDetail
10 Static oViewRight
11 Static oGetDiscount
12 Static oGetSubTot
13 Static oGetQtde
14 Static oGetQtdBrinde
15 Static oSayTotal
16 Static oBrowse
17 Static oArea
18 Static oPOSDesk
19 Static oMdlPrin
20 Static oPanelMVC
21 Static oPanel1
22 Static oPanel2
23 Static nPanelActive := 0
24 Static lBtnActive := .F.
25 Static lHomolPaf := STBHomolPaf() //Homologação PAF-ECF
26
27 Static oBtnRegItem := Nil
28 Static oBtnClient := Nil
29 Static oBtnCancItem := Nil
30 Static oBtnOrc := Nil
31 Static oBtnDesc := Nil
32 Static oBtnConProd := Nil
33
34 Static nAtivo := CSS_BTN_ATIVO
35 Static nNormal := CSS_BTN_NORMAL
36
37 Static cOperation := "" //Operacao em execucao atual
38 Static cMsgPg := "" //Mensagem que retorna no final da funcao
39 Static oSayPag := Nil //Objeto Tsay de paginacao
40 Static oSemaforo := Nil //Objeto semaforo de conexao
41 Static cLastPnlOpen := "" //Guarda ultimo panel chamado na troca de panels
42
43 Static lPdvOn := ExistFunc("STFPdvOn") .AND. STFPdvOn()
44 Static lClientHtml := GetCliHTML()
45
46 //-----
47 /*{Protheus.doc} STIPosMain
48 Chamada principal para a tela do POS
49
50 @param
51 @return
52 @obs

```

```

53 @sample
54 /**
55 //-----
56 Function STIPosMain()
57
58 Local   aSemana           := {STR0001,STR0002, STR0003, STR0004, STR0005, STR0006, STR0007} // "Domingo","Segunda-Feira", "Terça-Feira", "Quarta-Feira", "Quinta-Feira",
    "Sexta-Feira", "Sábado"
59 Local   oTimer            := NIL // Objeto Timer
60 Local   nInterval         := 60000 // 60.000 milisegundos 1 em 1 min
61 Local   cCor               := SuperGetMv( "MV_LJCOLOR",,"07334C") // Cor da tela
62 Local   aMenus             := {} // Itens do menu F2
63 Local   nI                 := 0 // Variavel de loop
64 Local   lOnlyMenuFiscal    := .F. // Se somente Menu Fiscal
65 Local   lPafEcf            := .F. // Habilitar PAF-ECF
66 Local   lTestCon           := IIf( ExistFunc("LjGetCPDV"), LjGetCPDV()[2] , .F. ) // Testar conexao
67 Local   lMvTestCon         := SuperGetMv( "MV_LJTESTC",,.F. ) // Verifica se faz ou nao o teste de conexao
68 Local   nTimeD             := 60 * 1000 /*segundos*/ * 60 /*minutos*/ * 4 /*horas*/ //Todo: parametro de subida de dados 4 horas
69 Local   nTimeU             := 15 * 1000 /*segundos*/ * 60 /*minutos*//* 60 /*horas*/ //Todo: paramero de subida de dados 30 minutos
70 Local   cIntegration       := "" //Verifica a Integracao
71 Local   lMobile            := .F. //Verifica se é mobile
72 Local   lIsPafNfce         := .F.
73
74 //Seta que o TOTVS PDV é Online
75 If ExistFunc("STFPdvOn")
76     STFSetsOnPdv(STFPdvOn())
77 EndIf
78
79 //Valida o tamanho da tela.
80 aRes := GetScreenRes()
81
82 If ExistFunc("LjxAjSX") //Função para alteração de dicionários
83     LjxAjSX()
84 EndIf
85
86 /* Inicialização */
87 If !STFStart()
88     ClearGlbValue("lConnHost")
89     ClearGlbValue("lPosOpen")
90     Return
91 EndIf
92
93
94 LjGrvLog(/*cNumControl*/, "TOTVS PDV INICIADO",/*xVar*/)//Grava Log =====
95
96 lOnlyMenuFiscal := STFGetCfg( "lOnlyMenuFiscal" )
97 lPafEcf          := STFGetCfg("lPafEcf") // PAF-ECF
98 lIsPafNfce       := STFGetCfg("lPAFNfce")
99
100 LjGrvLog(/*cNumControl*/, "PAF-ECF",lPafEcf)//Grava Log =====
101 LjGrvLog(nil , "PDV-ONLINE",lPdvOn)//Grava Log =====
102
103 // Abre a tela em fullscreen

```

```

104 FWVldFullScreen()
105
106 /* Carregar menus somente apos o STFStart */
107 aMenus := STBLst0perCashier()
108
109 POSBaseColor( cCor )
110
111 cIntegration := STFGetCfg( "cIntegration" , IIF(!SuperGetMv("MV_LJ RMS",,.F.), "DEFAULT", "RMS" )) //Verifica a Integracao
112 If ValType(cIntegration) <> "C"
113     cIntegration := "DEFAULT"
114 EndIf
115 LjGrvLog(*cNumControl*,"Tipo de Integração",cIntegration)//Grava Log =====
116
117 lMobile := STFGetCfg("lMobile", .F. ) //Verifica se é mobile
118
119 lMobile := ValType(lMobile) = "L" .AND. lMobile
120 LjGrvLog(*cNumControl*,"Modo mobile",lMobile)//Grava Log =====
121
122 //Verifica se fluxo de execucao de rotinas monitoradas foi interrompido
123 If FindFunction("STWFlowControl")
124     STWFlowControl()
125 EndIf
126
127 DEFINE DIALOG oDlg TITLE STR0008 PIXEL STYLE nOr(WS_VISIBLE,WS_POPUP) //Grafico Radar
128
129 oDlg:nWidth := aRes[1]
130 oDlg:nHeight := aRes[2]
131
132 oPOSDesk := POSDesktop():New( oDlg )
133
134 // Monta Menu principal
135 oPOSDesk:SetTLMenu( STR0009, VK_F2 ) // "Menu"
136
137 //Se nao for acesso soh ao menu fiscal carrega menus
138 If !STFGetCfg("lOnlyMenuFiscal")
139     For nI := 1 To Len(aMenus)
140         If Len(aMenus[nI]) > 4
141             oPOSDesk:AddTLMenuItem( aMenus[nI][2], &('{||' + 'STIPoS0pc("' + aMenus[nI][5] + '"')' + ', ' + aMenus[nI][3] + '}') )
142
143             Else
144                 oPOSDesk:AddTLMenuItem( aMenus[nI][2], &('{||' + aMenus[nI][3] + '}') )
145             EndIf
146         Next nI
147     Else
148         For nI := 1 To Len(aMenus)
149             If aMenus[nI][4] == "M" //item do Menu fiscal/Offline
150                 oPOSDesk:AddTLMenuItem( aMenus[nI][2], &('{||' + aMenus[nI][3] + '}') )
151             EndIf
152         Next nI
153     EndIf
154
155     oPOSDesk:AddTLMenuItem( STR0010, {|| STFExit() ,oDlg:End() } ) // "Sair"

```

```

156
157   If lMvTestCon .AND. cIntegration <> "FIRST"
158       MsgRun(STR0027, '', {|| STFTTestRemoteExecute(lTestCon)})
159   EndIf
160
161   // Monta menu no canto superior direito
162   If lPAFECF .OR. lIsPafNfce
163       oPOSDesk:SetTRMenu( STR0011, VK_F12, {|| STBMenFis(.T., .T., !lOnlyMenuFiscal) } )   //"Menu Fiscal"
164   Else
165       oPOSDesk:SetTRMenu( STR0012 , VK_F12 , {|| .T. } )   //"TOTVS PDV"
166   EndIf
167
168   //Seta informacoes gerais na tela
169   oPOSDesk:SetTInfo( iif(lPdvOn,'PDV-ONLINE'+ " | ",",") +STR0013 + " " + cEstacao + " | " + STR0014 + " " + cUserName )   //"PDV:" ... "Operador:"
170
171   //Em versoes Mobile nao exibe essa opcao. Somente no Menu superior
172   If ExistFunc("STFLatestMsg") .AND. !lMobile
173       oPOSDesk:SetBInfo( STR0043 + Space(10) + aSemana[Dow(Date())] + " " + DToC(Date()) + " | " + Left(Time() ,5) )
174   Else
175       oPOSDesk:SetBInfo(Space(10) + AllTrim(aSemana[Dow(Date())]) + " " + AllTrim(DToC(Date()))) + " | " + AllTrim(Left(Time() ,5)) )
176   EndIf
177
178
179   oPOSDesk:Activate()
180
181   //Inicializa com a mensagem em branco
182   oPOSDesk:SetBAlert( "" )
183
184   If lMobile
185
186       oBarBottom := oPOSDesk:GetPanelLogoBottom()
187
188       //Versao demonstrativa
189       //Exibe label especifico
190       If STFTTypeOperation() == "DEMONSTRACAO"
191           @ 003, 013  BUTTON oBtnBuy PROMPT STR0044 + CRLF + STR0045  SIZE 50,015  ACTION {|| FwAlertInfo( STR0045 + ": fly01.com.br") } OF oBarBottom  PIXEL //"SAIBA
192           COMO" ### "CONTRATAR" ### "Saiba como contratar em"
193           oBtnBuy:Align := CONTROL_ALIGN_RIGHT
194           oBtnBuy:SetCss(POSCSS( GetClassName(oBtnBuy), CSS_BTN_BUY) )
195           oBarBottom:Refresh()
196       Else
197           //Mobile Default não exibe label inferior
198           oBarBottom:Hide()
199       EndIf
200   EndIf
201
202   oView := ViewDef()
203   oView:SetOwner( oPOSDesk:GetDesktop() )
204   oView:SetUseCursor(.F.)
205
206   oTimer := TTimer():New(nInterval, {|| STITimeMsg(oPOSDesk, AllTrim(aSemana[Dow(Date())]) ) , lMobile }, oDlg )

```

```

207 oTimer:Activate()
208
209 If cIntegration == "FIRST" .and. FindFunction("STWDownData")
210 oTimerD := TTimer():New(nTimeD, {|| ConOut("Iniciando Job STWDownData " + Dtoc(Date()) + " " + Time() ), StartJob("STWDownData", GetEnvServer(), .T., cEmpAnt,
cFilAnt, STFGetStat("CODIGO"), "5", "T" ) }, oDlg )
211 oTimerD:Activate()
212 EndIf
213
214 If cIntegration == "FIRST" .and. FindFunction("STWUpData")
215 oTimerD := TTimer():New(nTimeU, {|| ConOut("Iniciando Job STWUpData " + Dtoc(Date()) + " " + Time()), StartJob("STWUpData", GetEnvServer(), .T., cEmpAnt, cFilAnt,
STFGetStat("CODIGO"))}, oDlg )
216 oTimerD:Activate()
217
218
219 EndIf
220
221 ACTIVATE DIALOG oDlg CENTER ON INIT ( oView:Activate(), STIniPos(), IIF(STIShowPagGrid(),STIMsgPag(),) ) VALID IIF(STFExit(), STIPosClDsk(), .F.)
222
223 Return .T.
224
225
226 //-----
227 Static Function STIniPos()
228
229 //Verifica a redução Z pendente
230 If STFGetCfg("lUseECF") .and. !STFGetCfg( "lOnlyMenuFiscal" )
231 aRet := STFFireEvent( ProcName(0) ,; // Nome do processo
232 "STChkZReduction" ,; // Nome do evento
233 { .T. } )
234 EndIf
235
236
237 Return
238
239 //-----
240 /*{Protheus.doc} STIBtnActivate
241 Ativa os botoes das operacoes e seta os respectivos atalhos
242
243 @param
244 @return
245 @obs
246 @sample
247 */
248 //-----
249 Function STIBtnActivate()
250
251 Local lAtvCnsPro := SuperGetMV("MV_LJTPSCH", .F., 0) <> 2 //0= legado; 1= codigo de barras; 2= bloqueia
252
253 /* Registro de item */
254 SetKey(VK_F3, {|| STIRegItemInterface() })
255
256 /* Selecao de cliente */

```

```

257 SetKey(VK_F4, {|| STICustomerSelection()})
258
259 /* Cancelamento de item */
260 SetKey(VK_F5, { || STICancelItem() } )
261
262 /* Importar orcamento */
263 SetKey(VK_F6, { || STIImportSale() } )
264
265 /* Desconto na venda */
266 SetKey(VK_F7, { || STIDiscountTotal() } )
267
268 /* Consulta de Produto */
269 If lAtvCnsPro
270     SetKey(VK_F8, { || STIProdSearch() } )
271 Endif
272 /* Histórico de mensagens - LOG */
273 If FindFunction("STFLatestMsg")
274     SetKey(VK_F10, { || STFLatestMsg() } )
275 EndIf
276
277 lBtnActive := .T.
278
279 Return
280
281 //-----
282 /*{Protheus.doc} STIBtnDeActivate
283 Remove os botoes das operacoes e remove os respectivos atalhos
284
285 @param
286 @return
287 @obs
288 @sample
289 */
290 //-----
291 Function STIBtnDeActivate()
292
293 Local lAtvCnsPro      := SuperGetMV("MV_LJTPSCH", .F., 0) <> 2 //0= legado; 1= codigo de barras; 2= bloqueia
294
295 /* Registro de item */
296 SetKey(VK_F3, Nil )
297
298 /* Selecao de cliente */
299 SetKey(VK_F4, Nil )
300
301 /* Cancelamento de item */
302 SetKey(VK_F5, Nil )
303
304 /* Importar orcamento */
305 SetKey(VK_F6, Nil )
306
307 /* Desconto na venda */
308 SetKey(VK_F7, Nil )

```

```

309
310 /* Consulta de Produto */
311 If lAtvCnsPro
312     SetKey(VK_F8, Nil )
313 Endif
314
315 lBtnActive := .F.
316
317 Return
318
319 //-----
320 /*{Protheus.doc} ModelDef
321 Model para a criacao da tela principal
322
323 @param
324 @return
325 @obs
326 @sample
327 */
328 //-----
329 Static Function ModelDef()
330
331 Local oStruct0 //Estrutura do model
332 Local oStruDt1 //Estrutura do grid
333
334 oStruct0 := FWFormModelStruct():New()
335 oStruct0:AddTable( "XXW", {"XXW_CODIGO"}, "Cadastro de Dashboard" )
336 oStruct0:AddIndex( 1,"01","XXW_CODIGO","Código","",",",.T. )
337 oStruct0:AddField( "Código", "Código do Dashboard", "XXW_CODIGO", "C", 6, 0,/*bValid*/,/*bWhen*/,/*aValues*/,.T./*lObrigat*/,/*bInit*/,.T./*lKey*/,.T./*lNoUpd*/,/*lVirtual*/)
338
339 oMdlPrin:= MPFormModel():New("APCFG080",/*Pre-Validacao*/,/*Pos-Validacao*/)
340 oMdlPrin:AddFields("APCFG080_DASH", NIL, oStruct0,/*Pre-Validacao*/,/*Pos-Validacao*/,/*Carga*/{|| {}})
341
342
343 oStruDt1 := StructCupomVirtual()
344 oMdlPrin:AddGrid('CUP_GRID','APCFG080_DASH',oStruDt1)
345
346 oMdlPrin:SetOnlyQuery( "APCFG080_DASH" )
347 oMdlPrin:SetOperation(4)
348
349 Return oMdlPrin
350
351 //-----
352 /*{Protheus.doc} ViewDef
353 View principal do POS
354
355 @param
356 @return
357 @obs
358 @sample
359 */
360 //-----

```

```

361 Static Function ViewDef()
362
363 oView := FWFormView():New()
364
365 oView:Setmodel(ModelDef())
366
367 oView:CreateHorizontalBox( 'BOX3', 100)
368
369 //-----
370 // Área de operação
371 //-----
372 oView:CreateVerticalBox( 'BOX4', 50, 'BOX3')
373 //-----
374 // OtherObject Area do Logo
375 //-----
376 oView:AddOtherObject('OTHEROBJECT6',{|o| STILogo(o) },,)
377 oView:CreateHorizontalBox( 'BOX_OBJECT6', 130, 'BOX4', .T.)
378 oView:SetOwnerView('OTHEROBJECT6','BOX_OBJECT6')
379
380 //-----
381 // OtherObject Menu de Botões
382 //-----
383 oView:AddOtherObject('OTHEROBJECT7',{|o| STIPnlAcoes(o) },,)
384 oView:CreateHorizontalBox( 'BOX_OBJECT7', 100, 'BOX4')
385 oView:SetOwnerView('OTHEROBJECT7','BOX_OBJECT7')
386
387 //-----
388 // Área do grid
389 //-----
390 oView:CreateVerticalBox( 'BOX5', 50, 'BOX3')
391
392 oView:AddOtherObject('OTHEROBJECT9',{|o| SDTIDetalh(o) },,)
393 oView:SetOwnerView('OTHEROBJECT9','BOX5')
394
395 Return oView
396
397 //-----
398 /*{Protheus.doc} StructCupomVirtual
399 Estrutura ModelGrid do Cupom Virtual
400
401 @param
402 @return
403 @obs
404 @sample
405 */
406 //-----
407 Static Function StructCupomVirtual()
408
409 Local oStruct := FWFormModelStruct():New() //Estrutura
410
411 oStruct:AddTable( "SL2" , { "L2_FILIAL" } , "Itens" )
412

```

```

413 oStruct:AddField(           ;
414     "Item"                   , ; // [01] Titulo do campo
415     "Item"                   , ; // [02] Desc do campo
416     "L2_ITEM"                , ; // [03] Id do Field
417     "C"                      , ; // [04] Tipo do campo
418     TamSX3("L2_ITEM")[1]     , ; // [05] Tamanho do campo
419     0                        , ; // [06] Decimal do campo
420     Nil                      , ; // [07] Code-block de validacao do campo
421     Nil                      , ; // [08] Code-block de validacao When do campo
422     Nil                      , ; // [09] Lista de valores permitido do campo
423     Nil                      , ; // [10] Indica se o campo tem preenchimento obrigatorio
424     Nil                      , ; // [11] Code-block de inicializacao do campo
425     NIL                      , ; // [12] Indica se trata-se de um campo chave
426     NIL                      , ; // [13] Indica se o campo pode receber valor em uma operacao de update.
427     )                        , ; // [14] Indica se o campo e virtual
428
429
430 oStruct:AddField(           ;
431     "Cod."                   , ; // [01] Titulo do campo
432     "Cod."                   , ; // [02] Desc do campo
433     "L2_PRODUTO"             , ; // [03] Id do Field
434     "C"                      , ; // [04] Tipo do campo
435     TamSX3("L2_PRODUTO")[1] , ; // [05] Tamanho do campo
436     0                        , ; // [06] Decimal do campo
437     Nil                      , ; // [07] Code-block de validacao do campo
438     Nil                      , ; // [08] Code-block de validacao When do campo
439     Nil                      , ; // [09] Lista de valores permitido do campo
440     Nil                      , ; // [10] Indica se o campo tem preenchimento obrigatorio
441     Nil                      , ; // [11] Code-block de inicializacao do campo
442     NIL                      , ; // [12] Indica se trata-se de um campo chave
443     NIL                      , ; // [13] Indica se o campo pode receber valor em uma operacao de update.
444     )                        , ; // [14] Indica se o campo e virtual
445
446
447 oStruct:AddField(           ;
448     "Descrição"              , ; // [01] Titulo do campo
449     "Descrição"              , ; // [02] Desc do campo
450     "L2_DESCRI"              , ; // [03] Id do Field
451     "C"                      , ; // [04] Tipo do campo
452     TamSX3("L2_DESCRI")[1] + 10, ; // [05] Tamanho do campo
453     0                        , ; // [06] Decimal do? campo
454     Nil                      , ; // [07] Code-block de validacao do campo
455     Nil                      , ; // [08] Code-block de validacao When do campo
456     Nil                      , ; // [09] Lista de valores permitido do campo
457     Nil                      , ; // [10] Indica se o campo tem preenchimento obrigatorio
458     Nil                      , ; // [11] Code-block de inicializacao do campo
459     NIL                      , ; // [12] Indica se trata-se de um campo chave
460     NIL                      , ; // [13] Indica se o campo pode receber valor em uma operacao de update.
461     )                        , ; // [14] Indica se o campo e virtual
462
463 oStruct:AddField(           ;
464     "Preço Un."              , ; // [01] Titulo do campo

```

```

465         "Preço Un."          , ; // [02] Desc do campo
466         "L2_VRUNIT"         , ; // [03] Id do Field
467         "N"                  , ; // [04] Tipo do campo
468         TamSX3("L2_VRUNIT")[1], ; // [05] Tamanho do campo
469         2                    , ; // [06] Decimal do campo
470         Nil                   , ; // [07] Code-block de validacao do campo
471         Nil                   , ; // [08] Code-block de validacao When do campo
472         Nil                   , ; // [09] Lista de valores permitido do campo
473         Nil                   , ; // [10] Indica se o campo tem preenchimento obrigatorio
474         Nil                   , ; // [11] Code-block de inicializacao do campo
475         NIL                   , ; // [12] Indica se trata-se de um campo chave
476         NIL                   , ; // [13] Indica se o campo pode receber valor em uma operacao de update.
477         ) // [14] Indica se o campo e virtual
478
479 oStruct:AddField(           , ;
480         "Qtd"                , ; // [01] Titulo do campo
481         "Qtd"                , ; // [02] Desc do campo
482         "L2_QUANT"           , ; // [03] Id do Field
483         "N"                  , ; // [04] Tipo do campo
484         TamSX3("L2_QUANT")[1], ; // [05] Tamanho do campo
485         2                    , ; // [06] Decimal do campo
486         Nil                   , ; // [07] Code-block de validacao do campo
487         Nil                   , ; // [08] Code-block de validacao When do campo
488         Nil                   , ; // [09] Lista de valores permitido do campo
489         Nil                   , ; // [10] Indica se o campo tem preenchimento obrigatorio
490         Nil                   , ; // [11] Code-block de inicializacao do campo
491         NIL                   , ; // [12] Indica se trata-se de um campo chave
492         NIL                   , ; // [13] Indica se o campo pode receber valor em uma operacao de update.
493         ) // [14] Indica se o campo e virtual
494 oStruct:AddField(           , ;
495         "Un. Medida"         , ; // [01] Titulo do campo
496         "Un. Medida"         , ; // [02] Desc do campo
497         "L2_UM"              , ; // [03] Id do Field
498         "C"                  , ; // [04] Tipo do campo
499         TamSX3("L2_UM")[1] , ; // [05] Tamanho do campo
500         0                    , ; // [06] Decimal do campo
501         Nil                   , ; // [07] Code-block de validacao do campo
502         Nil                   , ; // [08] Code-block de validacao When do campo
503         Nil                   , ; // [09] Lista de valores permitido do campo
504         Nil                   , ; // [10] Indica se o campo tem preenchimento obrigatorio
505         Nil                   , ; // [11] Code-block de inicializacao do campo
506         NIL                   , ; // [12] Indica se trata-se de um campo chave
507         NIL                   , ; // [13] Indica se o campo pode receber valor em uma operacao de update.
508         ) // [14] Indica se o campo e virtual
509
510
511 oStruct:AddField(           , ;
512         "Desconto"           , ; // [01] Titulo do campo
513         "Desconto"           , ; // [02] Desc do campo
514         "L2_VALDESC"         , ; // [03] Id do Field
515         "N"                  , ; // [04] Tipo do campo
516         TamSX3("L2_VALDESC")[1], ; // [05] Tamanho do campo

```

```

517         2          , ; // [06] Decimal do campo
518         Nil        , ; // [07] Code-block de validacao do campo
519         Nil        , ; // [08] Code-block de validacao When do campo
520         Nil        , ; // [09] Lista de valores permitido do campo
521         Nil        , ; // [10] Indica se o campo tem preenchimento obrigatorio
522         Nil        , ; // [11] Code-block de inicializacao do campo
523         NIL        , ; // [12] Indica se trata-se de um campo chave
524         NIL        , ; // [13] Indica se o campo pode receber valor em uma operacao de update.
525                 , ; // [14] Indica se o campo e virtual
526
527 oStruct:AddField(      ;
528     "Valor"            , ; // [01] Titulo do campo
529     "Valor"            , ; // [02] Desc do campo
530     "L2_VLRITEM"       , ; // [03] Id do Field
531     "N"                , ; // [04] Tipo d?o campo
532     TamSX3("L2_VLRITEM")[1], ; // [05] Tamanho do campo
533     2                  , ; // [06] Decimal do campo
534     Nil                , ; // [07] Code-block de validacao do campo
535     Nil                , ; // [08] Code-block de validacao When do campo
536     Nil                , ; // [09] Lista de valores permitido do campo
537     Nil                , ; // [10] Indica se o campo tem preenchimento obrigatorio
538     Nil                , ; // [11] Code-block de inicializacao do campo
539     NIL               , ; // [12] Indica se trata-se de um campo chave
540     NIL               , ; // [13] Indica se o campo pode receber valor em uma operacao de update.
541                 , ; // [14] Indica se o campo e virtual
542
543
544 Return(oStruct)
545
546
547 //-----
548 /*{Protheus.doc} StructGridView
549 Estrutura da view
550
551 @param
552 @return
553 @obs
554 @sample
555 */
556 //-----
557 Static Function StructGridView()
558
559 Local oStruct      := FWFormViewStruct():New() // Estrutura
560 Local oPanelMVC    := STIGetPanel()           // Objeto MVC
561 Local aPosicao := {}
562 Local nPos
563 Local nI
564 Local aBackup := {}
565
566 oStruct:AddField(      ;
567     "L2_ITEM"          , ; // [01] cIdField      ID do Field
568     "0"                , ; // [02] cOrdem      Ordem do campo

```

```

569      "Item"      , ; // [03] cTitulo      Título do campo
570      "Item"      , ; // [04] cDescric    Descrição completa do campo
571      NIL          , ; // [05] aHelp      Array com o help dos campos
572      "C"          , ; // [06] cType      Tipo
573      Nil          , ; // [07] cPicture   Picture do Campo
574      Nil          , ; // [08] bPictVar    Bloco de Picture var
575      Nil          , ; // [09] cLookup    Chave para ser usado no Loopup
576      .F.          , ; // [10] lCanChange  Lógico dizendo se o campo pode ser alterado
577      Nil          , ; // [11] cFolder    Id da folder onde o Field está
578      NIL          , ; // [12] cGroup      Id do Group onde o field está
579      NIL          , ; // [13] aCaomboValues Array com os valores do combo
580      NIL          , ; // [14] nMaxLenCombo Tamanho máximo da maior opção do combo
581      NIL          , ; // [15] cIniBrow     Inicializador do Browse
582      NIL          , ; // [16] lVirtual    Indica se o campo é Virtual
583      NIL          , ; // [17] cPictVar    Picture Variável
584      NIL          , ; // [18] lInsertLine Indica pulo de linha após o campo
585      (oPanelMVC:nWidth / 2) * 0.12) // [19] nWidth Indica a largura fixa da coluna do grid
586
587 If STGetCfg("lPafEcf") .AND. lHomolPaf
588   oStruct:AddField(
589     "L2_PRODUTO" , ; // [01] cIdField    ID do Field
590     "1"           , ; // [02] cOrdem      Ordem do campo
591     "Cod."        , ; // [03] cTitulo      Título do campo
592     "Cod."        , ; // [04] cDescric    Descrição completa do campo
593     NIL           , ; // [05] aHelp      Array com o help dos campos
594     "C"           , ; // [06] cType      Tipo
595     Nil           , ; // [07] cPicture   Picture do Campo
596     Nil           , ; // [08] bPictVar    Bloco de Picture var
597     Nil           , ; // [09] cLookup    Chave para ser usado no Loopup
598     .F.           , ; // [10] lCanChange  Lógico dizendo se o campo pode ser alterado
599     Nil           , ; // [11] cFolder    Id da folder onde o Field está
600     NIL           , ; // [12] cGroup      Id do Group onde o field está
601     NIL           , ; // [13] aCaomboValues Array com os valores do combo
602     NIL           , ; // [14] nMaxLenCombo Tamanho máximo da maior opção do combo
603     NIL           , ; // [15] cIniBrow     Inicializador do Browse
604     NIL           , ; // [16] lVirtual    Indica se o campo é Virtual
605     NIL           , ; // [17] cPictVar    Picture Variável
606     NIL           , ; // [18] lInsertLine Indica pulo de linha após o campo
607     (oPanelMVC:nWidth / 2) * 0.15) // [19] nWidth Indica a largura fixa da coluna do grid
608 EndIf
609
610 oStruct:AddField(
611   "L2_DESCRI"    , ; // [01] cIdField    ID do Field
612   "2"            , ; // [02] cOrdem      Ordem do campo
613   "Descrição"    , ; // [03] cTitulo      Título do campo
614   "Descrição"    , ; // [04] cDescric    Descrição completa do campo
615   NIL            , ; // [05] aHelp      Array com o help dos campos
616   "C"            , ; // [06] cType      Tipo
617   Nil            , ; // [07] cPicture   Picture do Campo
618   Nil            , ; // [08] bPictVar    Bloco de Picture var
619   Nil            , ; // [09] cLookup    Chave para ser usado no Loopup
620   .F.            , ; // [10] lCanChange  Lógico dizendo se o campo pode ser alterado

```

```

621 Nil , ; // [11] cFolder Id da folder onde o Field está
622 Nil , ; // [12] cGroup Id do Group onde o field está
623 Nil , ; // [13] aCaomboValues Array com os valores do combo
624 Nil , ; // [14] nMaxLenCombo Tamanho máximo da maior opção do combo
625 Nil , ; // [15] cIniBrow Inicializador do Browse
626 Nil , ; // [16] lVirtual Indica se o campo é Virtual
627 Nil , ; // [17] cPictVar Picture Variável
628 Nil , ; // [18] lInsertLine Indica pulo de linha após o campo
629 (oPanelMVC:nWidth / 2) * 0.48) // [19] nWidth Indica a largura fixa da coluna do grid
630
631 oStruct:AddField( ;
632 "L2_VRUNIT" , ; // [01] cIdField ID do Field
633 "3" , ; // [02] cOrdem Ordem do campo
634 "Preço Un." , ; // [03] cTitulo Título do campo
635 "Preço" , ; // [04] cDescric Descrição completa do campo
636 Nil , ; // [05] aHelp Array com o help dos campos
637 "N" , ; // [06] cType Tipo
638 PesqPict("SL2","L2_VRUNIT"), ; // [07] cPicture Picture do Campo
639 Nil , ; // [08] bPictVar Bloco de Picture var
640 Nil , ; // [09] cLookup Chave para ser usado no Loop
641 .F. , ; // [10] lCanChange Lógico dizendo se o campo pode ser alterado
642 Nil , ; // [11] cFolder Id da folder onde o Field está
643 Nil , ; // [12] cGroup Id do Group onde o field está
644 Nil , ; // [13] aCaomboValues Array com os valores do combo
645 Nil , ; // [14] nMaxLenCombo Tamanho máximo da maior opção do combo
646 Nil , ; // [15] cIniBrow Inicializador do Browse
647 Nil , ; // [16] lVirtual Indica se o campo é Virtual
648 Nil , ; // [17] cPictVar Picture Variável
649 Nil , ; // [18] lInsertLine Indica pulo de linha após o campo
650 (oPanelMVC:nWidth / 2) * 0.34) // [19] nWidth Indica a largura fixa da coluna do grid
651
652 oStruct:AddField( ;
653 "L2_QUANT" , ; // [01] cIdField ID do Field
654 "4" , ; // [02] cOrdem Ordem do campo
655 "Qtd" , ; // [03] cTitulo Título do campo
656 "Qtd" , ; // [04] cDescric Descrição completa do campo
657 Nil , ; // [05] aHelp Array com o help dos campos
658 "N" , ; // [06] cType Tipo
659 PesqPictQt("L2_QUANT") , ; // [07] cPicture Picture do Campo
660 Nil , ; // [08] bPictVar Bloco de Picture var
661 Nil , ; // [09] cLookup Chave para ser usado no Loop
662 .F. , ; // [10] lCanChange Lógico dizendo se o campo pode ser alterado
663 Nil , ; // [11] cFolder Id da folder onde o Field está
664 Nil , ; // [12] cGroup Id do Group onde o field está
665 Nil , ; // [13] aCaomboValues Array com os valores do combo
666 Nil , ; // [14] nMaxLenCombo Tamanho máximo da maior opção do combo
667 Nil , ; // [15] cIniBrow Inicializador do Browse
668 Nil , ; // [16] lVirtual Indica se o campo é Virtual
669 Nil , ; // [17] cPictVar Picture Variável
670 Nil , ; // [18] lInsertLine Indica pulo de linha após o campo
671 (oPanelMVC:nWidth / 2) * 0.20) // [19] nWidth Indica a largura fixa da coluna do grid
672

```

```

673 If STFGetCfg("lPafEcf") .AND. lHomolPaf
674   oStruct:AddField(
675     "L2_UM"           , ; // [01] cIdField      ID do Field
676     "5"               , ; // [02] cOrdem       Ordem do campo
677     "Unid."           , ; // [03] cTitulo      Título do campo
678     "Unid."           , ; // [04] cDescric     Descrição completa do campo
679     NIL               , ; // [05] aHelp       Array com o help dos campos
680     "C"               , ; // [06] cType       Tipo
681     PesqPict("SL2","L2_UM"), ; // [07] cPicture  Picture do Campo
682     Nil               , ; // [08] bPictVar     Bloco de Picture var
683     Nil               , ; // [09] cLookup     Chave para ser usado no Loopup
684     .F.               , ; // [10] lCanChange  Lógico dizendo se o campo pode ser alterado
685     Nil               , ; // [11] cFolder     Id da folder onde o Field está
686     NIL               , ; // [12] cGroup      Id do Group onde o field está
687     NIL               , ; // [13] aComboValues Array com os valores do combo
688     NIL               , ; // [14] nMaxLenCombo Tamanho máximo da maior opção do combo
689     NIL               , ; // [15] cIniBrow     Inicializador do Browse
690     NIL               , ; // [16] lVirtual     Indica se o campo é Virtual
691     NIL               , ; // [17] cPictVar     Picture Variável
692     NIL               , ; // [18] lInsertLine  Indica pulo de linha após o campo
693     (oPanelMVC:nWidth / 2) * 0.13) // [19] nWidth Indica a largura fixa da coluna do grid
694 EndIf
695
696 oStruct:AddField(
697   "L2_VALDESC"       , ; // [01] cIdField      ID do Field
698   "6"                 , ; // [02] cOrdem       Ordem do campo
699   "Desc."             , ; // [03] cTitulo      Título do campo
700   "Desc."             , ; // [04] cDescric     Descrição completa do campo
701   NIL                 , ; // [05] aHelp       Array com o help dos campos
702   "N"                 , ; // [06] cType       Tipo
703   PesqPict("SL2","L2_VALDESC"), ; // [07] cPicture  Picture do Campo
704   Nil                 , ; // [08] bPictVar     Bloco de Picture var
705   Nil                 , ; // [09] cLookup     Chave para ser usado no Loopup
706   .F.                 , ; // [10] lCanChange  Lógico dizendo se o campo pode ser alterado
707   Nil                 , ; // [11] cFolder     Id da folder onde o Field está
708   NIL                 , ; // [12] cGroup      Id do Group onde o field está
709   NIL                 , ; // [13] aComboValues Array com os valores do combo
710   NIL                 , ; // [14] nMaxLenCombo Tamanho máximo da maior opção do combo
711   NIL                 , ; // [15] cIniBrow     Inicializador do Browse
712   NIL                 , ; // [16] lVirtual     Indica se o campo é Virtual
713   NIL                 , ; // [17] cPictVar     Picture Variável
714   NIL                 , ; // [18] lInsertLine  Indica pulo de linha após o campo
715   (oPanelMVC:nWidth / 2) * 0.33) // [19] nWidth Indica a largura fixa da coluna do grid
716
717 oStruct:AddField(
718   "L2_VLRITEM"       , ; // [01] cIdField      ID do Field
719   "7"                 , ; // [02] cOrdem       Ordem do campo
720   "Valor"             , ; // [03] cTitulo      Título do campo
721   "Valor"             , ; // [04] cDescric     Descrição completa do campo
722   NIL                 , ; // [05] aHelp       Array com o help dos campos
723   "N"                 , ; // [06] cType       Tipo
724   PesqPict("SL2","L2_VLRITEM"), ; // [07] cPicture  Picture do Campo

```

```

725 Nil , ; // [08] bPictVar Bloco de Picture var
726 Nil , ; // [09] cLookup Chave para ser usado no Loop
727 .F. , ; // [10] lCanChange Lógico dizendo se o campo pode ser alterado
728 Nil , ; // [11] cFolder Id da folder onde o Field está
729 NIL , ; // [12] cGroup Id do Group onde o field está
730 NIL , ; // [13] aCaomboValues Array com os valores do combo
731 NIL , ; // [14] nMaxLenCombo Tamanho máximo da maior opção do combo
732 NIL , ; // [15] cIniBrow Inicializador do Browse
733 NIL , ; // [16] lVirtual Indica se o campo é Virtual
734 NIL , ; // [17] cPictVar Picture Variável
735 NIL , ; // [18] lInsertLine Indica pulo de linha após o campo
736 (oPanelMVC:nWidth / 2) * 0.32) // [19] nWidth Indica a largura fixa da coluna do grid
737
738 If ExistBlock("STC0DB2")
739 aBackup := oStruct:aFields
740 for nI := 1 to len(oStruct:aFields)
741 Add (aPosicao, {oStruct:aFields[nI][1], oStruct:aFields[nI][2]} )
742 Next nI
743 aPosicao := ExecBlock("STC0DB2",.F.,.F.,{aPosicao})
744 If ValType(aPosicao) == "A" .AND. Len(aPosicao) >= 1
745 for nI := 1 to len(aPosicao)
746 nPos := aScan(oStruct:aFields,{|x| AllTrim(x[1]) == AllTrim(aPosicao[nI][1])})
747 If nPos > 0
748 oStruct:aFields[nPos][2] := aPosicao[nI][2]
749 else
750 oStruct:aFields := aBackup
751 MsgInfo("Erro no retorno do ponto de entrada STC0DB2, posições incorretas" ,"Atenção")
752 Exit
753 EndIf
754 Next nI
755 EndIf
756
757 EndIf
758
759 Return(oStruct)
760
761
762 //-----
763 /*/{Protheus.doc} STILogo
764 Exibe o logo da loja no topo da tela
765
766 @param
767 @return
768 @obs
769 @sample
770 */
771 //-----
772 Static Function STILogo( o )
773
774 Local cCSS := "" //CSS Do logo
775 Local oLogo := Nil //Objeto de bitmap do logo principal
776 Local lMobile := STGetCfg("lMobile", .F.) //Smart Client Mobile

```

```

777 Local oSayVDem      := Nil                                //Label Versao demonstrativa
778 Local cClientDir    := GetClientDir()                     //Caminho do diretorio do smartclient exe: "C:\totvs\bin\SmartClient\smartclient.ini"
779 Local lUnix         := IsSrvUnix()                         //Verifica se eh server linux
780 Local nPos          := Rat( IIf( lUnix, "/", "\", ) , cClientDir ) //Posicao da ultima barra
781
782 Default o := Nil
783
784 If !( nPos == 0 )
785     cClientDir := SubStr( cClientDir, 1, nPos )
786 EndIf
787
788 cCSS := POSCSS( GetClassName(o), CSS_PANEL_LOGO, "FFFFFF" )
789
790 //Busca o Logo no Diretório smartclient
791 If File( cClientDir + "logopos.jpg" )
792
793     o:SetCSS( cCSS )
794
795     oLogo := TBitmap():New( 15,10,(o:nWidth/2)-(15),(o:nHeight/2)-(20),,,.T.,0,;
796     {|| FWAlertInfo( STR0053 + CRLF + ; //"Melhor Resolução para o seu Logo."
797     STR0054 + AllTrim(STR((o:nWidth)-(15*2))) + " x " + STR0055 + ;//"Largura: " + "Altura: "
798     AllTrim(STR((o:nHeight)-(20*2))) + STR0056 ) },{||},.F.,.F.,,.F.,.T.)//" em Pixel."
799
800     oLogo:Load( Nil , cClientDir + "logopos.jpg" )
801
802
803 //Em versoes Mobile o logo eh alterado
804 ElseIf ValType(lMobile) == "L" .AND. lMobile
805
806     @ 011,005 REPOSITORY oLogo OF 0 NOBORDER SIZE o:nWidth/2-7 , o:nHeight/2-15 PIXEL
807
808     //Versao demonstrativa exhibe label especifico
809     If STFTYPEOperation() == "DEMONSTRACAO"
810
811         @ 000,000 SAY oSayVDem PROMPT STR0047 SIZE o:nClientHeight/2,15 CENTER OF oLogo PIXEL //"VERSÃO EXPERIMENTAÇÃO"
812         oSayVDem:SetCSS( POSCSS (GetClassName(oSayVDem), CSS_LABEL_TRIAL))
813         oSayVDem:Align := CONTROL_ALIGN_BOTTOM
814
815     EndIf
816
817 //Valida nome com espacos pois em algumas versoes o Wizard add imagem com espacos
818 If oLogo:ExistBmp(PADR("LOGOP05",20))
819     ShowBitmap(oLogo,PADR("LOGOP05",20),"")
820 Else
821     ShowBitmap(oLogo,"LOGOP05","")
822 EndIf
823
824 oLogo:lStretch := .T.
825
826 o:SetCSS( cCSS )
827
828 Else

```

```

829
830     o:SetCSS( cCSS )
831
832     @ 001,010 REPOSITORY oLogo OF 0 NOBORDER SIZE o:nWidth/2-20,o:nHeight/2-1 PIXEL
833     //-----
834     // Alteração feita para reaplicar o estilo pois a partir do binário
835     // 131227A o CSS é aplicado no objeto ADVPL e não mais no objeto Qt.
836     // Com isso o oLogo não consegue mais herdar o CSS do container.
837     //-----
838     oLogo:SetCSS( StrTran( cCSS, "TPanelCss", "TBitmap" ) )
839
840     ShowBitmap(oLogo,"LOGOPOS","")
841
842 EndIf
843
844 Return .T.
845
846 //-----
847 /*/{Protheus.doc} STIPnlAcoes
848 Cria os box's das acoes com usuario
849
850 @param
851 @return
852 @obs
853 @sample
854 */
855 //-----
856 Static Function STIPnlAcoes( o )
857
858 Local oTop           //Objeto de cabecalho
859 Local oContent       //Objeto de interacao com usuario
860
861 o:SetCSS( POSCSS (GetClassName(o), CSS_PANEL_CONTEXT ))
862
863 oArea := o
864
865 @ 000,000 BITMAP oTop RESOURCE "x.png" NOBORDER SIZE o:nWidth/2,070 OF o ADJUST PIXEL
866 oTop:SetCSS( POSCSS (GetClassName(oTop), CSS_PANEL_OPTION_TOP ))
867 oTop:ReadClientCoors(.T.,.T.)
868
869 STIMenuBtn(oTop)
870
871 oContent := tPanel():New(oTop:nHeight/2, 000, "", o,,,,, o:nWidth/2, (o:nHeight-oTop:nHeight)/2)
872 oContent:SetCSS( POSCSS (GetClassName(oContent), CSS_PANEL_CONTENT ))
873 oContent:ReadClientCoors(.T.,.T.)
874 oContent:LostFocus := {} oBtnConfirm:SetFocus() } // Tratamento para não perder o foco do Cancelar VENDA
875 STIPnlCont(oContent)
876
877 Return
878
879
880 //-----

```

```

881 /*{Protheus.doc} STIMenuBtn
882 PaineL dos botoes
883
884 @param
885 @return
886 @obs
887 @sample
888 */
889 //-----
890 Static Function STIMenuBtn( o )
891
892 Local nTop      := 10 //Altura dos botoes
893 Local nLeft     := 10 //Horizontal dos botoes
894 Local nMargin  := 05 //Tamanho horizontal
895 Local nWidth    //Lagura dos botoes
896 Local nHeight   //Altura dos botoes
897 Local nX       := 1 //Posicao horizontal dos botoes
898 Local nY       := 1 //Posicao vertical dos botoes
899 Local lMobile   := STFCfg( "lMobile" , .F. )
900 Local lExibAtalho := .T. //Valida se Exibe atalhos ex: F2, F5 etc em versoes mobile nao exibe
901 Local lAtvCnsPro := SuperGetMV( "MV_LJTPSCH" , .F. , 0 ) <> 2 //0= legado; 1= codigo de barras; 2= bloqueia
902
903
904 lMobile := ValType(lMobile) == "L" .AND. lMobile
905 lExibAtalho := !lMobile
906
907 nWidth := o:nWidth-35
908 nWidth := (nWidth/2-nMargin*2)/3
909 nHeight := o:nHeight-30
910 nHeight := (nHeight/2-nMargin)/2
911
912 /* Registro de item */
913 @ nTop+(nX-1)*(nHeight+nMargin), nLeft+(nY-1)*(nWidth+nMargin) BUTTON oBtnRegItem PROMPT IIf(lExibAtalho,STR0015,STR0037) SIZE nWidth,nHeight ACTION {||STIRegItemInterface()
    } WHEN {|| lBtnActive } OF o PIXEL // "(F3) Reg. de item" ### "Registro de Item"
914 oBtnRegItem:SetCSS( POSCSS (GetClassName(oBtnRegItem), CSS_BTN_NORMAL ))
915 oBtnRegItem:LCANGOTFOCUS := .F.
916 nY := 2
917
918 /* Cliente */
919 @ nTop+(nX-1)*(nHeight+nMargin), nLeft+(nY-1)*(nWidth+nMargin) BUTTON oBtnClient PROMPT IIf(lExibAtalho,STR0016,STR0038) SIZE nWidth,nHeight ACTION {||
    STICustomerSelection()} WHEN {|| IIF(STDPBLength("SL2") == 0, lBtnActive, .F. ) } OF o PIXEL
920 oBtnClient:SetCSS( POSCSS (GetClassName(oBtnClient), CSS_BTN_NORMAL ))
921 oBtnClient:LCANGOTFOCUS := .F.
922 nY := 3
923
924 /* Cancelamento de item */
925 @ nTop+(nX-1)*(nHeight+nMargin), nLeft+(nY-1)*(nWidth+nMargin) BUTTON oBtnCancItem PROMPT IIf(lExibAtalho,STR0017,STR0039) SIZE nWidth,nHeight ACTION {|| STICancelItem() }
    WHEN {|| IIF(STDPBLength("SL2") == 0, .F., lBtnActive) } OF o PIXEL
926 oBtnCancItem:SetCSS( POSCSS (GetClassName(oBtnCancItem), CSS_BTN_NORMAL ))
927 oBtnCancItem:LCANGOTFOCUS := .F.
928 nX := 2
929 nY := 1

```

```

930
931 /* Orçamento Ou Vendedor em versao mobile*/
932 @ nTop+(nX-1)*(nHeight+nMargin), nLeft+(nY-1)*(nWidth+nMargin) BUTTON oBtnOrc PROMPT IIf(lExibAtalho,STR0018,STR0040) SIZE nWidth,nHeight ACTION {||
    IIf(lMobile,STISalesmanSelection(),STIImportSale()) } WHEN {|| lBtnActive } OF o PIXEL
933 oBtnOrc:SetCSS( POSCSS (GetClassName(oBtnOrc), CSS_BTN_NORMAL ))
934 oBtnOrc:LCANGOTFOCUS := .F.
935 nY := 2
936
937 /* Desconto na venda */
938 @ nTop+(nX-1)*(nHeight+nMargin), nLeft+(nY-1)*(nWidth+nMargin) BUTTON oBtnDesc PROMPT IIf(lExibAtalho,STR0019,STR0041) SIZE nWidth,nHeight ACTION { || STIDiscountTotal() }
    WHEN {|| IIF(STDPBLength("SL2") == 0, .F., lBtnActive) } OF o PIXEL
939 oBtnDesc:SetCSS( POSCSS (GetClassName(oBtnDesc), CSS_BTN_NORMAL ))
940 oBtnDesc:LCANGOTFOCUS := .F.
941 nY := 3
942
943 /* Consulta de Produtos */
944 @ nTop+(nX-1)*(nHeight+nMargin), nLeft+(nY-1)*(nWidth+nMargin) BUTTON oBtnConProd PROMPT IIf(lExibAtalho,STR0020,STR0042) SIZE nWidth,nHeight ACTION { || STIProdSearch() }
    WHEN {|| (lBtnActive .AND. lAtvCnsPro) } OF o PIXEL
945 oBtnConProd:SetCSS( POSCSS (GetClassName(oBtnConProd), CSS_BTN_NORMAL ))
946 oBtnConProd:LCANGOTFOCUS := .F.
947
948 Return
949
950 //-----
951 /*{Protheus.doc} STIPnlCont
952 Abre a primeira tela de abertura de caixa
953
954 @param
955 @return
956 @obs
957 @sample
958 */
959 //-----
960 Static Function STIPnlCont( oContent )
961
962 oContent:SetCSS( POSCSS (GetClassName(oContent), CSS_PANEL_CONTEXT ))
963 STICreateObjects(oContent)
964
965 /* Abre a View de abertura de caixa */
966
967 //Se for somente menu fiscal nao abre a rotina inicial
968 If !STFGetCfg("lOnlyMenuFiscal")
969     STIOpenCash()
970 EndIf
971
972
973 Return .T.
974
975 //-----
976 /*{Protheus.doc} SDTIDetalh
977 Criacao dos box's de detalhes, cabecalho, itens e rodape
978

```

```

979 @param
980 @return
981 @obs
982 @sample
983 /*/
984 //-----
985 Static Function SDTIDetalh( o )
986
987 Local oTop          := Nil //Objeto para cabecalho
988 Local oContent      := Nil //Objeto para itens
989 Local oBottom       := Nil //Objeto para rodape
990
991 oViewRight := o
992
993 @ 000,000 BITMAP oTop RESOURCE "x.png" NOBORDER SIZE 000,055 OF o ADJUST PIXEL
994 oTop:Align := CONTROL_ALIGN_TOP
995 oTop:SetCSS( POSCSS (GetClassName(oTop), CSS_PANEL_HEADER ))
996 oTop:ReadClientCoors(.T.,.T.)
997
998 STICab( oTop )
999
1000 @ 000,000 BITMAP oBottom RESOURCE "x.png" NOBORDER SIZE 000,055 OF o ADJUST PIXEL
1001 oBottom:Align := CONTROL_ALIGN_BOTTOM
1002 oBottom:SetCSS( POSCSS (GetClassName(oBottom), CSS_PANEL_FOOTER ))
1003 oBottom:ReadClientCoors(.T.,.T.)
1004
1005 STIRodape( oBottom )
1006
1007 @ 000,000 BITMAP oContent RESOURCE "x.png" NOBORDER SIZE 000,000 OF o ADJUST PIXEL
1008 oContent:Align := CONTROL_ALIGN_ALLCLIENT
1009 oContent:ReadClientCoors(.T.,.T.)
1010
1011 STIItems( oContent )
1012
1013 @ (o:nClientHeight/2)-50,4 BITMAP oSemaforo RESOURCE "FRT0FFLINE" NOBORDER SIZE 16,16 OF o ADJUST PIXEL
1014 oSemaforo:ReadClientCoors(.T.,.T.)
1015
1016 STILoadSem()
1017 Return
1018
1019 //-----
1020 /*/{Protheus.doc} STICab
1021 Cabeçalho da tela
1022
1023 @param
1024 @return
1025 @obs
1026 @sample
1027 /*/
1028 //-----
1029 Static Function STICab( o )
1030

```

```

1031 Local nTop      := 10 //Altura do cabeçalho
1032 Local nLeft := 10 //Horizontal do cabeçalho
1033 Local oSay      //Objeto label
1034 Local cNomeEmp  := SM0->M0_NOMECON // Nome da Empresa
1035 Local cEndEmp   := SM0->M0_ENDENT  // Endereço Empresa
1036
1037
1038 @ nTop,nLeft SAY oSay PROMPT "<b>" + cNomeEmp + "</b><br>" + cEndEmp + "<br>" SIZE o:nWidth/2-35,40 CENTER OF o PIXEL HTML
1039 oSay:SetCSS( POSCSS (GetClassName(oSay), CSS_LABEL_HEADER ))
1040 Return .T.
1041
1042 //-----
1043 /*{Protheus.doc} STIIItems
1044 Criação do grid dos itens
1045
1046 @param
1047 @return
1048 @obs
1049 @sample
1050 */
1051 //-----
1052 Static Function STIIItems( o )
1053
1054 Local oContent //Conteudo do css
1055 Local oStruGrd //Estrutura da grid
1056 Local lMobile := STFGetCfg("lMobile", .F.) //Smart Client Mobile
1057 Local lExibScroll := .T. //Exibe scroll na Grid
1058 Local oCodProd := Iif(FindFunction("STIGetCodP"),STIGetCodP(),NIL) //Retorna o GET de digitacao do produto para poder dar foco no objeto na recuperacao da venda
1059
1060 lMobile := ValType(lMobile) == "L" .AND. lMobile
1061
1062 lExibScroll := !lMobile
1063
1064 oContent := POSBrwContainer(o)
1065
1066 oViewDetail := FWFormView():New(oView)
1067 oViewDetail:SetOwner(oContent)
1068 oViewDetail:SetModel(oView:oModel)
1069 oViewDetail:setUseCursor(.F.)
1070 oViewDetail:SetOperation(oView:GetOperation())
1071
1072 // Grid com Paginação em Botoes
1073 If STIShowPagGrid()
1074     oViewDetail:CreateHorizontalBox( 'BOX1', 90)
1075     oViewDetail:CreateHorizontalBox( 'BOX2', 10)
1076 Else
1077     //Grid Padrao
1078     oViewDetail:CreateHorizontalBox( 'BOX1', 100)
1079 EndIf
1080
1081 oStruGrd := StructGridView()
1082 oViewDetail:AddGrid( 'VIEW_CUPOM' , oStruGrd, 'CUP_GRID' )

```

```

1083 oViewDetail:SetOwnerView("VIEW_CUPOM","BOX1")
1084 oViewDetail:SetViewProperty("VIEW_CUPOM","SETCSS",{POSCSS("TGRID", CSS_BROWSE)})
1085 oViewDetail:SetViewProperty("VIEW_CUPOM","ENABLENEWGRID")
1086 oViewDetail:SetViewProperty("VIEW_CUPOM","GRIDVSCROLL",{lExibScroll})
1087 oViewDetail:SetViewProperty("VIEW_CUPOM","GRIDROWHEIGHT",{40})
1088 oViewDetail:SetViewProperty("VIEW_CUPOM","GRIDCANGOTFOCUS",{.F.})
1089 oViewDetail:SetViewProperty("VIEW_CUPOM","GRIDDOUBLECLICK",{||.F.})
1090 oViewDetail:SetViewProperty(" *", "GRIDN0ORDER")
1091
1092 // Grid com Paginação em Botoes
1093 If STIShowPagGrid()
1094     oViewDetail:AddOtherObject('PAGPDV', {oPanel| STIObjPag(oPanel)} )
1095     oViewDetail:SetOwnerView("PAGPDV","BOX2")
1096
1097     //Nao Exibe linhas na Grid
1098     If GetBuild() >= "7.00.131227A-20140123"
1099         oViewDetail:GetViewObj("VIEW_CUPOM")[3]:OBrowse:Browse():lShowGrid := .F.
1100     EndIf
1101 EndIf
1102
1103 oViewDetail:SetAfterViewActivate( {oViewDetail| DesalbitaColChange(oViewDetail)})
1104
1105 oViewDetail:Activate()
1106
1107 If STBIsRecovered()
1108     STIGridCupRefresh()
1109     STIBtnActivate()
1110     STFMessage(ProcName(),"STOP",STR0021) //"Venda recuperada com sucesso!"
1111     STFShowMessage(ProcName())
1112
1113     //Apos a recuperacao da venda, o foco da get de produto era perdida, entao eh forçado o foco abaixo
1114     If oCodProd <> NIL
1115         oCodProd:SetFocus()
1116     EndIf
1117 EndIf
1118
1119 Return .T.
1120
1121 //-----
1122 /*{Protheus.doc} STIRodape
1123 Criação das barra dos totalizadores
1124
1125 @param
1126 @return
1127 @obs
1128 @sample
1129 */
1130 //-----
1131 Static Function STIRodape( o )
1132
1133 Local nLeft := 020 // Posição esquerda
1134 Local nTop := 001 // Posição superior

```

```

1135 Local oGet                      // Objeto Get
1136 Local oSay                      // Objeto Say
1137 Local oTotal    := STFGetTot()    //Recebe o Objeto totalizador
1138 Local lMobile   := STFGetCfg("lMobile" , .F. ) //PDV em Versao Mobile
1139
1140 lMobile := ValType(lMobile) == "L" .AND. lMobile
1141
1142 @ nTop+2,nLeft SAY oSay PROMPT STR0022 SIZE 038,010 RIGHT OF o PIXEL HTML    //"Volumes:"
1143 oSay:SetCSS( POSCSS (GetClassName(oSay), IIf(lMobile,CSS_LABEL_SUBTITLE,CSS_LABEL_NORMAL)))
1144
1145 @ nTop,nLeft+oSay:nWidth/1.8 GET oGetQtde VAR STIQtdItens() SIZE 100,010 OF o PIXEL
1146 oGetQtde:SetCSS( POSCSS (GetClassName(oGetQtde), IIf(lMobile,CSS_GET_TOTAL,CSS_GET_NORMAL)))
1147 oGetQtde:lActive := .F.
1148
1149 nTop += 11
1150
1151 @ nTop+2,nLeft SAY oSay PROMPT STR0023 SIZE 037,010 RIGHT OF o PIXEL HTML    //"Subtotal:"
1152 oSay:SetCSS( POSCSS (GetClassName(oSay), IIf(lMobile,CSS_LABEL_SUBTITLE,CSS_LABEL_NORMAL)))
1153
1154 @ nTop,nLeft+oSay:nWidth/1.73 GET oGetSubTot VAR AllTrim(Str(oTotal:GetValue("L1_VALMERC"),12,2)) SIZE 100,010 OF o PIXEL
1155 oGetSubTot:SetCSS( POSCSS (GetClassName(oGetSubTot), IIf(lMobile,CSS_GET_TOTAL,CSS_GET_NORMAL)))
1156 oGetSubTot:lActive := .F.
1157
1158 nTop += 11
1159
1160 @ nTop+2,nLeft SAY oSay PROMPT STR0024 SIZE 042,010 RIGHT OF o PIXEL HTML    //"Desconto:"
1161 oSay:SetCSS( POSCSS (GetClassName(oSay), IIf(lMobile,CSS_LABEL_SUBTITLE,CSS_LABEL_NORMAL)))
1162
1163 @ nTop,nLeft+oSay:nWidth/2 GET oGetDiscount VAR AllTrim(Str(oTotal:GetValue("L1_DESCONT"),12,2)) SIZE 100,010 OF o PIXEL
1164 oGetDiscount:SetCSS( POSCSS (GetClassName(oGetDiscount), IIf(lMobile,CSS_GET_TOTAL,CSS_GET_NORMAL)))
1165 oGetDiscount:lActive := .F.
1166
1167 If FindFunction("STIQtdBrindes")
1168     nTop += 11
1169     @ nTop+2,nLeft SAY oSay PROMPT "Brinde(s):" SIZE 040,010 RIGHT OF o PIXEL HTML    //"Brinde(s):" Leandro
1170     oSay:SetCSS( POSCSS (GetClassName(oSay), IIf(lMobile,CSS_LABEL_SUBTITLE,CSS_LABEL_NORMAL)))
1171
1172     @ nTop,nLeft+oSay:nWidth/1.9 GET oGetQtdBrinde VAR STIQtdBrindes() SIZE 100,010 OF o PIXEL
1173     oGetQtdBrinde:SetCSS( POSCSS (GetClassName(oGetQtdBrinde), IIf(lMobile,CSS_GET_TOTAL,CSS_GET_NORMAL)))
1174     oGetQtdBrinde:lActive := .F.
1175 EndIf
1176
1177 nTop := 006
1178
1179 @ nTop,o:nWidth/2-40 SAY oSay PROMPT STR0025 SIZE o:nWidth/2-35,40 OF o PIXEL HTML    //"Total"
1180 oSay:SetCSS( POSCSS (GetClassName(oSay), IIf(lMobile,CSS_LABEL_SUBTITLE,CSS_LABEL_NORMAL))) //rafael
1181
1182 nTop += 10
1183
1184 @ nTop,o:nWidth/2-120 SAY oSayTotal PROMPT AllTrim(Str(oTotal:GetValue("L1_VLRTOT"),12,2)) SIZE 100,40 RIGHT OF o PIXEL HTML
1185 oSayTotal:SetCSS( POSCSS (GetClassName(oSayTotal), CSS_LABEL_TOTAL))
1186

```

```

1187 Return .T.
1188
1189 //-----
1190 /*{Protheus.doc} STIGetDlg
1191 Retorno do objeto oDlg
1192
1193 @param
1194 @return oDlg - Objeto principal da tela
1195 @obs
1196 @sample
1197 */
1198 //-----
1199 Function STIGetDlg()
1200 Return oArea
1201
1202 //-----
1203 /*{Protheus.doc} STIGetRDlg
1204 Retorno do objeto oViewRight
1205
1206 @param
1207 @return oDlg - Objeto principal da tela
1208 @obs
1209 @sample
1210 */
1211 //-----
1212 Function STIGetRDlg()
1213 Return oViewRight
1214
1215 //-----
1216 /*{Protheus.doc} STIGetRDlg
1217 Retorno do objeto oViewRight
1218
1219 @param
1220 @return oDlg - Objeto principal da tela
1221 @obs
1222 @sample
1223 */
1224 //-----
1225 Function STISetRDlg()
1226
1227 oViewRight:FreeChildren()
1228 SDTIDetalh(oViewRight)
1229
1230 Return
1231
1232 //-----
1233 /*{Protheus.doc} STIGetObjTela
1234 Retorna o Objeto de Tela
1235
1236 @param
1237 @return oPOSDesk
1238 @obs

```

```

1239 @sample
1240 /*/
1241 //-----
1242 Function STIGetObjTela()
1243 Return oPOSDesk
1244
1245 //-----
1246 /*/{Protheus.doc} STISetMsg
1247 Seta mensagem de tela
1248
1249 @param   cMsg - Mensagem para exibicao
1250 @return
1251 @obs
1252 @sample
1253 /*/
1254 //-----
1255 Function STISetMsg(cMsg)
1256
1257 Default cMsg := ""
1258
1259 oPOSDesk:SetBAAlert( cMsg )
1260
1261 Return Nil
1262
1263
1264 //-----
1265 /*/{Protheus.doc} STIGridCupRefresh
1266 Sincroniza a cesta com o grid do cupom virtual da interface
1267
1268 @param
1269 @return
1270 @obs
1271 @sample
1272 /*/
1273 //-----
1274 Function STIGridCupRefresh(nItemDe,nItemAte)
1275
1276 Local oModelCesta      := STDGPBModel()           // Model de venda
1277 Local oModelSL2        := oModelCesta:GetModel("SL2DETAIL") // Model Itens Orc.
1278 Local oModel           := oMdlPrin                // Model principal
1279 Local oModelCup         := oModel:GetModel("CUP_GRID")   // Model Cup.Grid
1280 Local oStruct           := oModelCup:GetStruct()        // Model Estrutura
1281 Local aFieldsCup        := oStruct:aFields            // Objeto Estrutura Campos
1282 Local nI                := 0                        // Contador I
1283 Local nJ                := 0                        // Contador J
1284 Local oGridView         := oViewDetail:GetViewObj("VIEW_CUPOM")[3] // Objeto GridView
1285 Local xValue            :=                          // Variável auxiliar
1286
1287 Default nItemDe         := 1                        // Item Inicial que sera atualizado o cupom virtual da interface
1288 Default nItemAte        := oModelSL2:Length()       // Item Final que sera atualizado o cupom virtual da interface
1289
1290 If nItemAte <= oModelSL2:Length()

```

```

1291
1292  /* Sincroniza o Grid */
1293  For nI := nItemDe To nItemAte
1294
1295      /* Posiciona */
1296      oModelSL2:GoLine(nI)
1297
1298      If oModelCup:Length() < nI
1299          /*Adiciona uma nova linha no modelo e no Grid*/
1300          oGridView:OBrowse:nAt := oGridView:OBrowse:nLen
1301          oModelCup:AddLine()
1302      Else
1303          oViewDetail:GoLine("CUP_GRID",nI)
1304          oGridView:OBrowse:nAt := nI
1305      EndIf
1306
1307      /* Sincroniza valores */
1308      For nJ := 1 To Len(aFieldsCup)
1309
1310          If AllTrim(aFieldsCup[nJ][3]) == 'L2_DESCRI' .AND. oModelSL2:IsDeleted(nI)
1311              xValue := SubStr(xValue + ' - CANCELADO',1,TamSX3("L2_DESCRI")[1] + 10)
1312          ElseIf AllTrim(aFieldsCup[nJ][3]) == 'L2_VRUNIT'
1313              xValue := oModelSL2:GetValue('L2_PRC TAB')
1314          Else
1315              xValue := oModelSL2:GetValue(aFieldsCup[nJ][3])
1316          EndIf
1317
1318          oModelCup:LoadValue( aFieldsCup[nJ][3] , xValue )
1319          oGridView:setValue(aFieldsCup[nJ][3],xValue ,nI)
1320
1321      Next nJ
1322
1323      /* Sincroniza deleção */
1324      If oModelSL2:IsDeleted(nI)
1325
1326          oModelCup:GoLine(nI)
1327          oModelCup>DeleteLine( .F. , .T. )
1328
1329      Else
1330
1331          oModelCup:GoLine(nI)
1332          If oModelCup:IsDeleted(nI)
1333
1334              oModelCup:UnDeleteLine()
1335
1336          EndIf
1337
1338      EndIf
1339
1340  Next nI
1341
1342

```

```

1343 EndIf
1344
1345 oGridView:Refresh(, .T.)
1346
1347 If oModelSL2:Length() > oGridView:obrowse:obrowse:GetVisibleRows()
1348     oGridView:obrowse:gobottom()
1349 EndIf
1350 //Paginacao do Grid por botoes
1351 If STIShowPagGrid()
1352     STIMsgPag()
1353 EndIf
1354 Return Nil
1355
1356 //-----
1357 /*{Protheus.doc} STIGetMdlPri
1358 Retorna o model da cesta
1359
1360 @param
1361 @return
1362 @obs
1363 @sample
1364 */
1365 //-----
1366 Function STIGetMdlPri()
1367 Return oMdlPrin
1368
1369 //-----
1370 /*{Protheus.doc} STICreateObjects
1371 Cria os objetos principais da tela
1372
1373 @param
1374 @return
1375 @obs
1376 @sample
1377 */
1378 //-----
1379 Function STICreateObjects( oPanel )
1380
1381 Default oPanel      := Nil
1382
1383 If oPanel <> Nil
1384
1385     oPanelMVC := oPanel
1386
1387     oPanel1 := TPanel():New(00,00,"",oPanelMVC,,,,,oPanelMVC:nWidth/2,oPanelMVC:nHeight/2)
1388     oPanel2 := TPanel():New(00,00,"",oPanelMVC,,,,,oPanelMVC:nWidth/2,oPanelMVC:nHeight/2)
1389     oPanel1:SetCSS( POSCSS (GetClassName(oPanel1), CSS_PANEL_CONTEXT))
1390     oPanel2:SetCSS( POSCSS (GetClassName(oPanel2), CSS_PANEL_CONTEXT))
1391 EndIf
1392
1393 Return Nil
1394

```

```

1395
1396 //-----
1397 /*{Protheus.doc} STIGetPanel
1398 Retorna o objeto do painel principal
1399
1400 @param
1401 @return
1402 @obs
1403 @sample
1404 */
1405 //-----
1406 Function STIGetPanel()
1407 Return(oPanelMVC)
1408
1409 //-----
1410 /*{Protheus.doc} STIExchangePanel
1411 Alterna os painéis do Contexto
1412
1413 @param   bPanel Bloco de construção do próximo panel
1414 @return
1415 @obs
1416 @sample
1417 */
1418 //-----
1419 Function STIExchangePanel( bPanel, lTtotal )
1420 Local lItCkAct := ExistFunc("STIItCkAct") .And. STIItCkAct()
1421
1422 Default bPanel := {} }
1423 Default lTtotal := .F.
1424
1425 If SuperGetMv( "MV_LJCFSPG",,.F.) .AND. STDGPBasket("SL1","L1_VLRTOT") > 0 .AND. "STIPAYMENT" $ cLastPnlOpen
1426     STFMessage(ProcName(),"NOYES",STR0057) //"Tem certeza que deseja sair e abandonar os pagamentos informados?"
1427     If !STFShowMessage(ProcName())
1428         Return Nil
1429     EndIf
1430 EndIf
1431
1432 // -- Caso saia da tela de finalização de venda recupera os pagamentos do orçamento.
1433 If "STIPAYMENT()" $ cLastPnlOpen .AND. STBIsImpOrc() .AND. !"STIPANDISCTOTAL()" $ Upper(GetCBSource(bPanel))
1434     STISetPayImp(.F.)
1435 EndIf
1436
1437 If "STIPANCONFCASH()" $ cLastPnlOpen
1438     STISetRDlg()
1439 Endif
1440
1441 cLastPnlOpen := Upper(GetCBSource(bPanel)) //Guarda ultimo panel chamado na troca de panels
1442
1443 Do Case
1444     Case lTtotal
1445         //Nao é possível utilizar a logica do nPanelActive = 0 ou igual 1 ou igual a 2 porque no momento de finalizar a Venda o nPanelActive é igual a 2
1446         //porém quando o nPanelActive é igual 1 o sistema exibe a tela de brinde mas ocorre erro log no oPanel1 que não existe.

```

```

1447 // e quando o o nPanelActive é igual a 2 o sistema não exibe a tela de brinde e ocorre erro log no oPanel2 que não existe
1448 // por esse motivo foi criado essa consistencia:
1449 If ValType(oPanel1) == "0" .And. !lItCkAct
1450     oPanel1:FreeChildren()
1451     oPanel1 := Nil
1452 EndIf
1453 STIFocoReg()
1454 oPanel1 := EVal(bPanel)
1455
1456 oPanel1:Show()
1457
1458 oPanel1:Refresh()
1459
1460 nPanelActive := 2
1461
1462 Case nPanelActive == 0
1463
1464     If ValType(oPanel1) == "0" .And. !lItCkAct
1465         oPanel1:FreeChildren()
1466         oPanel1 := Nil
1467     EndIf
1468
1469     oPanel1 := EVal(bPanel)
1470     nPanelActive := 1
1471
1472 Case nPanelActive == 1
1473
1474     If ValType(oPanel2) == "0" .And. !lItCkAct
1475         LjGrvLog("STIExchangePanel" , "Destroi oPanel2 Hparent: "+ Str(oPanel2:Hparent) + " oPanel2 Hwnd : " + Str(oPanel2:Hwnd) )
1476         oPanel2:FreeChildren()
1477         FreeObj(oPanel2)
1478         LjGrvLog("STIExchangePanel" , "oPanel2 - "+ Iif (ValType(oPanel2) == "0", "Não foi possivel Destruir", "Destruiu"))
1479     EndIf
1480
1481     oPanel2 := EVal(bPanel) //0 Panel2 é que exibe a tela de brinde e qdo a variavel nPanelActive é igual a 1 ele exibe porem ocorre erro log no oPanel1
1482     LjGrvLog("STIExchangePanel" , "Constroi oPanel2 Hparent: "+ Str(oPanel2:Hparent) + " oPanel2 Hwnd : " + Str(oPanel2:Hwnd) )
1483     oPanel2:Show()
1484
1485     oPanel2:Refresh()
1486
1487     If ValType(oPanel1) == "0" .And. !lItCkAct
1488         iF GetCliHTML() .AND. (oPanel2:Hparent < oPanel1:Hparent)
1489             LjGrvLog("STIExchangePanel" , "Destroi oPanel1 Hparent: "+ Str(oPanel1:Hparent) + " oPanel2 Hwnd : " + Str(oPanel1:Hwnd) )
1490             oPanel1:FreeChildren()
1491             FreeObj(oPanel1)
1492             LjGrvLog("STIExchangePanel" , "oPanel1 - "+ Iif (ValType(oPanel1) == "0", "Não foi possivel Destruir", "Destruiu"))
1493         Else
1494             oPanel1:Hide()
1495         Endif
1496     EndIf
1497
1498     nPanelActive := 2

```

```

1499
1500     Case nPanelActive == 2
1501
1502         If ValType(oPanel1) == "0" .And. !lItCkAct
1503             LjGrvLog("STIExchangePanel" , "Destroi oPanel1 Hparent: "+ Str(oPanel1:Hparent) + " oPanel2 Hwnd : " + Str(oPanel1:Hwnd) )
1504             oPanel1:FreeChildren()
1505             FreeObj(oPanel1)
1506             LjGrvLog("STIExchangePanel" , "oPanel1 - "+ Iif (ValType(oPanel1) == "0", "Não foi possível Destruir", "Destruiu"))
1507         EndIf
1508
1509         oPanel1 := Eval(bPanel)
1510         LjGrvLog("STIExchangePanel" , "Constroi oPanel1 Hparent: "+ Str(oPanel1:Hparent) + " oPanel1 Hwnd : " + Str(oPanel1:Hwnd) )
1511         oPanel1:Show()
1512
1513         oPanel1:Refresh()
1514
1515         If ValType(oPanel2) == "0" .And. !lItCkAct
1516             If GetCliHTML() .AND. (oPanel2:Hparent > oPanel1:Hparent)
1517                 LjGrvLog("STIExchangePanel" , "Destroi oPanel2 Hparent: "+ Str(oPanel2:Hparent) + " oPanel2 Hwnd : " + Str(oPanel2:Hwnd) )
1518                 oPanel2:FreeChildren()
1519                 FreeObj(oPanel2)
1520                 LjGrvLog("STIExchangePanel" , "oPanel2 - "+ Iif (ValType(oPanel2) == "0", "Não foi possível Destruir", "Destruiu"))
1521             Else
1522                 oPanel2:Hide()
1523             Endif
1524         EndIf
1525
1526         nPanelActive := 1
1527
1528     EndCase
1529
1530     oPOSDesk:SetBAAlert( "" )
1531
1532     Return
1533
1534
1535     //-----
1536     /*{Protheus.doc} STIQtdItens
1537     Atualiza o contador de itens
1538
1539     @param
1540     @return
1541     @obs
1542     @sample
1543     */
1544     //-----
1545     Function STIQtdItens()
1546
1547     Local cRet      := ""                //Variavel de retorno
1548     Local oModel    := oMdlPrin          //Model Principal
1549     Local oModelCup := oModel:GetModel("CUP_GRID") //Model do grid
1550     Local nI        := 0                //Variavel de loop

```

```

1551 Local nQtdItens := 0 //Acumulador - Qtde.Itens
1552
1553 For nI := 1 To oModelCup:Length()
1554     oModelCup:GoLine(nI)
1555     If !oModelCup:IsDeleted()
1556         nQtdItens += oModelCup:GetValue('L2_QUANT')
1557     EndIf
1558 Next nI
1559
1560 cRet := AllTrim(Str(nQtdItens)) + ' ' + STR0026 //Iten(s)
1561
1562 Return cRet
1563
1564 //-----
1565 /*{Protheus.doc} STIQtdBrindes
1566 Atualiza o contador de brindes
1567
1568 @param
1569 @return
1570 @obs
1571 @sample
1572 */
1573 //-----
1574 Function STIQtdBrindes()
1575 Local cRet := "" //Variavel de retorno
1576 Local oModel := STDGPBModel()
1577 Local nQtdItens := If( AliasInDic("MGC"),STDPBLength("MGC"),0) // quantidade total de registros na cesta MGC
1578 Local nCont := 0 // contador de brindes ok
1579 Local nX := 0 // variavel de controle
1580
1581 For nX := 1 to nQtdItens
1582     If oModel:GetModel("MGCBRINDE"):acols[nX][13] == .F. // não esta deletado
1583         nCont:= nCont + 1
1584     Endif
1585 Next nX
1586
1587 cRet := alltrim(str(nCont)+ ' ' + "Brindes")
1588
1589 Return cRet
1590 //-----
1591 /*{Protheus.doc} STIFocuFirst
1592 Na abertura da tela da um setfocus no primeiro get
1593
1594 @param
1595 @return
1596 @obs
1597 @sample
1598 */
1599 //-----
1600 Function STIFocusFirst()
1601
1602 Local oGetCash := Nil //Get da abertura de caixa

```

```

1603
1604 oGetCash := STIGetCash()
1605 oGetCash:SetFocus()
1606
1607 Return .T.
1608
1609 //-----
1610 /*{Protheus.doc} STIFocuFirst
1611 Executa a atualização do rodape da mensagem
1612 @param
1613 @return
1614 @obs
1615 @sample
1616 */
1617 //-----
1618 Function STITimeMsg(oObj, cSemana, lMobile)
1619     Default oObj := oPOSDesk
1620     Default cSemana := ""
1621
1622     Default lMobile := .F.
1623     //Em versoes Mobile nao exibe essa opcao. Somente no Menu superior
1624     If FindFunction("STFLatestMsg") .AND. !lMobile
1625         oObj:SetBInfo( STR0043 + Space(10) + AllTrim(cSemana) + " " + AllTrim(DToC(Date())) + " | " + AllTrim(Left(Time(), 5)) ) //"F10 - Log de Mensagens |"
1626     Else
1627         oObj:SetBInfo(Space(10) + AllTrim(cSemana) + " " + AllTrim(DToC(Date())) + " | " + AllTrim(Left(Time(), 5)) )
1628     EndIf
1629 STILoadSem()
1630 Return
1631
1632 //-----
1633 /*{Protheus.doc} STIChangeCssBtn
1634 Faz a troca do CSS qdo clica em alguns dos botoes
1635 @param
1636 @return
1637 @obs
1638 @sample
1639 */
1640 //-----
1641 Function STIChangeCssBtn(cButton)
1642
1643 Local aButtons := {'oBtnRegItem','oBtnClient','oBtnCancItem','oBtnOrc','oBtnDesc','oBtnConProd'} //Array com nome dos botes
1644 Local nI := 0 //Variavel de loop
1645
1646 Default cButton := ''
1647
1648
1649 For nI := 1 To Len(aButtons)
1650     If Empty(cButton)
1651         IF !lClientHtml
1652             &(aButtons[nI] + ':SetCss(POSCSS(GetClassName('+ aButtons[nI] + '),nNormal))')
1653         Endif
1654     Else

```

```

1655      If aButtons[nI] == cButton
1656          &(aButtons[nI] + ':SetCss(POSCSS(GetClassName('+ aButtons[nI] + '),nAtivo))')
1657      ElseIf !Empty(cButton)
1658          &(aButtons[nI] + ':SetCss(POSCSS(GetClassName('+ aButtons[nI] + '),nNormal))')
1659      EndIf
1660  EndIf
1661
1662 Next nI
1663
1664
1665 Return .T.
1666
1667 //-----
1668 /*{Protheus.doc} STIPosClDsk
1669 Libera o objeto oPOSDesk que não está sendo liberado e está ocorrendo erro na função STIMessage
1670 @param
1671 @return .t.
1672 @obs
1673 @sample
1674 */
1675 //-----
1676 Static Function STIPosClDsk()
1677
1678 FreeObj(oPOSDesk)
1679 oPOSDesk := NIL
1680
1681 Return .T.
1682
1683 //-----
1684 /*{Protheus.doc} STIPosG0pc
1685 Retorna a Operação em execução atual.
1686
1687 @param
1688 @return  Operação em execução atual
1689 @obs
1690 @sample
1691 */
1692 //-----
1693 Function STIPosG0pc()
1694 Return cOperation
1695
1696 //-----
1697 /*{Protheus.doc} STIPosS0pc
1698 Seta a Operação em execução atual.
1699
1700 @param
1701 @return Nil
1702 @obs
1703 @sample
1704 */
1705 //-----
1706 Function STIPosS0pc(cOper)

```

```

1707 cOperation := cOper
1708 Return Nil
1709
1710 //-----
1711 /*{Protheus.doc} DesalbitaColChange
1712 Altera propriedade da view pois e' feito setvalue no view e o remote segura e dispara muitas
1713 alteracoes e pode dar stackoverflow
1714 @param
1715 @return Nil
1716 @obs
1717 @sample
1718 */
1719 //-----
1720 Static function DesalbitaColChange(oViewDetail)
1721 Local aViewInfo := aClone(oViewDetail:GetViewObj("VIEW_CUPOM"))
1722 Local oGridView := aViewInfo[3] // Objeto GridView
1723
1724 //Removido o controle de Coluna alterada, pois e' feito setvalue no view e o remote segura e dispara muitas
1725 //Alteracoes e pode dar stackoverflow
1726 oGridView:oBrowse:oBrowse:bColumnChanged := {||.T.}
1727
1728 aViewInfo := aSize(aViewInfo,0)
1729 aViewInfo := Nil
1730
1731 Return Nil
1732
1733
1734 //-----
1735 /*{Protheus.doc} STIShowPagGrid
1736 Funcao que habilita paginacao por botoes na GRID
1737 @param
1738 @return .T.
1739 @obs
1740 @sample
1741 */
1742 //-----
1743 Function STIShowPagGrid()
1744
1745 Local lMobile := STFGetCfg("lMobile", .F.) //Smart Client Mobile
1746 Local lRet := .F. //Retorno
1747
1748 //Versao Mobile exibe Grid com paginacao por botao
1749 lRet := ValType(lMobile) == "L" .AND. lMobile
1750
1751 Return lRet
1752
1753
1754 //-----
1755 /*{Protheus.doc} STINext
1756 Funcao de proximo da paginacao do grid
1757 @param
1758 @return .T.

```

```

1759 @obs
1760 @sample
1761 /*/
1762 //-----
1763 Function STINext()
1764
1765 Local oGridView      := oViewDetail:GetViewObj("VIEW_CUPOM")[3] //Objeto GridView
1766 Local nItensGrid     := SuperGetMv('MV_LJQTITE',,0)           //Qtde de itens que cabe na grid
1767 Local nPos           := oGridView:OBrowse:nAt                //Posicao atual do ponteiro na view
1768 Local lMobile       := STFGetCfg("lMobile" , .F. )           //E mobile?
1769
1770 If nItensGrid == 0
1771     If lMobile
1772         nItensGrid := Val(Str(oGridView:OBrowse:OBrowse:nClientHeight / oGridView:OBrowse:nRowHeight - 2,2))
1773     Else
1774         nItensGrid := Val(Str(oGridView:OBrowse:OBrowse:nClientHeight / oGridView:OBrowse:nRowHeight - 1,2))
1775     EndIf
1776 EndIf
1777
1778 If nPos != STDPBLength("SL2") .AND. STDPBLength("SL2") > nItensGrid
1779     If nPos == 1
1780         oGridView:OBrowse:nAt := nItensGrid + 1
1781     Else
1782         If (nPos + nItensGrid) < STDPBLength("SL2")
1783             oGridView:OBrowse:nAt := nPos + nItensGrid
1784         Else
1785             oGridView:OBrowse:nAt := STDPBLength("SL2")
1786         EndIf
1787     EndIf
1788     oGridView:OBrowse:Refresh(,.T.)
1789     STIMsgPag(oGridView)
1790 EndIf
1791
1792 Return .T.
1793
1794
1795 //-----
1796 /*/{Protheus.doc} STIReturn
1797 Funcao de retorno da paginacao do grid
1798 @param
1799 @return .T.
1800 @obs
1801 @sample
1802 /*/
1803 //-----
1804 Function STIReturn()
1805
1806 Local oGridView := oViewDetail:GetViewObj("VIEW_CUPOM")[3] // Objeto GridView
1807 Local nPosItem := 0 //Posicao que ira mostrar no grid
1808 Local nItensGrid := SuperGetMv('MV_LJQTITE',,0) //Qtde de itens que cabe na grid
1809 Local nQtdeItens := 0 //Quantidade de itens
1810 Local lMobile := STFGetCfg("lMobile" , .F. ) //E mobile?

```

```

1811
1812 If nItensGrid == 0
1813     If lMobile
1814         nItensGrid := Val(Str(oGridView:OBrowse:OBrowse:nClientHeight / oGridView:OBrowse:nRowHeight - 2,2))
1815     Else
1816         nItensGrid := Val(Str(oGridView:OBrowse:OBrowse:nClientHeight / oGridView:OBrowse:nRowHeight - 1,2))
1817     EndIf
1818 EndIf
1819
1820 nQtdeItens := nItensGrid * 2
1821
1822 If !(oGridView:OBrowse:nAt <= nItensGrid) .AND. STDPBLength("SL2") > nItensGrid
1823     If STDPBLength("SL2") <= nQtdeItens
1824         oGridView:OBrowse:nAt := 1
1825     Else
1826         nPosItem := oGridView:OBrowse:nAt - nItensGrid
1827         If nPosItem < 0 .OR. nPosItem <= nItensGrid
1828             oGridView:OBrowse:nAt := 1
1829         Else
1830             oGridView:OBrowse:nAt := nPosItem
1831         EndIf
1832     EndIf
1833     oGridView:OBrowse:Refresh(, .T.)
1834     STIMsgPag(oGridView)
1835 EndIf
1836
1837 Return .T.
1838
1839 //-----
1840 /*{Protheus.doc} STIMsgPag
1841 Funcao que monta a mensagem da quantidade de itens (de/ate)
1842 @param
1843 @return .T.
1844 @obs
1845 @sample
1846 */
1847 //-----
1848 Static Function STIMsgPag(oGridView)
1849
1850 Local nNumIni := 0 //Numero inicial da paginacao
1851 Local nNumFin := 0 //Numero final da paginacao
1852 Local nItensGrid := SuperGetMv('MV_LJQTITE',,0) //Qtde de itens que cabem na grid
1853 Local lMobile := STFGetCfg("lMobile", .F. ) //E versao mobile?
1854
1855 Default oGridView := IIF(oGridView == Nil,oViewDetail:GetViewObj("VIEW_CUPOM")[3],oGridView)
1856
1857 If nItensGrid == 0
1858     If lMobile
1859         nItensGrid := Val(Str(oGridView:OBrowse:OBrowse:nClientHeight / oGridView:OBrowse:nRowHeight - 2,2))
1860     Else
1861         nItensGrid := Val(Str(oGridView:OBrowse:OBrowse:nClientHeight / oGridView:OBrowse:nRowHeight - 1,2))
1862     EndIf

```

```

1863 EndIf
1864
1865 If oGridView <> Nil
1866
1867     If ProcName(1) == "STIGRIDCUPREFRESH" .AND. STDPBLength("SL2") > nItensGrid
1868         nNumIni := (oGridView:OBrowse:nAt - nItensGrid) + 1
1869     Else
1870         nNumIni := oGridView:OBrowse:nAt
1871     EndIf
1872     nNumFin := oGridView:OBrowse:nAt + (nItensGrid - 1)
1873
1874     If nNumFin > STDPBLength("SL2") .AND. STDPBLength("SL2") > 0
1875         nNumFin := STDPBLength("SL2")
1876     ElseIf STDPBLength("SL2") == 0
1877         nNumIni := 0
1878         nNumFin := 0
1879     EndIf
1880
1881     cMsgPg := STR0048 + AllTrim(Str(nNumIni)) + "-" + AllTrim(Str(nNumFin)) + STR0049 + AllTrim(Str(STDPBLength("SL2"))) + STR0050 // "Mostrando " ### " de " ### " itens"
1882
1883     oSayPag:Refresh()
1884
1885 EndIf
1886
1887 Return .T.
1888
1889
1890 //-----
1891 /*{Protheus.doc} STI0bjPag
1892 Cria os objetos para paginacao da grid
1893 @param
1894 @return .T.
1895 @obs
1896 @sample
1897 */
1898 //-----
1899 Static Function STI0bjPag(oPanel)
1900
1901 oPnlPag := TPanel():New(00,00,"",oPanel,,,,,700,50)
1902 @ 001, 030  BUTTON oBtnAnt PROMPT "< " + STR0051  SIZE 50,15 ACTION {|| STIReturn()} OF oPnlPag PIXEL // "Anterior"
1903 @ 001, (oBtnAnt:nWidth/2) + 10  SAY oSayPag PROMPT cMsgPg SIZE 100,18 CENTER OF oPnlPag PIXEL HTML
1904 @ 001, (oBtnAnt:nWidth/2) + (oSayPag:nWidth/2) + 15  BUTTON oBtnPro PROMPT STR0052 + " >"  SIZE 50,15 ACTION {|| STINext()} OF oPnlPag PIXEL // "Próximo"
1905
1906
1907 oSayPag:SetCSS( POSCSS (GetClassName(oSayPag), CSS_LABEL_HEADER ))
1908 oBtnAnt:SetCSS( POSCSS (GetClassName(oBtnAnt), CSS_BTN_NORMAL ))
1909 oBtnPro:SetCSS( POSCSS (GetClassName(oBtnPro), CSS_BTN_NORMAL ))
1910
1911 Return Nil
1912 //-----
1913 /*{Protheus.doc} STIFocoReg
1914 Foaca no botão Reg.Item

```

```

1915 @param
1916 @return
1917 @obs
1918 @sample
1919 */
1920 //-----
1921 Function STIFocoReg()
1922     oBtnRegItem:SetFocus()
1923 Return
1924
1925 //-----
1926 /*{Protheus.doc} STIAtuRoda
1927 Força a atualização do Rodapé
1928 @param
1929 @return
1930 @obs
1931 @sample
1932 */
1933 //-----
1934 Function STIAtuRoda()
1935     oViewRight:Refresh()
1936 Return
1937 /*{Protheus.doc} STILoadSem
1938 Carrega imagem do semaforo de conexao com a Retaguarda
1939 @param lTestCon - Se testa conexao
1940 @param lOnline - Se conectado
1941 @return
1942 @obs
1943 @sample
1944 */
1945 //-----
1946
1947 Function STILoadSem(lTestCon, lOnline)
1948
1949 Default lTestCon := .T. //Verifica se faz teste de conexao
1950 Default lOnline := .F. //Status para alteracao imagem
1951
1952 If ValType(oSemaforo) == "0"
1953     //Testa conexao
1954     If lTestCon
1955         lOnline := IIF(GetPvProfString(CSECA0, CCHAVE, '0', GetAdv97()) == '0', .F., .T.)
1956     EndIf
1957
1958     //Carrega imagem
1959     If lOnline .OR. lPdvOn
1960         oSemaforo:SetBMP("FRTONLINE") //Conectado
1961     Else
1962         oSemaforo:SetBMP("FRTOFFLINE") //Sem conexao
1963     EndIf
1964
1965     oSemaforo:Refresh()
1966 EndIf

```

```

1967
1968 Return Nil
1969
1970 /*{Protheus.doc} GetTpClient
1971     Retorna se o Client WebAgent
1972     @return lCliHtml, lógico, se HTML = .T.
1973 */
1974 Function GetCliHTML()
1975
1976 Local cVerWebApp      := ""
1977 Local lCliHtml        := .F.
1978
1979 //Verifica se o SmartClient é via WebApp ou não.
1980 GetRemoteType(@cVerWebApp)
1981 lCliHtml:= 'HTML' $ UPPER(cVerWebApp)
1982
1983 LjGrvLog("GetCliHTML","Acesso realizado pelo WebAgent versão do WebApp",cVerWebApp)
1984
1985 Return lCliHtml
1986
1987
1988 //-----
1989 /*{Protheus.doc} StiMataObj
1990 Elimina a instância dos objetos da tela antes de trocar de painal
1991 @param oMainPanel, Object, Objeto da tela
1992 */
1993 //-----
1994 Function StiMataObj(oMainPanel, cPanelAnt)
1995
1996 Default oMainPanel := Nil // Objeto que contem a tela.
1997 Default cPanelAnt  := "" //Não pode pegar a variável static cLastPnlOpen, sempre receber via parametro.
1998
1999 If lClientHtml .And. oMainPanel <> Nil
2000     oMainPanel:FreeChildren()
2001     FreeObj(oMainPanel)
2002     oMainPanel:=NIL
2003 Endif
2004
2005 Return

```

Componentes/Interface/STIProdSearch.prw

556 linhas após remoção de metadados de autoria/data

```

1 #Include 'Protheus.ch'
2 #INCLUDE 'FWBROWSE.CH'
3 #INCLUDE "PARMTYPE.CH"
4 #INCLUDE "FWEVENTVIEWCONSTS.CH"
5 #INCLUDE "FWADAPTEREAI.CH"
6 #INCLUDE "FWMVCDEF.CH"
7 #INCLUDE "POSCSS.CH"
8 #INCLUDE "STPOS.CH"
9 #INCLUDE "STIPRODSEARCH.CH"
10
11 Static aRecno      := {}
12 Static nTpSrchrPro := SuperGetMV("MV_LJTPSCH", .F., 0)
13 Static lOnlyCodBar := (nTpSrchrPro==1)
14 Static lHomolPaf   := STBHomolPaf()
15 Static lDisableBtn := .F.
16 Static cGetProduct := Iif(lOnlyCodBar, Space(TamSx3("LK_CODBAR")[1]), Space(40)) // Variável do Get
17 Static cPictValUnit := PesqPict("SL2","L2_VRUNIT")
18 Static oLbLProdDados:= Nil
19 Static oLbLProdVal  := Nil // Variável para armazenar o valor do produto
20 Static oMainPanel   := Nil
21
22
23 //-----
24 /*{Protheus.doc} STIProdSearch
25 Funcao responsavel por chamar a troca do painel atual pelo painel de selecao de produtos.
26
27
28 */
29 //-----
30
31 Function STIProdSearch()
32
33 Local lMobile := STFGetCfg("lMobile", .F.) //Smart Client Mobile
34
35 STIChangeCssBtn("oBtnConProd")
36
37 If ValType(lMobile) == "L" .AND. lMobile
38     //Tela especifica versao mobile
39     STIExchangePanel( { || STIPanMobProdSearch() } )
40 Else
41     //Tela Padrao
42     STIExchangePanel( { || STIPanProdSearch() } )
43 EndIf
44
45 Return
46
47 //-----
48 /*{Protheus.doc} STIPanProdSearch
49 Funcao responsavel pela criacao do Painel de selecao de produtos.
50
51 @return oMainPanel - Objeto contendo o painel principal de selecao de clientes.
52 */

```

```

53 //-----
54 Static Function STIPanProdSearch()
55
56 Local oPanelMVC      := STIGetPanel()           // Objeto do Panel onde a interface de selecao de clientes sera criada
57
58 Local oLblCab        := Nil                    // Objeto do label "Seleção de Cliente"
59 Local oLblSearch     := Nil                    // Objeto do label "Digite aqui para filtrar"
60
61 /*/
62     Get Combo de Busca
63 /*/
64 Local oGetList        := Nil                    // TComboBox Escolha do campo a ser utilizado na busca
65 Local cGetList        := ""                    // Variável do Get
66
67
68 /*/
69     Get de Busca
70 /*/
71 Local oGetSearch      := Nil                    // TGet Cancelamento de Item
72 Local lPAFECEf        := STFGetCfg("lPafEcf")
73
74
75 /*/
76     Botão "Selecionar Cliente"
77 /*/
78 Local oButton         := Nil                    // Botao "Selecionar Cliente"
79
80 Local oListFont        := TFont():New("Courier New") // Fonte utilizada no listbox
81 Local oLbllist        := Nil
82 Local bConfirm         := { || STIConfirmProd(oGetList), SetKey(19, {|| .F.})}
83 Local cLabelCons       := STR0003 + IIF(!lPAFECEf, "", STR0012) /// UN / Sit Trib/ IAT / IPPT
84 Local oImg            := Nil
85 Local oLblInfProd      := Nil
86 Local cDados          := "Descrição do Produto" + CRLF + CRLF + CRLF + "Código"
87 Local lBusca          := .F. //Executou a busca do componente
88 Local CValor          := CRLF + CRLF + CRLF + "Valor"
89 Local oLblValProd      := Nil
90 Local cPathImg         := AllTrim(SuperGetMV("MV_LJIMAGE", .F., "")) // Caminho da imagem do produto Opcional
91 Local cColor           := Nil
92 Local cMascara         := "@" + Iif(lOnlyCodBar, Padr("R",TamSx3("LK_CODBAR")[1],"9"), "!")
93
94 cGetProduct := Iif(lOnlyCodBar, Space(TamSx3("LK_CODBAR")[1]), Space(40))
95
96 If oMainPanel <> Nil
97     LjGrvLog("STIProdSearch" , "Dextroi oMainPanel Hparent: "+ Str(oMainPanel:Hparent) + " oPanel1 Hwnd : " + Str(oMainPanel:Hwnd) )
98     oMainPanel:FreeChildren()
99     FreeObj(oMainPanel)
100 Endif
101
102 oMainPanel      := TPanel():New(00,00,"",oPanelMVC,,,,,oPanelMVC:nWidth/2,(oPanelMVC:nHeight)/2) // Painel de Get de Consulta
103
104 If ExistBlock("STClrBProd")

```

```

105     cColor := ExecBlock("STClrBProd",.F.,.F.)
106
107     If !Empty(cColor) .AND. ValType( cColor) == "C"
108         oMainPanel:SetCSS("QFrame{ background-color: " + cColor + "}")
109     EndIf
110
111 EndIf
112
113 /*/
114     Objetos
115 /*/
116
117 oLbLCab:= TSay():New(POSVERT_CAB,POSHOR_1,{||Iif(!onlyCodBar, STR0022, STR0001)},oMainPanel,,,,,.T.,,,) //"Consulta de Produtos"
118 oLbLCab:SetCSS( POSCSS (GetClassName(oLbLCab), CSS_BREADCUMB ))
119
120 oLbLSearch := TSay():New(oPanelMVC:nHeight/26.92,POSHOR_1, {||STR0002}, oMainPanel,,,,,.T.) //'Pesquisar Produto:'
121 oLbLSearch:SetCSS( POSCSS (GetClassName(oLbLSearch), CSS_LABEL_FOCAL ))
122
123 oGetSearch:= TGet():New(     oPanelMVC:nHeight/14.955,POSHOR_1,{|u| If(PCount()>0,(lBusca := .F.,cGetProduct:=u), cGetProduct)}, ;
124                             oMainPanel,120 ,ALTURAGET,cMascara,,,,,.T.,,,,,,"cGetProduct") //colocar o valid
125 oGetSearch:SetCSS( POSCSS (GetClassName(oGetSearch), CSS_GET_NORMAL ))
126 oGetSearch:bLostFocus := { || If(!lBusca, STLostFocus(oGetSearch,oGetList,@cGetProduct, @lBusca), IIf(oGetList:Len() > 1,(oGetList:SetFocus(),Nil),oButton:SetFocus()))}
127
128 oLbLList := TSay():New(oPanelMVC:nHeight/8.412,POSHOR_1, {|| cLabelCons }, oMainPanel,,,,,.T.) //'Descrição / Código / Valor'
129 oLbLList:SetCSS( POSCSS (GetClassName(oLbLList), CSS_LABEL_FOCAL ))
130
131 oGetList := TListBox():Create(oMainPanel, oPanelMVC:nHeight/6.73, POSHOR_1, {|u| If(PCount()>0,cGetList:=u,cGetList)}, , LARG_LIST_CONSULT , oPanelMVC:nHeight/9 ,{|o|
132     IF(lBusca, STIChangeInfo( o, oImg ), ) },,,,,.T.,,bConfirm,oListFont)
133 oGetList:SetCSS( POSCSS (GetClassName(oGetList), CSS_LISTBOX ))
134
135 oLbLInfProd := TSay():New(oPanelMVC:nHeight/3.6378,POSHOR_1,{||cDados},oMainPanel,,,,,.T.,,,100) // Descrição e Código do Produto
136 oLbLInfProd:SetCSS( POSCSS (GetClassName(oLbLInfProd), CSS_LABEL_NORMAL ))
137
138 oLbLValProd := TSay():New(oPanelMVC:nHeight/3.6378,POSHOR_1+130,{||cValor},oMainPanel,,,,,.T.,,,100) // Valor do Produto
139 oLbLValProd:SetCSS( POSCSS (GetClassName(oLbLValProd), CSS_LABEL_NORMAL ))
140
141 //Verifica se usa foto de arquivo
142 If Empty(cPathImg)
143     @ oPanelMVC:nHeight/3.6378,oPanelMVC:nWidth/3.01886 REPOSITORY oImg OF oMainPanel NOBORDER SIZE 100,100 PIXEL
144 Else
145     oImg := TBitmap():New( oPanelMVC:nHeight/3.6378,oPanelMVC:nWidth/3.01886,100,100,"",.T.,oMainPanel,{||},,.F.,.F.,,.F.,.T.,.F.)
146     oImg:lStretch:= .T.
147 EndIf
148 oImg:lVisibleControl := .F.
149
150 oButton := TButton():New( POSVERT_BTNFOCAL,POSHOR_BTNFOCAL,STR0004+CRLF+STR0014,oMainPanel,bConfirm, ; //"Selecionar Produto" + "CTRL+S"
151                             LARGBTN,ALTURABTN,,,,.T.,,,{||IIF(Len(aRecno) > 0, .T., .F.)})
152 oButton:SetCSS( POSCSS (GetClassName(oButton), CSS_BTN_FOCAL ))
153
154 oButCan := TButton():New( POSVERT_BTNFOCAL,POSHOR_1,STR0013+CRLF+STR0015,oMainPanel,{|| STIRetItem() }, ; //"Cancela Pesquisa" + "(CTRL+A)"
155                             LARGBTN,ALTURABTN,,,,.T.,,,{||.T.})
156 oButCan:SetCSS( POSCSS (GetClassName(oButCan), CSS_BTN_FOCAL ))

```

```

156
157 SetKey(19, bConfirm)
158 SetKey(1, {|| STIRetItem() })
159
160 oGetSearch:SetFocus()
161 If GetFocus() <> oGetSearch:HWND
162     oGetSearch:SetFocus()
163 Endif
164
165
166 Return oMainPanel
167
168
169 //-----
170 /*{Protheus.doc} STIPanMobProdSearch
171 Funcao responsavel pela criacao do Painel de selecao de produtos versao Mobile.
172
173 @return oMainPanel - Objeto contendo o painel principal de selecao
174 */
175 //-----
176 Static Function STIPanMobProdSearch()
177
178 Local oPanelMVC           := STIGetPanel()           // Objeto do Panel onde a interface de selecao de produtos sera criada
179 Local oLblCab             := Nil                     // Objeto do label "Seleção de produtos"
180 Local oLblSearch          := Nil                     // Objeto do label "Digite aqui para filtrar"
181 Local oGetList            := Nil                     // TComboBox Escolha do campo a ser utilizado na busca
182 Local cGetList            := ""                     // Variável do Get
183 Local oGetSearch          := Nil                     // TGet busca do item
184 Local oGetQtde            := Nil                     // TGet quantidade do item
185 Local oGetVlrDesc         := Nil                     // TGet Valor desconto
186 Local oGetPercDesc        := Nil                     // TGet percentual de desconto
187 Local nGetQtde            := 1                      // Quantidade
188 Local nGetVlrDesc         := 0                      // Valor de desconto
189 Local nGetPercDesc        := 0                      // Percentual de desconto
190 Local oButton             := Nil                     // Botao "Selecionar produto"
191 Local oListFont           := TFont():New("Courier New") // Fonte utilizada no listbox
192 Local oLblList            := Nil                     // Objeto list
193 Local bConfirm            := { || }                 //Bloco de confirmacao
194 Local cLabelCons          := STR0003                // UN / Sit Trib/ IAT / IPPT
195 Local lBusca              := .F.                   //Valida se Executou a busca do componente
196 Local oButPesq           := Nil                     //Botao pesquisar
197 Local oLblQuantidade      := Nil                     //Label quantidade
198 Local oLblVlrDesc         := Nil                     //Label desconto valor
199 Local oLblPercDesc        := Nil                     //Label desconto percentual
200 Local oLblSimbPerc        := Nil                     //Label simbolo desconto percentual
201 Local lFindAll            := .T.                     //Busca em todos os registros caso deixe o get em branco
202 Local oButCan             := Nil                     //Botao Cancelar
203
204 bConfirm                  := { || STIConfirmProd( oGetList ,;
205                                     IIF(nGetQtde > 0 .AND. nGetQtde <= 999,AllTrim(STR(nGetQtde))+"*,""") + ; // Quantidade do item
206                                     IIF(nGetVlrDesc > 0 , AllTrim(STR(nGetVlrDesc)) + "-","" , IIF(nGetPercDesc > 0 , AllTrim(STR(nGetPercDesc)) + "%" , "" ) ) ) ;
207
208                                     // Valor de desconto

```

```

207                                     , SetKey(19, {|| .F.}) }
208
209 oMainPanel      := TPanel():New(00,00,"",oPanelMVC,,,,,oPanelMVC:nWidth/2,(oPanelMVC:nHeight)/2) // Paine de Get de Consulta
210
211 oLblCab:= TSay():New(POSVERT_CAB,POSHOR_1,{||STR0001},oMainPanel,,,,,.T.,,,) //"Consulta de Produtos"
212 oLblCab:SetCSS( POSCSS (GetClassName(oLblCab), CSS_BREADCUMB ))
213
214 oLblSearch := TSay():New(oPanelMVC:nHeight/26.92,POSHOR_1, {||STR0002}, oMainPanel,,,,,.T.) //'Pesquisar Produto:'
215 oLblSearch:SetCSS( POSCSS (GetClassName(oLblSearch), CSS_LABEL_FOCAL ))
216
217 //Get de Produtos
218 oGetSearch:= TGet():New( oPanelMVC:nHeight/14.955,POSHOR_1,{||u| If(PCount())>0,(lBusca := .F.,cGetProduct:=u), cGetProduct}}, ;
219 oMainPanel,120 ,ALTURAGET,"@!",,,,,,.T.,,,,,,"cGetProduct")
220 oGetSearch:SetCSS( POSCSS (GetClassName(oGetSearch), CSS_GET_NORMAL ))
221
222 //"Botao pesquisar"
223 oButPesq      := TButton():New( oPanelMVC:nHeight/14.955,POSHOR_1+130,"",oMainPanel,{ || If(!lBusca, STLostFocus(oGetSearch,oGetList,@cGetProduct, @lBusca , lFindAll ), .T.
224                                     )},20,20,,,,.T.,,,)
225 oButPesq:SetCSS( POSCSS (GetClassName(oButPesq), CSS_BTN_LUPA ))
226
227 oLblList := TSay():New(oPanelMVC:nHeight/8.412,POSHOR_1, {|| cLabelCons }, oMainPanel,,,,,.T.) //'Descrição / Código / Valor'
228 oLblList:SetCSS( POSCSS (GetClassName(oLblList), CSS_LABEL_FOCAL ))
229
230 oGetList := TListBox():Create(oMainPanel, oPanelMVC:nHeight/6.73, POSHOR_1, {||u| If(PCount())>0,cGetList:=u,cGetList}}, , LARG_LIST_CONSULT , oPanelMVC:nHeight/7 ,{||
231                                     },,,,,.T.,,bConfirm,oListFont)
232 oGetList:SetCSS( POSCSS (GetClassName(oGetList), CSS_LISTBOX ))
233
234 //Get de Quantidade
235 oLblQuantidade := TSay():New(oPanelMVC:nHeight/3.3,POSHOR_1,{||STR0017},oMainPanel,,,,,.T.,,,100) //"Quantidade"
236 oLblQuantidade:SetCSS( POSCSS (GetClassName(oLblQuantidade), CSS_LABEL_NORMAL ))
237
238 oGetQtde:= TGet():New( oPanelMVC:nHeight/3,POSHOR_1,{||u| If(PCount())>0,nGetQtde:=u,nGetQtde}}, ;
239 oMainPanel,40 ,ALTURAGET,"@E 999",{|| nGetQtde > 0 .AND. nGetQtde <= 999 },,,,,.T.,,,,,,"nGetQtde",,,,,.T.)
240 oGetQtde:SetCSS( POSCSS (GetClassName(oGetQtde), CSS_GET_NORMAL ))
241
242
243 //Get de valor de desconto
244 oLbLVlrDesc := TSay():New(oPanelMVC:nHeight/3.3,oPanelMVC:nWidth/7,{||STR0018},oMainPanel,,,,,.T.,,,100) //"Valor de Desconto"
245 oLbLVlrDesc:SetCSS( POSCSS (GetClassName(oLbLVlrDesc), CSS_LABEL_NORMAL ))
246
247 oGetVlrDesc:= TGet():New( oPanelMVC:nHeight/3,oPanelMVC:nWidth/7,{||u| If(PCount())>0,nGetVlrDesc:=u,nGetVlrDesc}}, ;
248 oMainPanel,60 ,ALTURAGET,"@E 999,999.99",,,,,.T.,,,,,,"nGetVlrDesc",,,,,.T.)
249 oGetVlrDesc:SetCSS( POSCSS (GetClassName(oGetVlrDesc), CSS_GET_NORMAL ))
250
251 //percentual de Desconto
252 oLbLPercDesc := TSay():New(oPanelMVC:nHeight/3.3,oPanelMVC:nWidth/3.5,{||STR0019},oMainPanel,,,,,.T.,,,100) //"Percentual de Desconto"
253 oLbLPercDesc:SetCSS( POSCSS (GetClassName(oLbLPercDesc), CSS_LABEL_NORMAL ))
254
255 oGetPercDesc:= TGet():New( oPanelMVC:nHeight/3,oPanelMVC:nWidth/3.5,{||u| If(PCount())>0,nGetPercDesc:=u,nGetPercDesc}}, ;
256 oMainPanel,60 ,ALTURAGET,"@E 99.99",,,,,.T.,,,,,,"nGetPercDesc",,,,,.T.)

```

```

257 oGetPercDesc:SetCSS( POSCSS (GetClassName(oGetPercDesc), CSS_GET_NORMAL ))
258
259 oLblSimbPerc := TSay():New(oPanelMVC:nHeight/2.9, oPanelMVC:nWidth/3.5 + ((oGetPercDesc:nWidth/2) + 3 ) ,{||"%"},oMainPanel,,,,,.T.,,,100)
260 oLblSimbPerc:SetCSS( POSCSS (GetClassName(oLblSimbPerc), CSS_LABEL_NORMAL ))
261
262 //Selecaoanr produto
263 oButton := TButton():New( POSVERT_BTNFOCAL,POSHOR_BTNFOCAL ,STR0004,oMainPanel,bConfirm, ; //"Selecionar Produto"
264 LARGBTN,ALTURABTN,,,,.T.,,,{||IIF(Len(aRecno) > 0, .T., .F.)})
265 oButton:SetCSS( POSCSS (GetClassName(oButton), CSS_BTN_FOCAL ))
266
267 //Cancelar
268 oButCan := TButton():New( POSVERT_BTNFOCAL,POSHOR_1,STR0013,oMainPanel,{|| STIRetItem() }, ; //"Cancela Pesquisa"
269 LARGBTN,ALTURABTN,,,,.T.,,,{||.T.})
270 oButCan:SetCSS( POSCSS (GetClassName(oButCan), CSS_BTN_NORMAL ))
271
272 SetKey(19, bConfirm)
273 SetKey(1, {|| STIRetItem() })
274
275 oGetSearch:SetFocus()
276
277 Return oMainPanel
278
279
280 //-----
281 /*{Protheus.doc} STISearchProd
282 Responsavel pela criação do filtro que sera passado ao Browse, com base nas informacoes digitadas pelo usuario.
283
284 @param cGetProduct - String digitada pelo usuario utilizada para pesquisar o produto.
285 @param cGetList - Campo que será utilizado para pesquisa.
286 @param lFindAll - Busca todos os registros e traz os primeiros que achar
287 */
288 //-----
289 Static Function STISearchProd(cGetProduct, oGetList, oGetSearch , lFindAll )
290 Local aArea := GetArea()
291 Local nSec1, nSec2
292 Local nLimit := SuperGetMV("MV_LJQTDPL",,20)
293 Local lMinGetSize := Len(AllTrim(oGetSearch:cText)) >= 3 .Or. ( Len(oGetSearch:cText) < 3 .AND. SubStr(oGetSearch:cText,1,1) $ "0123456789" )
294 Local aInfoItem := {}
295 Local aProducts := {}
296 Local nMoeda := STBGetCurrency()
297 Local oCliModel := STDGCLiModel() // Model do Cliente
298 Local cCliCode := oCliModel:GetValue("SA1MASTER","A1_COD")
299 Local cCliLoja := oCliModel:GetValue("SA1MASTER","A1_LOJA")
300 Local cMoedaSimb := SuperGetMV( "MV_SIMB" + Str(nMoeda,1 ) )
301 Local cFilter := IIF(FindFunction("STDIsPrdLike"), AllTrim(cGetProduct), "" + AllTrim(cGetProduct) + "" $ AllTrim(SB1->B1_DESC) .AND. SB1->B1_MSQLQL<>'1'") // Filtro
    que será utilizado para pesquisar o cliente desejado.
302 Local lRet := .T.
303 Local lPAFECE := STFGetCfg("lPafEcf")
304 Local cMsgAlert := ""
305 Local nPrice := 0
306 Local cFields := SuperGetMV("MV_LJCMP",,"")
307

```

```

308 Default lFindAll := .F. //Busca todos os registros e traz os primeiros que achar
309
310 nSec1 := Seconds()
311 oGetList:Reset()
312 aRecno := {}
313
314
315 If !Empty(cGetProduct) .OR. lFindAll
316
317     STFMessage(ProcName(),"STOP",STR0016) //"Pesquisando produto. Aguarde..."
318     STFShowMessage(ProcName())
319
320     CursorWait()
321
322     /* Busca item na base de dados */
323     aInfoItem := STWFindItem( cGetProduct, lPAFECE, lHomolPaf, lOnlyCodBar )
324
325     // Encontrou o item?
326     If !lFindAll .AND. aInfoItem[ITEM_ENCONTRADO]
327         If !aInfoItem[ITEM_BLOQUEADO]
328
329             If ExistFunc("STDPrckit") .AND. aInfoItem[ITEM_TIPO] == "KT"
330                 nPrice := STDPrckit( aInfoItem[ITEM_CODIGO] , cCliCode , Nil , cCliLoja , ;
331                                     nMoeda )
332             Else
333                 nPrice := STWFormPr( aInfoItem[ITEM_CODIGO] , cCliCode , Nil , cCliLoja , ;
334                                     nMoeda )
335                 nPrice := STBArred( nPrice * Iif(Empty(aInfoItem[ITEM_QTDE]),1, aInfoItem[ITEM_QTDE]))
336             Endif
337
338             If nPrice > 0
339                 nPrice := STBRound( nPrice )
340                 aAdd(aRecno,SB1->(Recno()))
341                 If !lPAFECE
342                     aAdd(aProducts,AllTrim(aInfoItem[ITEM_DESCRICA0])+" / "+AllTrim(aInfoItem[ITEM_CODIGO])+" /
343 "+cMoedaSimb+AllTrim(Transform(nPrice,PesqPict("SL1","L1_VLRTOT"))))
344                 Else
345                     aAdd(aProducts,AllTrim(aInfoItem[ITEM_DESCRICA0])+" / "+;
346                         AllTrim(aInfoItem[ITEM_CODIGO])+" / "+;
347                         cMoedaSimb+AllTrim(Transform(nPrice,PesqPict("SL1","L1_VLRTOT")))+" / "+;
348                         AllTrim(aInfoItem[ITEM_UNID_MEDIDA] )+" / "+;
349                         AllTrim(aInfoItem[ITEM_SITTRIB] ) + " / "+;
350                         AllTrim(aInfoItem[ITEM_IAT] )+" / "+;
351                         AllTrim(aInfoItem[ITEM_IPPT] ) //
352                 EndIf
353             EndIf
354         EndIf
355     ElseIf (lMinGetSize .OR. lFindAll) .AND. !lOnlyCodBar
356
357         If FindFunction("STDFindProd")
358             aProducts := STDFindProd( cFilter,          cCliCode,          cCliLoja, nMoeda, ;

```

```

359             nLimit,          lPAFECEf,          lHomolPaf, @aRecno, cFields)
360         Else
361             ConOut("Atualizar o fonte STDFindItem")
362         EndIf
363
364     Else
365         lRet := .F.
366         cMsgAlert := STR0005 //"É necessário digitar pelo menos 3 caracteres."
367     EndIf
368 EndIf
369
370 oGetList:SetArray(aProducts)
371
372 nSec2 := Seconds()-nSec1
373
374 ConOut("-----")
375 ConOut("0 tempo de pesquisa foi de "+AllTrim(Str(nSec2))+ " segundos.")
376 ConOut("-----")
377
378 If lMinGetSize
379     If nPrice == -999
380         cMsgAlert := STR0020 + CHR(13)+CHR(10) + STR0021 //"Tabela de preço fora de vigência." "Verifique o código da tabela contido no parâmetro MV_TABPAD"
381         lRet := .F.
382     ElseIf Len(aProducts) == 0
383         cMsgAlert := STR0006 //"Nenhum produto encontrado."
384         lRet := .F.
385     Else
386         If Len(aProducts) == nLimit
387             cMsgAlert := STR0007+" "+AllTrim(Str(Len(aProducts)))+ " "+STR0008 //"0 resultado foi limitado a" .. "produtos. Refine sua busca se for necessário."
388         ElseIf Len(aProducts) == 1
389             cMsgAlert := STR0009 //"1 produto foi encontrado."
390         Else
391             cMsgAlert := STR0010+" "+AllTrim(Str(Len(aProducts)))+ " "+STR0011 //"Foram encontrados" .. "produtos."
392         EndIf
393
394         oGetList:SetFocus()
395         oGetList:GoTop()
396
397     EndIf
398 EndIf
399
400 STFCleanInterfaceMessage()
401
402 If !Empty(cMsgAlert)
403     STFMessage(ProcName(),"STOP",cMsgAlert) //Exibe mensagem de alerta ao usuário
404     STFShowMessage(ProcName())
405 EndIf
406
407 CoNout(Time())
408 RestArea(aArea)
409 CursorArrow()
410

```

```

411 Return lRet
412
413 //-----
414 /*{Protheus.doc} STIConfirmProd
415 Responsavel por registrar o produto selecionado na venda.
416
417 @param oGetList      - Objeto da lista de produtos
418 @param cQtdDesc      - Texto para desconto e quantidade funciona idem ao get de produtos
419 */
420 //-----
421 Static Function STIConfirmProd( oGetList , cQtdDesc)
422 Local nRecno      := 0
423 Local nTotItem    := oGetList:Len()
424 Local nPos        := oGetList:GetPos()
425 Local lGrvPesq := AliasInDic("MFL") .AND. SuperGetMV("MV_LJGRVCS", .F., 0) == 1 //Valida se grava Consulta de produtos
426
427 Default cQtdDesc := ""
428
429 If Empty(cQtdDesc)
430     cQtdDesc := AllTrim(STIGetProd())
431 EndIf
432 If nPos > 0
433     nRecno := aRecno[nPos]
434     cGetProduct := Iif(lOnlyCodBar, Space(TamSx3("LK_CODBAR")[1]), Space(40))
435     SB1->(DbGoTo(nRecno))
436
437     If !(SuperGetMV("MV_LJMTDIT",,.F.) .AND. ("% $ cQtdDesc .Or. "-" $ cQtdDesc))
438         STIRegItemInterface(,.F.)
439     EndIf
440
441     STIItemregister(cQtdDesc + SB1->B1_COD)
442
443     // Grava Consulta na tabela MFL
444     If lGrvPesq // Se existir tabela MFL - Consulta de Preco pelo PDV
445         STIRecProdMFL({{SB1->B1_COD}})
446     EndIf
447     aRecno := {}
448 ElseIf nPos == 0 .And. nTotItem > 0 // Caso exista um item no ListBox mas ele não esteja posicionado, força o foco no objeto oGetList para atualizar a tela e não deixar o
449     botão "Selecionar Produto" travado pelo mouse
450     oGetList:SetFocus()
451 EndIf
452 Return .T.
453
454 //-----
455 /*{Protheus.doc} STIRetItem
456 Quando clicar em cancelar volta para a tela de registro de item
457
458 @param
459 */
460 //-----
461 Static Function STIRetItem()

```

```

462 CursorArrow()
463 cGetProduct := Iif(lOnlyCodBar, Space(TamSx3("LK_CODBAR")[1]), Space(40))
464 STIRegItemInterface()
465 Return .T.
466
467 //-----
468 /*{Protheus.doc} STIChangeInfo
469 Troca a imagem e as informacoes do produto ao rolar a lista de produtos.
470
471 @param
472 */
473 //-----
474 Static Function STIChangeInfo( oList, oImg )
475 Local nRecno      := 0                                // Recno do produto na SB1
476 Local nPos        := oList:GetPos()                  // Linha posicionada da listbox
477 Local aArea       := GetArea()                       // Guarda area corrente
478 Local nMoeda      := STBGetCurrency()                 // Moeda corrente
479 Local cMoedaSimb:= SuperGetMV( "MV_SIMB" + Str(nMoeda,1 ) ) // Simbolo da moeda corrente
480 Local cProdData  := ""                                // Dados do produto
481 Local oPanelMVC  := STIGetPanel()                    // Panel onde serao exibidas as informacoes do produto
482 Local nPrice     := 0                                // Preco do produto
483 Local cProdVal   := ""
484 Local cPathImg   := AllTrim(SuperGetMV("MV_LJIMAGE", .F., "")) // Caminho da imagem Opcional
485 Local cExtImg    := AllTrim(SuperGetMV("MV_LJIMGEX", .F., ".jpg")) // extensão da imagem Opcional
486 Local cMVLJB1COD:= AllTrim(SuperGetMV("MV_LJB1COD", "B1_COD")) //Campo que será utilizado para consulta da imagem do produto
487 Local cNomeImg  := ""
488
489 If nPos > 0 .AND. nPos <= Len(aRecno)
490     nRecno := aRecno[nPos]
491     cGetProduct := Iif(lOnlyCodBar, Space(TamSx3("LK_CODBAR")[1]), Space(40))
492     SB1->(DbGoTo(nRecno))
493
494     nPrice := STWFormPr( SB1->B1_COD, STDGPBasket( "SL1", "L1_CLIENTE" ), Nil, STDGPBasket( "SL1" , "L1_LOJA" ), nMoeda )
495
496     cProdData := SB1->B1_DESC+CRLF+CRLF+CRLF+AllTrim(SB1->B1_COD)
497     cProdVal  := CRLF+CRLF+CRLF+cMoedaSimb+AllTrim(Transform(nPrice,PesqPict("SL1","L1_VLRTOT")))
498
499     If ValType(oLblProdDados) <> "U"
500         oLblProdDados:SetText(cProdData)
501         oLblProdVal:SetText(cProdVal)
502     Else
503         oLblProdDados:= TSay():New(oPanelMVC:nHeight/3.6378+10,POSHOR_1,{||cProdData},oMainPanel,,,,,T,,,,100)
504         oLblProdDados:SetCSS( POSCSS (GetClassName(oLblProdDados), CSS_LABEL_FOCAL ))
505
506         oLblProdVal := TSay():New(oPanelMVC:nHeight/3.6378+10,POSHOR_1+130,{||cProdVal},oMainPanel,,,,,T,,,,100)
507         oLblProdVal:SetCSS( POSCSS (GetClassName(oLblProdVal), CSS_LABEL_FOCAL ))
508     EndIf
509
510     //Verifica se usa foto de arquivo
511     If Empty(cPathImg)
512         cNomeImg := AllTrim(SB1->B1_BITMAP)
513         If !Empty(cNomeImg) .And. oImg:ExistBmp(cNomeImg)

```

```

514         ShowBitmap(oImg,cNomeImg,"SEMFOTO")
515         oImg:lVisibleControl := .T.
516     Else
517         oImg:lVisibleControl := .F.
518     EndIf
519 Else
520     cNomeImg := AllTrim(SB1->(cMVLJB1COD))
521     If oImg:Load( Nil ,cPathImg + cNomeImg + cExtImg )
522         oImg:lVisibleControl := .T.
523     Else
524         oImg:SetEmpty()
525         oImg:lVisibleControl := .F.
526     EndIf
527 EndIf
528
529 EndIf
530
531 RestArea(aArea)
532
533 Return
534 //-----
535 /*{Protheus.doc} STLostFocus
536 Rotina executada ao mudar o foco do componente
537
538 @param oGetSearch - Objeto contendo o componente que será executada a rotina de perda de foco
539 @param oGetList    - Objeto da lista de produtos
540 @param lFindAll    - Busca todos os registros e traz os primeiros que achar
541 */
542 //-----
543 Static Function STLostFocus(oGetSearch,oGetList, cGetProduct, lBusca ,;
544                             lFindAll)
545
546 Default lFindAll := .F. //Busca todos os registros e traz os primeiros que achar
547
548 lBusca := .T.
549 If !Empty(cGetProduct) .OR. ( Empty(cGetProduct) .AND. lFindAll)
550     IF !STISearchProd(cGetProduct, oGetList, oGetSearch ,lFindAll )
551         cGetProduct:= Iif(lOnlyCodBar, Space(TamSx3("LK_CODBAR")[1]), Space(40))
552         oGetSearch:SetFocus()
553     EndIf
554 EndIf
555
556 Return .T.

```

Componentes/Interface/STIRelmpSale.PRW

268 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "POSCSS.CH"
4 #INCLUDE "STPOS.CH"
5 #INCLUDE "STIREIMPSALE.CH"
6
7 Static cSiglaSat      := IIF( ExistFunc("LjSiglaSat"),LjSiglaSat(), "SAT" )    //Retorna sigla do equipamento que esta sendo utilizado
8
9 //-----
10 /*{Protheus.doc} STIRelmpSale
11 Chamada Interface Reimpressão de Venda
12 @param   cGetSale - Numero da Venda
13 @param   cGetSerie - Série da Venda
14 @return
15 @obs
16 @sample
17 */
18 //-----
19 Function STIRelmpSale(cGetSale, cGetSerie)
20
21 Local nTamDoc := TamSx3("L1_DOC")[1] //Total do documento
22 Local lUseSat  := STFGetCfg("lUseSAT",.F.) //utiliza SAT
23
24 Default cGetSale := ""
25 Default cGetSerie := ""
26
27 If lUseSat
28     nTamDoc := 6
29 EndIf
30
31 //Verifica se não tem NFce em aberto
32
33 If !Empty(cGetSale) .AND. !Len(AllTrim(cGetSale)) > 6
34     cGetSale := Strzero( Val(cGetSale),nTamDoc)
35 EndIf
36
37 If !Empty(cGetSale)
38     STIRelSale(cGetSale, cGetSerie)
39 EndIf
40
41
42 Return
43
44 //-----
45 /*{Protheus.doc} STIPanReimpSale
46 Contexto para Reimpressao de Venda
47
48 @param
49 @return
50 @obs
51 @sample
52 */

```

```

53 //-----
54 Function STIPanReimpSale()
55
56 Local oPanSaleRemp := Nil           // Painel Reimpressão de Venda
57 Local oPanelMVC    := STIGetPanel() // Painel Principal
58 Local oBtnConfirm  := Nil           // Botao de confirmação
59 Local oBtnCancel   := Nil
60 Local oGetDoc       := Nil           // Get do Doc a ser Reimpresso
61 Local oLblSerie     := Nil
62 Local oGetSerie     := Nil
63 Local lEmitNfce     := Iif(ExistFunc("LjEmitNfCe"), LjEmitNfCe(), .F.) // Sinaliza se utiliza NFC-e
64 Local nGetHeiPerc   := 90           // Percentual da Altura do Get de Busca
65 Local oLblCancel    := Nil           // Objeto Label do cancelamento
66 Local cGetSale      := Space(TamSX3("L1_DOC")[1]) // Conteudo inserido no Get de Cancelamento.
67 Local cGetSerie     := STFGetStation("SERIE")
68 Local lUseSat       := STFGetCfg("lUseSAT",.F.) //utiliza SAT
69 Local cTextBtn      := STR0004 //"Reimprimir NFCe"
70
71 If lUseSat
72     cTextBtn := StrTran(STR0006,"SAT",cSiglaSat) //"Reimprimir SAT"
73 EndIf
74
75 //Captura o último documento emitido somente se a nfce estiver emitida
76 If STWPrinterStatus("5") <> 7
77 //Caputa o numero do ultimo documento emitido
78     STWNFCEInfo("2", @cGetSale)
79 EndIf
80
81 If Val(cGetSale) > 0
82     cGetSale := PadL(AllTrim(cGetSale), SL1->(TamSX3("L1_DOC")[1]), "0")
83 Else
84     //Caputa o numero do ultimo documento emitido na ultima venda emitida
85     cGetSale := STDCLastSale("L1_DOC", @cGetSerie)
86 EndIf
87 /*/
88     Panel Abertura de Caixa
89 /*/
90 oPanSaleRemp := TPanel():New(00,00,"",oPanelMVC,,,,,oPanelMVC:nWidth/2,(oPanelMVC:nHeight)/2)
91
92 oSay1 := TSay():New(POSVERT_CAB,POSHOR_1,{||STR0001},oPanSaleRemp,,,,,T,,,,) //"Confirma a reimpressão do documento?"
93 oSay1:SetCSS( POSCSS (GetClassName(oSay1), CSS_BREADCUMB ))
94
95 If lEmitNfce .AND. STFSaleTotal() == 0
96
97     oLblCancel := TSay():New(POSVERT_LABEL1,POSHOR_1,{||STR0002},oPanSaleRemp,,,,,T,,,,) //"Número do documento"
98     oLblCancel:SetCSS( POSCSS (GetClassName(oLblCancel), CSS_LABEL_FOCAL ))
99
100     oGetDoc := TGet():New(POSVERT_GET1,POSHOR_1,{|u| If(PCount())>0,cGetSale:=u,cGetSale}}, oPanSaleRemp,nGetHeiPerc,,,/*bValid*/,,,,,,T,,,,,"cGetSale")
101     oGetDoc:SetCSS( POSCSS (GetClassName(oGetDoc), CSS_GET_FOCAL ))
102
103
104     oLblSerie := TSay():New(POSVERT_LABEL1,POSHOR_2,{||STR0003},oPanSaleRemp,,,,,T,,,,) //"Série do documento"

```

```

105     oLblSerie:SetCSS( POSCSS (GetClassName(oLblCancel), CSS_LABEL_FOCAL ))
106
107
108     oGetSerie := TGet():New(POSVERT_GET1,POSHOR_2,{|u| If(PCount()>0,cGetSerie:=u,cGetSerie)}, oPanSaleRemp;;
109         nGetHeiPerC,,,/*bValid*/;;
110         ,,,,;
111         ,.T.,,,,;
112         ,,,,;
113         /*.F.*/,,"cGetSerie")
114     oGetSerie:SetCSS( POSCSS (GetClassName(oGetDoc), CSS_GET_FOCAL ))
115
116     iif (!lUseSat, oGetSerie:Disable(), nil)
117
118 EndIf
119
120 oBtnConfirm := TButton():New( POSVERT_BTNFOCAL,POSHOR_BTNFOCAL,cTextBtn,oPanSaleRemp,{ || STIReimpSale(cGetSale, cGetSerie), STIRegItemInterface() }, ; //"Reimprimir NFce"
    ou "Reimprimir SAT"
121         LARGBTN,ALTURABTN,,,,.T.)
122
123 oBtnCancel := TButton():New( POSVERT_BTNFOCAL,POSHOR_1,STR0005,oPanSaleRemp,{ || STIRegItemInterface() }, ; //"Cancelar"
124         LARGBTN,ALTURABTN,,,,.T.)
125
126 oBtnConfirm:SetCSS( POSCSS (GetClassName(oBtnConfirm), CSS_BTN_FOCAL ))
127 oBtnCancel:SetCSS( POSCSS (GetClassName(oBtnCancel), CSS_BTN_ATIVO ))
128
129 Return oPanSaleRemp
130
131
132 //-----
133 /*/{Protheus.doc} STIReSale
134 Acionado Cancelamento de Venda
135
136 @param
137 @return Nil
138 @obs
139 @sample
140 */
141 //-----
142 Function STIReSale(cGetSale, cGetSerie)
143
144 Local lUseSat := STFGetCfg("lUseSAT",.F.) //utiliza SAT
145
146 Default cGetSale := ""
147 Default cGetSerie := ""
148
149
150 If lUseSat
151     LJSatReImp( cGetSale, cGetSerie )
152 Else
153     // Função STWNFCEReImp() descontinuada
154 EndIf
155

```

```

156 STIGridCupRefresh() // Sincroniza a Cesta com a interface
157
158 STIRegItemInterface()
159
160 Return Nil
161
162
163 /*{Protheus.doc} STIPANRINFCE
164     Tela para seleção de cupom nfc-e para reimprimir. Se não informar nada, por default imprime o último
165     @type Function
166     */
167 Function STIPANRINFCE()
168
169 Local oPanReimp      := Nil
170 Local oPanelMVC      := STIGetPanel()
171 Local oBtnConfirm    := Nil
172 Local oBtnCancel     := Nil
173 Local oLblReImp      := Nil
174 Local oGetDoc        := Nil
175 Local nGetHeiPerc    := 90
176 Local cGetDOC        := Space(TamSX3("L1_DOC")[1])
177 Local cGetSerie      := STFGetStation("SERIE")
178 Local oSay1          := NIL
179 Local aInfCupom      := {}
180
181
182 cGetDOC := STDCSLastSale("L1_DOC")
183
184 oPanReimp := TPanel():New(00,00,"",oPanelMVC,,,,,oPanelMVC:nWidth/2,(oPanelMVC:nHeight)/2)
185 oPanReimp:bGotFocus := { || STWReaderFocus(.T.) }
186
187 oSay1:= TSay():New(POSVERT_CAB,POSHOR_1,{||STR0001},oPanReimp,,,,,.T.,,,)
188 oSay1:SetCSS( POSCSS (GetClassName(oSay1), CSS_BREADCUMB ))
189
190 oBtnConfirm := TButton():New( POSVERT_BTNFOCAL,POSHOR_BTNFOCAL,STR0004,oPanReimp,{ || IF( STIValNDoc(cGetDOC, cGetSerie, @aInfCupom
191     ),(LjNFCeReImp(aInfCupom),STIEXIT()),oGetDoc:SetFocus())},LARGBTN,ALTURABTN,,,,.T.)
192 oBtnConfirm:SetCSS( POSCSS (GetClassName(oBtnConfirm), CSS_BTN_FOCAL ))
193
194
195 oBtnCancel := TButton():New( POSVERT_BTNFOCAL,POSHOR_1,STR0005,oPanReimp,{ ||STIEXIT()}, LARGBTN,ALTURABTN,,,,.T.)
196 oBtnCancel:SetCSS( POSCSS (GetClassName(oBtnCancel), CSS_BTN_NORMAL ))
197
198 oBtnCancel:bLostFocus := {|| oBtnConfirm:SetFocus() }
199
200 oLblReImp := TSay():New(POSVERT_LABEL1,POSHOR_1,{|| STR0003+Space(10)+STR0002 },oPanReimp,,,,,.T.,,,)
201 oLblReImp:SetCSS( POSCSS (GetClassName(oLblReImp), CSS_LABEL_FOCAL ))
202
203 oGetSerie:= TGet():New(POSVERT_GET1,POSHOR_1,{|u| If(PCount())>0,cGetSerie:=u,cGetSerie)},oPanReimp,nGetHeiPerc-70,,,/*bValid*/,,,,.T.,,,,,,"cGetSerie")
204 oGetSerie:SetCSS( POSCSS (GetClassName(oGetSerie), CSS_GET_FOCAL ))
205
206 oGetDoc := TGet():New(POSVERT_GET1,POSHOR_1 + 80,{|u| If(PCount())>0,cGetDOC:=u,cGetDOC)},oPanReimp,nGetHeiPerc,,,/*bValid*/ ,,,,,.T.,,,,,,"cGetDOC")

```

```

207 oGetDoc:SetCSS( POSCSS (GetClassName(oGetDoc), CSS_GET_FOCAL ))
208
209 oGetDoc:bLostFocus := { || IF( STIVaNDoc(cGetDOC, cGetSerie, @aInfCupom ),oBtnConfirm:SetFocus(),oGetDoc:SetFocus()) }
210
211 Return oPanReimp
212
213 /*/{Protheus.doc} STIREIMPNFCE()
214     REIMPRESSÃO DO CUPOM NFC-E PELO TOTVS PDV.
215     @type Function
216     */
217 Function STIREIMPNFCE()
218
219
220 STIExchangePanel( { || STIPANRINFCE() } )
221
222 Return
223
224 /*/{Protheus.doc} STIEXIT()
225     SAIDA DA INTERFACE DA REIMPRESSAO
226     @type Function
227     */
228 Function STIEXIT()
229
230 STIExchangePanel( { || STIPanItemRegister() } )
231
232 Return
233
234 /*/{Protheus.doc} STIVaNDoc
235     FAZ DBSEEK NA SL1 PARA VER SE O CUPOM PODE SER REIMPRESSO
236     @type Static Function
237     @param cDOC, character , numero do documento a ser reimpresso
238     @param cSerie, character , número de série do cupom a ser reimpresso
239     @param aInfCupom, array , Retorna array com dados do cupom a ser reimpresso
240     @return lRet, lógico, .T. se conseguiu encontrar o cupom a ser reimpresso
241     */
242 Static Function STIVaNDoc(cDOC, cSerie, aInfCupom)
243 Local aAreaSL1 := SL1->(GetArea())
244 Local lRet := .F.
245
246 DbSelectArea("SL1")
247 SL1->(DbSetOrder(2))// L1_FILIAL+L1_SERIE+L1_DOC+L1_PDV
248 SL1->(DbSeek(xFilial("SL1")+PadR(AllTrim(cSerie),TamSX3("L1_SERIE")[1])+PadR(AllTrim(cDOC),TamSX3("L1_DOC")[1])))
249
250 lRet:= !SL1->(EOF()) .AND. !Empty(SL1->L1_DOC) .AND. !Empty(SL1->L1_KEYNFCE)
251
252 aInfCupom:={}
253 STFCleanInterfaceMessage()
254
255 If lRet
256     AADD(aInfCupom, SL1->L1_FILIAL)
257     AADD(aInfCupom, SL1->L1_NUM)
258     AADD(aInfCupom, SL1->L1_SITUA)

```

```
259     AADD(aInfCupom, SL1->L1_KEYNFCE)
260     AADD(aInfCupom, SL1->L1_DOC)
261 Else
262     STFMessage(ProcName(), "STOP", STR0007)
263     STFShowMessage(ProcName())
264 Endif
265
266 RestArea(aAreaSL1)
267
268 Return lRet
```

Componentes/Interface/STIReasonForDiscount.PRW

240 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "POSCSS.CH"
4 #INCLUDE "STIREASONFORDISCOUNT.CH"
5
6 #Define cMTDesIt    "1"    //Motivo de Desconto Item
7 #Define cMTDesTo    "2"    //Motivo de Desconto Total
8 #Define cMTDesAm    "3"    //Motivo de Desconto Ambos
9
10 Static aReasons      := {}    // Armazena Motivo de Desconto Interface
11 Static cReasonSelected := ""    // Armazena Código de Motivo de Desconto Selecionado
12 Static cObs          := ""    // Observacao do Motivo de Desconto (conforme tabela MDT)
13 //-----
14 /*{Protheus.doc} STIGetReason
15 Retorna o Motivo de Desconto selecionado
16
17 @param
18 @return cReasonSelected    Retorna o Motivo de Desconto selecionado
19 @obs
20 @sample
21 */
22 //-----
23 Function STIGetReason()
24
25 Return cReasonSelected
26
27 //-----
28 /*{Protheus.doc} STIGetObsReason
29 Retorna Descrição do Motivo de Desconto selecionado
30
31 @param
32 @return cObs    Retorna DDescrição do Motivo de Desconto selecionado
33 @obs
34 @sample
35 */
36 //-----
37 Function STIGetObsReason()
38
39 Return cObs
40
41 //-----
42 /*{Protheus.doc} STIReasons
43 Define os motivos de desconto a serem apresentados na interface
44
45 @param
46 @return aReasons    Retorna os Motivos de Desconto a serem apresentados na interface
47 @obs
48 @sample
49 */
50 //-----
51 Function STIReasons()
52

```

```

53 Local aDataReasons := {}      // Armazena os motivos de desconto cadastrados
54 Local nI           := 0      // Contador
55
56 If Len(aReasons) <= 0
57
58     /*/
59     Busca Registros multimidia cadastrados
60     /*/
61     aDataReasons := STDRFDGet(cMTDesIt)
62
63     For nI := 1 To Len(aDataReasons)
64
65         AADD( aReasons , cValToChar(nI) + " - " + aDataReasons[nI][2] )
66
67     Next nI
68
69 EndIf
70
71 Return aReasons
72
73
74 //-----
75 /*/{Protheus.doc} STISetReason
76 Armazena o Motivo de Desconto Selecionado
77
78 @param  oReasons      ListBox de Motivo de Desconto
79 @return
80 @obs
81 @sample
82 /*/
83 //-----
84 Function STISetReason(oReasons,nItemLine,cCodItem,cTesPad,nTotItem,oPanReasonD)
85
86 Local cSelected      := ""      // Armazena a Opção selecionada no ListBox
87 Local nSelected      := 0      // Armazena posição numérica da opção selecionada
88 Local aDataReasons   := {}      // Armazena os motivos de desconto cadastrados
89 Local cType          := "VALOR" //Tipo de Desconto - 'VALOR' para desconto no Total e 'ITEM' para desconto no item.
90
91 Default oReasons     := Nil
92 Default nItemLine    := 0
93 Default cCodItem     := ""
94 Default cTesPad      := ""
95 Default nTotItem     := 0
96 Default oPanReasonD  := Nil
97
98 If oReasons <> Nil .AND. oReasons:lvisiblecontrol == .T.
99
100     cSelected      := oReasons:GetSelText()
101     cSelected      := SubStr( cSelected , 1 , 1 )
102
103     nSelected      := Val(cSelected)
104

```

```

105   aDataReasons   := STDRFDGet(cMTDesIt)
106
107   cReasonSelected := aDataReasons[nSelected][1]
108   cObs            := aDataReasons[nSelected][2]
109
110
111   If !(cReasonSelected == "")
112
113       If !Empty(cCodItem) .AND. nItemLine > 0
114           STWSetRegIt(nItemLine,cCodItem,cTesPad,nTotItem)
115           cType := "ITEM"
116
117           Iif(ExistFunc("StiMataObj"),StiMataObj(oPanReasonD),Nil)
118
119           STIExchangePanel( { || STIPanItemRegister() })
120       Else
121           STDAddReason( nItemLine , cReasonSelected , cObs , cCodItem , cType )
122       EndIf
123
124
125   EndIf
126
127 EndIf
128
129 Return
130
131 //-----
132 /*/{Protheus.doc} STIReasonD
133 Monta a Tela contendo todos os motivos validos
134
135 @param   oReasons           ListBox de Motivo de Desconto
136 @return  oPanReasonD
137 @obs
138 @sample
139 */
140 //-----
141
142 Function STIReasonD(nItemLine,cCodItem,cTesPad,nTotItem)
143 Local aReasonD      := STIReasons()           //Retorna um array contendo todos os Motivos cadastrados
144 Local lRet          := .T.                   //Variavel de retorno da funcao
145 Local oReasonD      := Nil                   //ListBox para Motivos de Desconto
146 Local oPanReasonD   := Nil                   //Panel para os Motivos de Desconto
147 Local oPanelMVC := STIGetDlg()               //Dialog da tela principal
148 Local oButton       := Nil                   //Botao Confirma
149 Local oBtnCanc      := Nil                   //Botao Cancela
150
151 //Botão "Adicionar Mídia"
152 Local nBtnWidPerc    := oPanelMVC:nWidth/2 * 0.25           //Percentual de Largura do Botão "Adicionar Mídia"
153 Local nBtnHeiPerc    := oPanelMVC:nHeight/2 * 0.07          //Percentual de Altura do Botão "Adicionar Mídia"
154 Local nBtnVerPerc    := (oPanelMVC:nHeight/2 * 0.02) * 14   //Percentual da Posição Vertical do Botão "Adicionar Mídia"
155 Local nBtnHorPerc    := oPanelMVC:nWidth/2 * 0.40           //Percentual da Posição Horizontal do Botão "Adicionar Mídia"
156

```

```

157 //Botão "Cancelar"
158 Local nBtnCwidPerc      := oPanelMVC:nWidth/2 * 0.25           //Percentual de Largura do Botão "Adicionar Mídia"
159 Local nBtnChaiPerc      := oPanelMVC:nHeight/2 * 0.07          //Percentual de Altura do Botão "Adicionar Mídia"
160 Local nBtnCVerPerc      := (oPanelMVC:nHeight/2 * 0.02) * 14    //Percentual da Posição Vertical do Botão "Adicionar Mídia"
161 Local nBtnCHorPerc      := oPanelMVC:nWidth/2 * 0.71           //Percentual da Posição Horizontal do Botão "Adicionar Mídia"
162
163 //Objetos TSAY
164 Local nPosLb1Alt        := oPanelMVC:nHeight/2 * 0.02          //Posicao: Altura
165 Local nPosLb1Hoz        := oPanelMVC:nWidth/2 * 0.039           //Posicao: Horizontal
166 Local nPosLb2Alt        := (oPanelMVC:nHeight/2 * 0.02) * 3.7    //Posicao: Altura
167 Local nPosLb2Hoz        := oPanelMVC:nWidth/2 * 0.039           //Posicao: Horizontal
168
169 //Panel
170 Local nPosPnlHoz := ((oPanelMVC:nHeight/2)/2 * 0.2) + 135        //Posicao: horizontal do painel
171
172 //ListBox
173 Local nLisVerPerc      := 6                                     //Percentual de Posição Vertical do ListBox
174 Local nLisHorPerc      := 0.039                               //Percentual de Posição Horizontal do ListBox
175 Local nLisHeiPerc      := 0.11                                //Percentual de Altura do ListBox
176
177 //Panel Get de Mídia
178 oPanReasonD := TPanel():New(nPosPnlHoz,00,"",Nil,,,,,(oPanelMVC:nWidth/2),(oPanelMVC:nHeight/2) * 0.60)
179 oPanReasonD:SetCSS( POSCSS (GetClassName(oPanReasonD), CSS_PANEL_CONTEXT ))
180
181 //Objetos
182 oSay1 := TSay():New(nPosLb1Alt,nPosLb1Hoz,{|| "Vendas > Motivos de Desconto"},oPanReasonD,,,,,T,,,,)
183 oSay2 := TSay():New(nPosLb2Alt,nPosLb1Hoz,{|| "Selecione o Motivo:"},oPanReasonD,,,,,T,,,,)
184 oSay1:SetCSS( POSCSS (GetClassName(oSay1), CSS_BREADCUMB ))
185 oSay2:SetCSS( POSCSS (GetClassName(oSay2), CSS_LABEL_FOCAL ))
186
187 //
188 oReasonD := TListBox():Create(oWnd,nRow,nCol,bSetGet,aItems,nWidth,nHeight,bChange,bValid,nClrFore,nClrBack,lPixel,uParam13,bLDBLClick,oFont,uParam16,uParam17,bWhen,uParam19,uParam20,uParam21,uParam22,bRClick) --> oObjeto
189 oReasonD:SetCSS( POSCSS (GetClassName(oReasonD), CSS_LISTBOX ))
190
191 oButton := TButton():New(nBtnVerPerc,nBtnHorPerc,"Confirma",oPanReasonD,{ || Iif(STISelReason(oReasonD),STISetReason(oReasonD,nItemLine,cCodItem,cTesPad,nTotItem,oPanReasonD),Nil) },nBtnWidPerc,nBtnHeiPerc,,,,T.)
192 oButton:SetCSS( POSCSS (GetClassName(oButton), CSS_BTN_FOCAL ))
193
194 oBtnCanc := TButton():New( nBtnCVerPerc,nBtnCHorPerc,"Cancelar",oPanReasonD,{ || Iif(ExistFunc("StiMataObj"),StiMataObj(oPanReasonD),Nil),STIExchangePanel( { || STIPanItemRegister() } ) },nBtnCwidPerc,nBtnChaiPerc,,,,T.)
195 oBtnCanc:SetCSS( POSCSS (GetClassName(oBtnCanc), CSS_BTN_FOCAL ))
196
197 oButton:SetFocus()
198
199 Return oPanReasonD
200
201 //-----
202 /*/{Protheus.doc} STICLeReason
203 Limpa a variavel statica cReasonSelected
204

```

```

205 @param
206 @return Nil
207 @obs
208 @sample
209 /*/
210 //-----
211
212 Function STICleReason()
213
214 cReasonSelected := ""
215 cObs:=""
216
217 return
218
219 //-----
220 /*/{Protheus.doc} STISelReason
221 Valida se é possível prosseguir com o registro do item caso o motivo de desconto esteja ativo.
222
223 @param
224 @return lRet - Indica se é possível prosseguir.
225 @obs
226 @sample
227 /*/
228 //-----
229 Function STISelReason(oReasons)
230 Local lRet := .F.
231
232 If oReasons <> Nil .AND. oReasons:NAT == 0
233     lRet := .F.
234     STFMessage("STISelReason","STOP",STR0001) //"obrigatório selecionar o Motivo de Desconto"
235     STFShowMessage("STISelReason")
236 Else
237     lRet := .T.
238 EndIf
239
240 Return lRet

```

Componentes/Interface/STIRecSelCust.prw

65 linhas após remoção de metadados de autoria/data

```

1 #include 'totvs.ch'
2 #include "POSCSS.CH"
3 #include "STPOS.CH"
4
5
6 //-----
7 /*{Protheus.doc} STIRSelCus()
8 Funcao responsavel pela criacao do Painel de selecao de clientes.
9
10 @param   aCadCli - Array com listagem de cadastros que o Cliente possui que será apresentada para seleção na tela.
11 @return  aRet
12 @obs
13 @sample
14 */
15 //-----
16 Function STIRSelCus(aCadCli)
17
18 Local oPanelMVC      := STIGetPanel()           //Painel principal
19 Local oPanGetCust    := TPanel():New(00,00,"",oPanelMVC,,,,,oPanelMVC:nWidth/2,(oPanelMVC:nHeight)/2) //Objeto do painel
20 Local nTamAltListBox := oPanelMVC:nHeight * 0.19 //Tamanho: Altura do ListBox
21 Local nList          := 1                     //Para o TListBox
22 Local bOk            := {|| STDCallCustTit( nList ) } // Envia linha selecionada
23 Local oLst           := Nil // Objeto do ListBox
24 Local oSay1          := Nil
25 Local oSay2          := Nil
26 Local oBtnOk         := Nil // Objeto do Botão "Selecionar"
27 Local nLargura       := (oPanGetCust:nWidth / 2) - (2 * ((oPanGetCust:nWidth / 2) * 0.04)) //Largura do ListBox
28
29 /*oSay1 - "Recebimentos de títulos > Selecionar Cliente"*/
30 oSay1 := TSay():New(POSVERT_CAB,POSHOR_1,{||"Recebimentos de títulos > Selecionar Cliente"/*STR000X*/},oPanGetCust,,,,,T,,,,) //"Recebimentos de títulos > Selecionar
    Cliente"
31 oSay1:SetCSS( POSCSS (GetClassName(oSay1),CSS_BREADCUMB ))
32
33 /* oSay2 - "Nome / Código / Loja / CPF/CNPJ" */
34 oSay2 := TSay():New(POSVERT_LABEL1,POSHOR_1,{||/*STR00X*/"Código / Loja / Nome / CPF/CNPJ"},oPanGetCust,,,,,T,,,,) // "Código / Loja / Nome / CPF/CNPJ"
35 oSay2:SetCSS( POSCSS (GetClassName(oSay2),CSS_BREADCUMB))
36
37 /* oLst - Listbox com os cadastros do cliente */
38 oLst := TListBox():Create(oPanGetCust, POSVERT_GET1, POSHOR_1, {||u;if(Pcount(>0,nList:=u,nList)}, aCadCli, nLargura, nTamAltListBox,,;
39     ,,,T,,,bOk/*Acao ao clicar duas vezes*/,,,,,,)
40 oLst:SetCSS( POSCSS (GetClassName(oLst),CSS_LISTBOX ))
41
42 oLst:Select(1)
43 oLst:SetFocus()
44
45 /* Botao Seleciona Cliente */
46 oBtnOk := TButton():New(POSVERT_BTNFOCAL,POSHOR_BTNFOCAL,/*STR00XX*/"Selecionar",oPanGetCust,bOk,LARGBTN,ALTURABTN,,,,T.) //"Selecionar"
47 oBtnOk:SetCSS( POSCSS (GetClassName(oBtnOk),CSS_BTN_FOCAL ))
48
49 Return oPanGetCust
50
51 //-----

```

```
52 /*/{Protheus.doc} STIPnlSelCust
53 Abre o painel para selecao do Cliente
54
55 @param   aCadCli - Array com listagem de cadastros que o Cliente possui que serão apresentada para seleção na tela.
56 @return
57 */
58 //-----
59 Function STIPnlSelCust(aCadCli)
60
61 STIChangeCssBtn()
62 STIBtnDeActivate()
63 STIExchangePanel( { || STIRSelCus(aCadCli) } )
64
65 Return .T.
```

Componentes/Interface/STIRecShopCard.prw

178 linhas após remoção de metadados de autoria/data

```

1 #Include 'Protheus.ch'
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "FWEVENTVIEWCONSTS.CH"
4 #INCLUDE "FWADAPTEREAI.CH"
5 #INCLUDE "FWMVCDEF.CH"
6 #INCLUDE "POSCSS.CH"
7 #INCLUDE "STIRECSHOPCARD.CH"
8
9 //-----
10 /*{Protheus.doc} STIRecShopCard
11
12 @param
13 @return oMainPanel - Objeto contendo o painel principal de selecao de clientes.
14 */
15 //-----
16
17 Function STIRecShopCard(cItemCode)
18
19 STIExchangePanel( { || STIPanRecSC(cItemCode) } )
20
21 Return
22
23 //-----
24 /*{Protheus.doc} STIPanRecSC
25 Funcao responsavel pela criacao do Painel de recarga de Shop Card.
26
27 @return oMainPanel - Objeto contendo o painel principal de selecao de clientes.
28 */
29 //-----
30
31 Static Function STIPanRecSC(cItemCode)
32 Local oPanItCancel      := Nil                // Painel Cancelamento de Venda
33 Local oPanBtnSale       := Nil                // Painel de Botões da Venda
34 Local oPanelMVC         := STIGetPanel()      // Objeto do Panel onde a interface de selecao de clientes sera criada
35 Local oMainPanel       := TPanel():New(00,00,"",oPanelMVC,,,,,oPanelMVC:nWidth/2,(oPanelMVC:nHeight)/2) // Painel de Get de Consulta
36
37 Local nAltGets          := oMainPanel:nHeight/2 * 0.04                //Tamanho: Altura do Get
38
39 /*/
40 Título - "Recarga de Cartão Fidelidade"
41 /*/
42 Local oLblCab           := Nil                // Objeto do label "Recarga de Cartão Fidelidade"
43 Local nSay1HorPerc      := 0.015             // Percentual de Posição Horizontal do Say "Recarga de Cartão Fidelidade"
44
45 /*
46 Say - 'Num. Cartão'
47 */
48 Local oLblNumCart       := Nil                // Objeto do label 'Num. Cartão'
49 Local nSay2HorPerc      := oMainPanel:nWidth/2 * 0.06 // Posicao horizontal do label oLblNumCart
50 Local nSay2VerPerc      := oMainPanel:nHeight/2 * 0.06 // Posicao vertical do label oLblNumCart
51
52 /*/

```

```

53     Get de 'Num. Cartão'
54 /**/
55 Local oGetNumCart      := Nil                // TGet 'Num. Cartão'
56 Local cGetNumCart      := Space(30)          // Variável do Get
57 Local nHorNumCart      := oMainPanel:nWidth/2 * 0.06 // Posição Horizontal do Get
58 Local nVerNumCart      := oMainPanel:nHeight/2 * 0.13 // Posição Vertical do Get
59
60 /**/
61     Say - "Valor do Saldo"
62 /**/
63 Local oLblSaldo         := Nil                // Objeto do label "Valor do Saldo"
64 Local nSay3HorPerc      := oMainPanel:nWidth/2 * 0.06 // Posição Horizontal do Say "Valor do Saldo"
65 Local nSay3VerPerc      := oMainPanel:nHeight/2 * 0.20 // Posição Vertical do Say "Valor do Saldo"
66
67 /**/
68     Get de "Valor do Saldo"
69 /**/
70 Local oGetSaldo         := Nil                // TGet "Valor do Saldo"
71 Local cGetSaldo         := 0                  // Variável do Get
72 Local nGetHorPerc       := oMainPanel:nWidth/2 * 0.06 // Percentual da Posição Horizontal do Get
73 Local nGetVerPerc       := oMainPanel:nHeight/2 * 0.27 // Percentual da Posição Vertical do Get
74
75 /**/
76     Say - "Validade do Saldo"
77 /**/
78 Local oLblDtValid       := Nil                // Objeto do label "Validade do Saldo"
79 Local nSay4HorPerc      := oMainPanel:nWidth/2 * 0.32 // Posição Horizontal do Say "Validade do Saldo"
80 Local nSay4VerPerc      := oMainPanel:nHeight/2 * 0.20 // Posição Vertical do Say "Validade do Saldo"
81
82 /**/
83     Get de "Validade do Saldo"
84 /**/
85 Local oGetDtValid       := Nil                // TGet Validade do Saldo
86 Local cGetDtValid       := dDataBase+30       // Variável do Get
87 Local nHorDtVld        := oMainPanel:nWidth/2 * 0.32 // Posição Horizontal do Get
88 Local nVerDtVld        := oMainPanel:nHeight/2 * 0.27 // Posição Vertical do Get
89
90 /**/
91     Botão 'Confirmar'
92 /**/
93 Local oBtnOk            := Nil                // Botao 'Confirmar'
94 Local nBtnOkLarg        := oMainPanel:nWidth/2 * 0.25 // Largura do Botão 'Confirmar'
95 Local nBtnOkAlt        := oMainPanel:nHeight/2 * 0.08 // Altura do Botão 'Confirmar'
96 Local nBtnOkVer        := oMainPanel:nHeight/2 * 0.8 // Posição Vertical do Botão 'Confirmar'
97 Local nBtnOkHor        := oMainPanel:nWidth/2 * 0.65 // Posição Horizontal do Botão 'Confirmar'
98
99 /**/
100    Botão "Cancelar"
101 /**/
102 Local oBtnCa            := Nil                // Botao "Cancelar"
103 Local nBtnCaLarg        := oMainPanel:nWidth/2 * 0.25 // Largura do Botão "Cancelar"
104 Local nBtnCaAlt        := oMainPanel:nHeight/2 * 0.08 // Altura do Botão "Cancelar"

```

```

105 Local nBtnCaVer      := oMainPanel:nHeight/2 * 0.8    // Posição Vertical do Botão "Cancelar"
106 Local nBtnCaHor      := oMainPanel:nWidth/2 * 0.1     // Posição Horizontal do Botão "Cancelar"
107
108 /*
109     Posicoes do oCustomerPanel
110 */
111 Local nCustPanHor      := 15                          // Posicao horizontal de oCustomerPanel
112 Local nCustPanVer      := oMainPanel:nHeight/15       // Posicao vertical de oCustomerPanel
113 /*/
114     Objetos
115 /*/
116
117 oLblCab:= TSay():New(000,oMainPanel:nWidth * nSay1HorPerc,{||STR0001},oMainPanel,,,,,T,,,,) //"Recarga de Cartão Fidelidade"
118
119 oLblCab:SetCSS( POSCSS(CSS_BREADCUMB) )
120
121 /* Label e Get: Numero do Cartao */
122 oLblNumCart := TSay():New(nSay2VerPerc,nSay2HorPerc, {||STR0002}, oMainPanel,,,,,T,,,,) //'Num. Cartão'
123 oGetNumCart := TGet():New(nVerNumCart,nHorNumCart,{|u| If(PCount())>0,cGetNumCart:=u,cGetNumCart)},oMainPanel,,nAltGets,"@!",,,,,,T,,,,,"cGetNumCart")
124
125 oLblNumCart:SetCSS( POSCSS(CSS_LABEL_FOCAL) )
126 oGetNumCart:SetCSS( POSCSS(CSS_GET_NORMAL) )
127
128 /* Label e Get: Valor do Saldo */
129 oLblSaldo := TSay():New(nSay3VerPerc, nSay3HorPerc, {||STR0003}, oMainPanel,, ,,,,T,,,,) //'Valor do Saldo'
130 oGetSaldo := TGet():New(nGetVerPerc,nGetHorPerc,{|u| If(PCount())>0,cGetSaldo:=u,cGetSaldo)},oMainPanel,,nAltGets,"@E 99,999,999.99",,,,,,T,,,,,"cGetSaldo")
131
132 oLblSaldo:SetCSS( POSCSS(CSS_LABEL_FOCAL) )
133 oGetSaldo:SetCSS( POSCSS(CSS_GET_NORMAL) )
134
135 /* Label e Get: Validade do Saldo */
136 oLblDtValid := TSay():New(nSay4VerPerc, nSay4HorPerc, {||STR0004}, oMainPanel,, ,,,,T,,,,) //'Validade do Saldo'
137 oGetDtValid := TGet():New(nVerDtVld,nHorDtVld,{|u| If(PCount())>0,cGetDtValid:=u,cGetDtValid)},oMainPanel,,nAltGets,,,,,,T,,,,,"cGetDtValid")
138
139 oLblDtValid:SetCSS( POSCSS(CSS_LABEL_FOCAL) )
140 oGetDtValid:SetCSS( POSCSS(CSS_GET_NORMAL) )
141
142
143 /* Botoes Ok e Cancelar */
144 oBtnOk := TButton():New( nBtnOkVer,nBtnOkHor,STR0005,oMainPanel,{ ||STIConfSCRecharge(cGetNumCart,cGetSaldo,cGetDtValid,cItemCode) }, ;
145                          nBtnOkLarg,nBtnOkAlt,,,,T.) //'Confirmar'
146
147 oBtnCa := TButton():New( nBtnCaVer,nBtnCaHor,STR0006,oMainPanel,{ || STIExchangePanel( { || STIPanItemRegister() } ) }, ;
148                          nBtnCaLarg,nBtnCaAlt,,,,T.) //'Cancelar'
149
150 oBtnOk:SetCSS(POSCSS(CSS_BTN_FOCAL))
151 oBtnCa:SetCSS(POSCSS(CSS_BTN_FOCAL))
152
153 oGetNumCart:SetFocus()
154
155 Return oMainPanel
156

```

```
157 //-----
158 /*{Protheus.doc} STIConfSCRecharge
159
160
161 @return oMainPanel - Objeto contendo o painel principal de selecao de clientes.
162 */
163 //-----
164
165 Static Function STIConfSCRecharge(cGetNumCart,cGetSaldo,cGetDtValid,cItemCode)
166 Local lRet      := .F.
167 Local cCliCode  := STDGPBasket("SL1","L1_CLIENTE")
168 Local cCliLoja  := STDGPBasket("SL1","L1_LOJA")
169 Local cCliType  := STDGPBasket("SL1","L1_TIPOCLI")
170
171 lRet := STWUpdShopCardFunds(cGetNumCart,cGetSaldo,cGetDtValid)
172
173 If lRet
174     STWItemReg( 1, cItemCode, cCliCode, cCliLoja, 1, , , , cCliType, , cGetSaldo)
175     STICallPayment()
176 EndIf
177
178 Return lRet
```